5

密码学基础

MPC 与密码学之间的关系是显然的，两者深度共生且相互促进。MPC 本身就是现代密码学的一个核心分支与应用目标之一，其目标——使多个互不信任的参与方能够协同计算一个函数而各自输入保持私密——直接源于密码学对隐私与安全的根本追求。为实现这一目标，MPC 协议的设计、构建与安全性证明完全依赖于密码学提供的基础原语（如加密、数字签名、承诺、零知识证明等）和理论框架（如计算复杂性、模拟安全等）。同时，MPC 的研究也极大地反哺并推动了密码学本身的发展，它不仅是多种密码技术（如同态加密、混淆电路、秘密分享等）的集成与创新应用平台，其安全模型和协议范式也对密码学理论提出了新的挑战并拓展了其边界。因此，MPC 既是密码学思想的集大成体现，也是驱动密码学前沿发展的重要引擎。

作为一门研究信息及信息系统安全的学科，密码学起源于保密通信技术。一般将密码学分为密码编码学和密码分析学，如图5–1所示。而按照《深入浅出密码学–常用加密技术原理与应用》（Christof Paar 和 Jan Pelzl 著）一书中的观点，密码学中最常用的术语是密码编码学（cryptology），密码编码学又可进一步分为密码使用学（cryptography）和密码分析学，如图5–2所示。

密码编码学研究如何对信息进行编码以实现信息通信安全，旨在保持明文（或者是密钥）的秘密性并防止窃听者[1]知晓。这里假设窃听者完全能够截获发送方（Sender，S）和接收方（Receiver，R）之间的通信。从安全目标来看，密码编码学分为保密体制和认证体制。保密体制主要保证信息不被攻击者窃取，具体由对称加密和非对称加密体制来实现。认证体制又分为消息认证和实体认证，消息认证主要保证信息源不被假冒，通常

[1]窃听者也称为敌手（Adversary，$\mathcal{A}$）、攻击者、截取者、入侵者、敌人等。

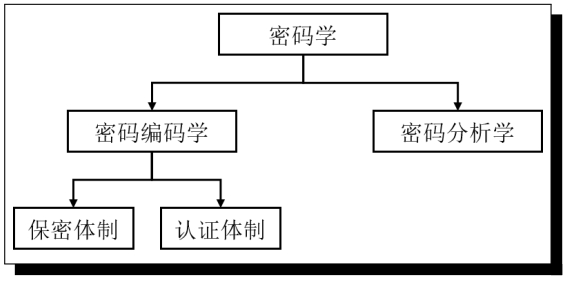

图 5–1 密码学体系

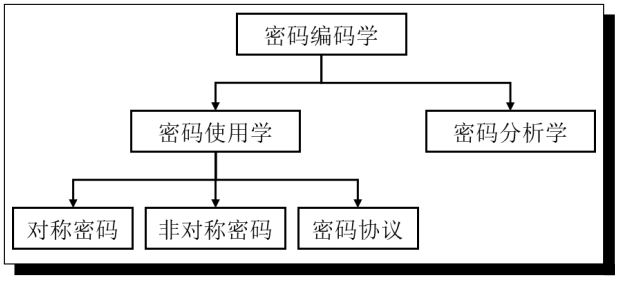

图 5–2 密码学领域

由消息认证体制和数字签名实现，实体认证主要保障信息发送方和接收方的身份真实性，通常由身份鉴别协议来实现。

密码分析学研究如何破解或攻击受保护的信息，旨在在不知道密钥的情况下恢复出明文。成功的密码分析可以恢复消息的明文或者密钥，与此同时，密码分析也可以验证出密码体制的弱点。在传统密码时期，加密方法是不公开的，但在现代密码时期，密码体制的设计和使用必须遵从科克霍夫（Auguste Kerckhoff，荷兰语言学家和密码学家）原则（Kerckhoffs' Principle），其内容是：即使除了密钥之外，关于该系统的一切都被公开，该密码系统也应该是安全的[1]。

密码学的内容非常庞杂，本书中着重介绍 MPC 中常用的密码学知识，对密码学感兴趣的读者请阅读其它密码学相关著作。

5.1 密码学中的攻击模型

从本节开始介绍密码学中的理论和方法。网络安全从业人员经常提到的一句话是“不知攻，焉知守”，兵法上也讲究“知己知彼，百战百胜”。在介绍具体加密算法和签名算法等安全技术之前，首先介绍一下密码学中的攻击方式。

一般来说，若密钥通过非密码分析方法被获取到，称为“泄露”（compromise）；对密码学算法（或密码系统）进行分析的尝试，称为“攻击”（attack）。长期以来，密码学家（cryptographer）需要密码分析者（cryptanalyst）帮助他们分析所构造的密码学原语的安全性，因此，密码学家常常是密码分析者，密码分析者也可能是密码学家。

根据科克霍夫原则，密码学的安全性应取决于对密钥的安全隐藏，密钥未泄露，密

[1]科克霍夫原则可以用英文表述为“The cipher method must not be required to be secret, and it must be able to fall into the hands of the enemy without inconvenience.”（密码方法本身不需要一定是保密的，而且它必须能被敌人轻易得到），或者“The security of a cryptosystem should depend solely on the secrecy of the key, and not on the secrecy of the algorithm or its implementation details.”（密码系统的安全性应该仅依赖于密钥的保密性，而不应依赖于算法的保密性或其实现细节）。

码系统就应该是相对安全的。但是，攻击者（这里指密码分析者）仍可以通过对密文和/或已知明文进行分析，获取到其它明文或加密密钥。根据攻击者所能获得的信息资源，可以将攻击方式分为六类：唯密文攻击、已知明文攻击、选择明文攻击、自适应选择明文攻击、选择密文攻击和选择密钥攻击。

5.1.1 唯密文攻击

唯密文攻击（Ciphertext-Only Attack，COA）中，攻击者（如前所述，攻击者通常也称为“敌手”，本书中不加区分地使用这两种说法）可通过窃听等方式获取到一些消息的密文，且这些消息都是用同一加密算法加密的。比如，攻击者截获了一封加密电报，但不知道原文内容，也不知道加密方式。攻击者的目的是恢复出尽可能多的明文，或者尽可能地获取到加密这些消息的密钥，以便可采用相同的密钥破解其它被加密的消息，如图5–3所示。

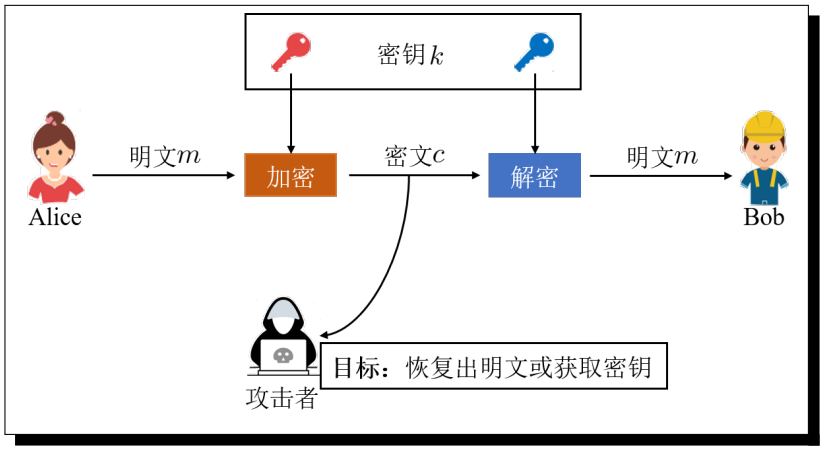

图 5–3 唯密文攻击示意

COA 是攻击者获知条件最少的攻击方式，如果一个密码系统经不起唯密文攻击，显然是不安全的。现代密码算法必须至少能抵抗 COA。

5.1.2 已知明文攻击

已知明文攻击（Known-Plaintext Attack，KPA）也称为 KPA 安全，是指攻击者除了得到消息密文，还可以通过某些方式获取到部分消息密文对应的明文的攻击方式。比如，攻击者截获了加密电报，并且通过间谍得知了其中某一段的原文内容。具体实例，比如在二战期间，盟军通过获取部分已知的日德密文/明文对，成功破译了“紫密”等密码机。攻击者的目标是根据被加密的信息推导出用来加密的密钥，或者等价的，即使没能

够找到密钥，也会试图推导出一种算法，这种算法可以对用相同密钥加密的任何新消息进行解密。KPA 的过程如图5–4所示。

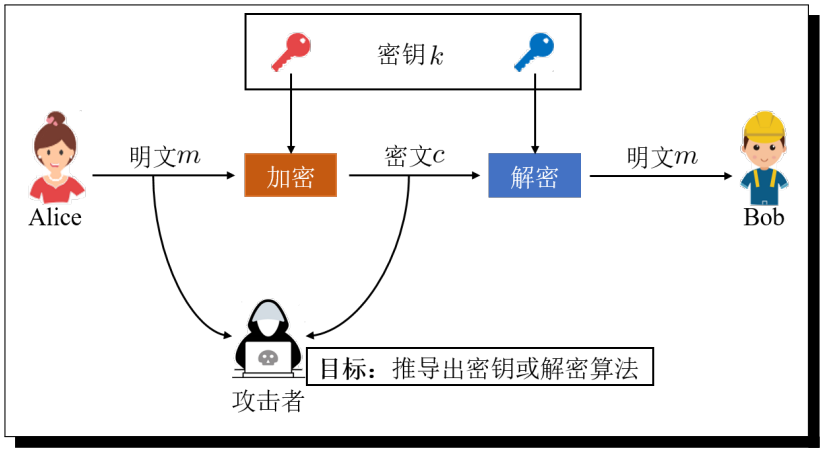

图 5–4 已知明文攻击示意

KPA 成功的关键在于获取明文。这可能通过推测（如标准文件头、常见问候语）、社会工程、系统漏洞或协议设计缺陷实现。现代分组密码（如 AES）通常要求能抵抗 KPA。对于 AES、RSA（正确使用）等现代强加密算法，KPA 本身并不构成威胁。这些算法在设计上就能抵抗此类攻击。问题通常出在实现错误、模式误用（如 ECB 模式）、密钥管理不当或算法本身已过时上。

5.1.3 选择明文攻击

选择明文攻击（Chosen-Plaintext Attack，CPA）也称为 CPA 安全，是指攻击者不仅可以获取到一些密文-明文消息对，而且可以主动选择任意明文，并获取其对应的密文的攻击方式，如图5–5所示。对于 CPA，可使用下面的例子加以说明：假如攻击者获取到了一些密文-明文对，且其知道：（1）密文是使用相同的确定性加密算法和密钥生成的，（2）明文是什么类型的，比如 11 位的电话号码。这样一来，他就可以用少得多的代价，尝试构造电话号码，并生成所有的明文-密文组合，从而可以在下一次拿到密文的时候，通过对比找出相应的明文。再比如，攻击者伪装成合法用户，向加密机输入其想要的任何内容，并记录输出结果。

在密码学理论研究领域的论文中，“安全模型”通常与 game（可译为“博弈”）、experiment、image、procedure 等概念相对应。为与大多数论文的表述保持一致，本书中也使用 game 这个词来指代安全模型。在安全性 game 中，这通常是通过将挑战者提供的加密预言机开放给攻击者来实现的，详见5.2.5节。

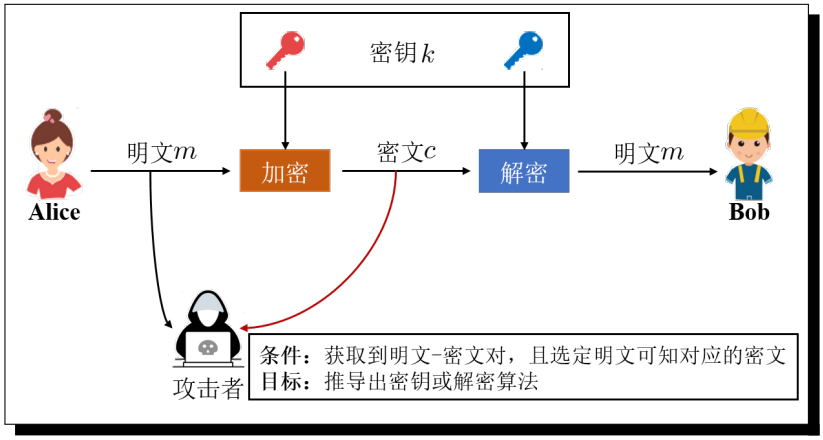

图 5–5 选择明文攻击示意

显然，这种攻击方式比 KPA 更加有效，因为攻击者能选择特定的明文块进行加密，那些消息块可能产生更多关于密钥的信息。攻击者的任务就是推导出用来加密消息的密钥或推导出一个算法，此算法可以对用相同密钥加密的任何新消息进行解密。CPA 是现代密码学（特别是公钥密码）的标准安全模型。如果一个算法能抵抗 CPA，通常认为它是安全的。

5.1.4 自适应选择明文攻击

自适应选择明文攻击（Adaptive-chosen-plaintext Attack，A-CPA）是 CPA 的一种特殊情况，是比 CPA 具有更多权限的攻击方式。因此，A-CPA 是 CPA 的加强版。

A-CPA 与普通的 CPA 的区别是：

（1）在普通的 CPA 中，攻击者虽然可以选择被加密的明文并获得相应的密文，但他们无法根据之前的加密结果对后续的选择进行修正。而在 A-CPA 中，密码分析者具有这种修正选择的能力，这使得攻击更具灵活性和深度。

（2）在 CPA 中，攻击者可以选择一大块被加密的明文。而在 A-CPA 中，攻击者可以进行“交互式”攻击，根据反馈调整策略。比如，攻击者可以选择较小的明文块，然后再基于第一块的结果选择另一个明文块，以此类推。

A-CPA 通常用于对加密系统的安全性进行评估和测试。通过模拟攻击者的行为，可以揭示加密系统中可能存在的漏洞和弱点。为了抵御自适应选择明文攻击，加密系统需要设计得足够强大和灵活。例如，可以采用更复杂的加密算法、增加密钥的长度和复杂度、以及实施更严格的访问控制等措施来提高加密系统的安全性。CPA 和 A-CPA 在公钥密码学中通常是等价的，因为公钥是公开的，攻击者可以无限次加密。

5.1.5 选择密文攻击

选择密文攻击（Chosen Ciphertext Attack，CCA）也称为 CCA 安全，是指攻击者能够制造或选择不同的密文，并能得到对应的解密后的明文，是一种比 KPA 更强的攻击方式，如图5–6所示。比如，攻击者获得了某个解密机，或者攻击者是保密系统内部的员工，可以有一定的权限获得某些密文的原文。再比如，攻击者截获了密文，对其进行微小修改后发送给接收者，然后观察接收者的反应或错误信息。在安全性 game 中，这通常是通过将挑战者提供的解密预言机开放给攻击者来实现的，攻击者可以选择一些消息密文，然后通过解密预言机获得这些密文对应的明文，详见5.2.5节。

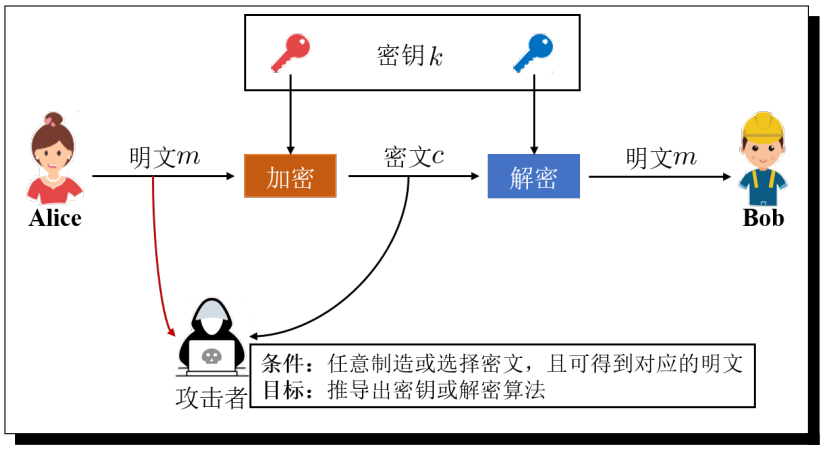

图 5–6 选择密文攻击示意

如果一个密码系统能够抵抗 CCA，必然能够抵抗 COA 和 KPA。有时，CPA 和 CCA 合并在一起，称为选择文本攻击（Chosen-text Attack，CTA）。一个密码系统只有能够抵抗 CPA 和 CCA，才能算作是安全的。

5.1.6 选择密钥攻击

选择密钥攻击（Chosen-key Attack，CKA）是指攻击者可以部分控制或选择密钥，或者能够获取使用不同密钥加密的密文，通常用于分析哈希函数或密钥派生函数的弱点。CKA 并不表示攻击者真的能够选择密钥，只表示攻击者具有不同的密钥间关系的有关知识。比如，攻击者可以尝试使用与目标密钥有某种关联的密钥进行攻击，观察其相关性。

在 CKA 中，密码分析者可以利用已知的密钥关系来分析加密算法中的漏洞，从而获取更多的密钥信息。例如，如果一个加密算法是基于某个数学难题的，那么密码分析者可以通过解决这个数学难题来获取更多的密钥信息。另外，CKA 还可以通过尝试不

同的密钥来破解加密算法。例如，如果一个加密算法使用了某个哈希函数，那么密码分析者可以尝试不同的输入来生成哈希值，然后通过比较生成的哈希值和已知的哈希值来破解加密算法。

这种方法在类似于差分密码分析的相关密钥分析中有所应用，主要用于分析算法的抗碰撞能力。但由于此方法比较晦涩和奇特，且获得大量的密钥关系或者尝试大量的密钥需要花费大量的时间和计算资源，因此并不是很现实的攻击手段，在实际应用中很少见。

5.1.7 攻击模型对比

上述密码学攻击模型的对比如表5–1所示。

表 5–1 密码学攻击模型对比

攻击模型	攻击者拥有的能力	安全强度要求
COA	仅能获取到密文	现代密码算法的基本要求（最低防御等级）
KPA	能获取到部分“明文-密文”对	基础要求，大多数分组密码（如 AES）需满足。
CPA	可以选择任意明文，并获取其对应的密文	**标准安全要求**（如 IND-CPA），公钥加密的基准
A-CPA	可自适应地根据之前结果选择下一个明文	通常与 CPA 等价，是公钥密码学的标准模型
CCA	可以选择任意密文，并获取其解密后的明文	**高级安全要求**（如 IND-CCA2），确保密文不可篡改
CKA	可以部分控制或选择密钥	特定分析场景（如哈希函数碰撞攻击）的要求

表5–1中，IND-CPA（Indistinguishability under CPA）是现代加密算法（如 AES-GCM, RSA-OAEP）的黄金标准，意味着攻击者即使能任意加密，也无法区分两个明文的密文；IND-CCA 是更强的安全模型，能防止密文被篡改，其中的 IND-CCA2（Indistinguishability under Adaptive Chosen-Ciphertext Attack）是密码学中衡量加密方案安全性的最高标准之

一。它描述了一个攻击模型，要求加密算法即使在攻击者能够自适应地选择密文并获取解密结果的情况下，依然保持安全。

5.2 密码学中的安全性

了解了密码学中的攻击模型后，再来看密码学中的安全性。密码学用于保证信息的安全性，这往往是借助数学上可证明困难性的问题来实现的。首先，应该明确密码学中所说的“安全”是什么，或者说密码学要实现的安全性目标是什么。

5.2.1 密码学中的安全性分类

安全性是相对而言的，没有绝对的安全性。从抵抗攻击（破解所需时间和所需资源）的角度来看，密码学中的安全性可分为四种（或四个安全层次）[1]：

- 信息论安全性（理想安全性）：密文没有给出关于明文的任何信息，因此密码分析者是无从通过密文来获得明文的信息的。
- 无条件安全性（unconditionally secure）：即使采用最好的可能算法，拥有无限的计算资源，也不可能攻破密文。
- 计算安全性（computational secrecy）：即使采用最好的可能算法，使用“实际的”计算资源来攻破密文也是困难的。
- 实际安全性：采用最著名的算法，使用“实际的”计算资源来攻破密文也是困难的。

通常希望一个密码体制尽可能安全，但一般来说，我们不知道针对一个给定任务的“最好的可能算法”是什么，或者不知道最好的可能算法的复杂性是怎样的，因此，最好的可能算法往往是未知的或者是有条件的。虽然“实际安全性”这一说法非正式且不够准确，但考虑到实际面临的情形和安全需求，**追求实际安全性才是永远的真理**。由于信息论安全性和计算安全性是最常提到的概念，本节重点阐述这两种安全性。

5.2.1.1 信息论安全性

当攻击者拥有无限计算资源也无法破解密码系统时，称该密码系统满足信息论安全性（information theoretic security），也称为理想安全性。以加密为例，可以定义一种完美安全性（perfect security 或 perfect secrecy）：

定义 5.1 给定一个随机密钥 k，用其加密明文 m 得到密文 c，$\mathrm{Enc}(k,m)=c$，如果 $P(M=m|C=c)=P(M=m)$，即在有密文和没有密文的情况下，猜出明文 m 的概率是

一样大的，也就是说，攻击者通过密文 c 不能得到任何有意义的信息，此时可以认为达到了完美安全性，也即信息论安全性。

信息论安全性的最简单的例子是一次一密的加密算法，复杂些的例子包括承诺算法等。

总之，研究已表明，要达到信息论安全性，至少需要满足两个条件：

- 密钥的长度一定要大于或等于明文的长度。
- 必须采用一次一密（通常采用一次性密码本 one-time-pad）的加密方式。

显然，在现实应用中，这种无条件安全性可能是不切实际的。为此，人们只得退而求其次，希望达到计算安全性。

5.2.1.2 计算安全性

如果一个密码算法（或称为密码系统、密码学系统）在给定的时间（比如概率多项式时间 PPT）和资源内无法被攻破，则称其为计算安全的，这与攻击者的能力、目标等条件密切相关。

对于信息论安全性和计算安全性，可以给出一种形式化定义。假设对于某个密码方案，攻击者最多能执行 t 个操作，攻击成功的概率不高于 P，则该方案是 (t, P)-安全的。以攻击密钥长度为 λ 比特的对称密码算法为例，当 $t = 1$ 时，成功的概率为 $P = 1/2^{\lambda}$；当 $t = 2^{\lambda}$ 时，成功的概率为 $P = 1$，即其中一定有一次攻击是成功的。此时给出的是破解该密码方案难度的下限，这意味着，没有攻击者能够在执行小于 t 次操作的情况下，获得概率为 P 的成功率，但是这并不是说攻击者执行 t 次操作就恰好可以达到 P 的成功率，也不说明需要具体多少次才能攻击成功。

为便于描述，可以对该形式化定义进行简化，当成功攻击某密码方案至少需要 t 次操作时，称其为 t-安全，这里实际上是假设成功率 P 接近于 1。

另外，也可以通过比特数来表示安全性，比如“n 比特安全”说明攻破某密码方案至少需要执行 2^n 次操作，显然，这一表示方法与需要的操作数之间是对数关系，即：将需要的操作数取对数，就得到了定义安全性的比特数，可以用该比特数确定该密码方案的安全强度。

虽然比特安全性在比较不同密码的安全强度时很有用，但是其没有提供足够的关于攻击成本的信息，因此，使用该表示方法时需要进一步明确其它因素，比如破解过程是否可并行、内存大小、是否可预计算（比如使用彩虹表破解哈希值）、攻击目标的数量等。

5.2.1.3 实际安全性

对于实际应用中的密码系统而言，由于至少存在一种破解方法，即暴力攻击法，因此都不能满足无条件安全性，只能提供计算安全性。密码系统要达到实际安全性，需要满足以下准则：

- 破译该密码系统的实际计算量（包括计算时间或费用）十分巨大，以致于在实际上是无法实现的。
- 破译该密码系统所需要的计算时间超过被加密信息有用的生命周期。
- 破译该密码系统的费用超过被加密信息本身的价值。

如果一个密码系统能够满足以上准则之一，就可以认为是满足实际安全性的。

5.2.2 可证明安全性

为了评估某密码算法、方案、协议的安全性，一般会使用数学证明。这在密码学中被称为可证明安全性，它可以证明某个密码方案至少和解决另一个已知的困难性问题是同等困难的，只要困难性问题仍然存在，那么该方案就是安全的，这种证明方式被称为“规约”，其来自复杂性理论。但这种评估方法存在的问题是它只说明了这个密码方法的安全性与某个困难问题相关，没有完全证明问题本身的安全性，并给出它们之间的等价性证明。

根据其所使用的困难性问题的类型，可以将可证明安全性分为两种: 与数学问题相关（破解这类方案至少与解决一些数学难题一样困难）、与密码问题相关（这类方案是与另一个密码方案比较，并证明只有破解第一个密码方案时，才能破解第二个密码方案）。

5.2.3 带有中止的安全（中止安全性）

带有中止的安全（secure-with-abort）也称之为中止安全性，是 MPC 和密码学协议中一种较弱但实际可行的安全保证。它允许恶意攻击者（腐化方）在协议执行后，阻止部分或全部诚实参与方获得正确的计算结果，但确保诚实方要么获得正确输出，要么什么也得不到（即中止），而不会得到错误或误导性的输出。中止安全性是安全协议或安全假设的一种属性，旨在在通信双方之间建立安全通信，并在一方决定中止协议时能够确保另一方能够得知该决定，并且在这个过程中仍然保持安全性。

这个概念通常出现在分布式系统中，系统中的各个节点需要相互通信并协调行动。如果一个节点决定离开系统（即“中止”），它需要向其他节点发送一个消息，让它们知道它已经离开了。这个消息应该是在安全的环境下发送的，也就是说，如果这个消息被

拦截了，发送者应该能够知道这个情况，并且可以采取适当的行动防止被拦截者假冒。

在具有这种安全属性的协议中，通信双方需要通过安全密钥交换来建立一个安全通信通道。一旦建立了安全通信通道，双方就可以通过该通道进行安全通信，包括发送和接收加密消息。当一方决定中止协议时，它需要向另一方发送一个中止消息。这个中止消息可以是明文消息或加密消息，以确保另一方能够得知该决定。一旦收到中止消息，另一方可以采取适当的措施来应对该决定，例如关闭通信通道或寻找新的通信通道。

带有中止的协议的一个重要特点是，它确保了一方在决定中止协议时可以及时通知另一方，并且另一方可以采取适当的措施来应对该决定。这有助于保护通信双方的安全性，并避免由于一方的不透明中止而导致另一方遭受潜在的安全威胁。

在 MPC 中，有时会出现某个参与方中止计算的情况，比如该参与方被敌手腐化，想要通过中止计算来破坏 MPC 协议执行过程。为了应对这种情况，可以使用带有中止的 MPC 协议来保证其他参与方的利益不受损害，同时保证任何一方都无法篡改最终的计算结果。因此，带有中止的 MPC 是一种在保证数据隐私和计算安全性的前提下，应对某个参与方中止计算的计算模型。但是，带有中止的安全并不是理想的安全属性，因为本质上来说，该安全属性可使得敌手获得运行拒绝服务（Denial-of-Service attacks，DoS）攻击的能力。

从腐化（corruption）的角度来说，带有中止的安全是一种实用性妥协（compromise）。对于许多计算问题，在恶意方可能占多数（不诚实大多数）的情况下，要实现“保证输出送达”（Guaranteed Output Delivery，GOD）或“公平性”（fairness）在理论上不可能或会导致协议效率极低。带有中止的安全放松了对输出送达的绝对要求，从而允许设计出更高效、更可行的协议，同时仍能保障：

- 隐私性：恶意方无法获取诚实方的私有输入信息。
- 正确性：诚实方如果获得输出，则该输出一定是正确的。

总结来说，带有中止的安全是密码学协议中一个关键的安全模型，它平衡了安全性与效率，使得在存在主动恶意攻击者的现实环境中，构建可行的安全计算协议成为可能。

5.2.4 密码学常用安全模型

为了衡量一个算法、协议或密码系统的安全性，定义安全模型是可行的方法。在密码学中，安全模型是指“安全性假设”。在对密码学方案进行安全性证明时，常用到的两个安全模型是标准模型（Standard Model，SM）和随机预言机模型（Random Oracle Model，ROM）。

5.2.4.1 标准模型

在标准模型下，敌手只受时间和计算能力的约束，而没有其他限制。如果在此条件下，可以将密码学方案归约到困难性问题上，则称为该归约是基于标准模型的，也称方案具有在标准模型下的可证明安全性。

然而，实际情况是很多方案在标准模型下建立安全性归约是比较困难的，也就是难以证明方案在标准模型下的安全性。因此，为了降低证明的难度，往往在安全性归约过程中加入其他的假设条件。

5.2.4.2 随机预言机模型

随机预言机模型是在安全模型的基础上加入了针对哈希函数的随机预言机假设。这种假设下进行安全归约得到的安全性也称为随机预言机模型下的可证明安全性。随机预言机模型是从哈希函数抽象出来的一种模型，是在可证明安全中被广泛使用的证明方法。

随机预言机可以理解为一种完美的、理想的哈希函数，满足如下性质：

- 一致性：对输入数据非常敏感，对于相同的输入，其输出必然相同；
- 可计算性：计算过程可以在多项式时间内完成；
- 均匀分布：预言机的输出在取值空间内均匀分布，且无碰撞。

在随机预言机模型中，假定敌手不能利用哈希函数的弱点来攻击密码学方案（因为随机预言机是理想的哈希函数，它没有传统哈希函数的弱点可被利用），否则该方案不安全。换句话说，将方案中的实际哈希函数换成随机预言机后，如果敌手仍然可以成功攻击，则证明该方案不安全。

随机预言机虽然广泛应用于密码学方案的证明，为可证明安全提供了很大的方便，但人们对随机预言机模型下的安全性证明的有效性仍存在争议。在现实中，并不存在这样一种完美的哈希函数，因此在随机预言机模型下安全的一些方案，在使用真实的哈希函数之后，就不安全了。因为随机预言机模型下的安全性证明是具有很强假设性的，即敌手不能够利用哈希函数的弱点，这种假设过于理想化。在现实中，哈希函数是确定的，其输出并不能保证是完全随机且均匀分布的。所以，随机预言机模型下证明安全的一些方案，在用真实的哈希函数代替随机预言机后，就不再安全了。尽管如此，基于随机预言机模型的安全性证明除了哈希函数外的环节都可以达到安全要求，目前大多数的可证明安全方案也是基于随机预言机模型的。因此，随机预言机模型仍被认为是可证明安全中最成功的应用。

5.2.5 不可区分性安全模型

数据的机密性、完整性和实体的抗否认性是一个信息安全系统所要保障的主要目标，也是一个密码算法所应满足的基本指标。纵观整个安全性定义的发展，数据机密性是人们一直以来的核心诉求。具体来说，如果声称一个加密算法实现了数据机密性，那么就需要证明这个算法输出的密文与等长的随机字符串看起来是不可区分的，由此敌手就不能从密文中获得任何有关明文和加密密钥的信息。从香农的完美安全性到语义安全性（semantic security），人们一直希望一个加密算法的输出应足够随机，以使得其不会泄露关于明文和密钥的任何信息。为了描述算法的机密性，人们提出了“不可区分性”（INDistinguishability，IND）这一概念。然而，一个算法达成不可区分性时所处于的安全模型可能是不同的，因此，在讨论一个加密算法的不可区分性的强度时，需要明确其面向的安全模型。

除了前述的标准模型和随机预言机模型，在密码学中，语义安全、IND-CPA 与 IND-CCA 也是常用的不可区分性安全模型。CPA 指的是选择明文攻击（Chosen Plaintext Attack），CCA 指的是选择密文攻击（Chosen Ciphertext Attack），详见5.1节。为便于理解，可对这三种安全模型作如下直观的描述（非严格定义）：

- **语义安全**：单纯考察密文，不能得到与明文相关的信息，即：密文不会泄露明文；
- **IND-CPA（选择明文攻击下的不可区分性）安全**：密文不会泄露明文的统计特征。举例来说，对字母表里的所有字母进行轮换，这种加密方案是语义安全的，因为单看一个字母显然不知道它原本对应的字母；但该方案不是 IND-CPA 安全的，因为给定一个加密文本，根据统计原理，出现最多的那个字母最有可能对应的字母是“E”，从而有可能泄露明文的统计特征；
- **IND-CCA（选择密文攻击下的不可区分性）安全**：无法伪造可正常解密的密文。以如下场景为例：某公司发给财务人员一个加密邮件，里面包含了某员工这个月的工资信息，虽然无法解密，但是该员工也许可以在密文上做手脚，将原本 3000 元的工资改成 10000 元（即伪造可正常解密的密文），从而让财务解密完发给该员工更多工资，而 IND-CCA 安全的方案可阻止这种情况的发生。

一般来说，这三种安全模型之间的关系为：仅语义安全的方案，可以通过添加随机数，使其达到 IND-CPA 安全；仅 IND-CPA 安全的方案，可以通过添加密文完整性验证机制，使其达到 IND-CCA 安全。另外，在公钥密码体制中，由于加密算法使用的密钥是公开的，所有人都可以对任何消息进行正确的加密，因此语义安全与 IND-CPA 安全是等价的；而在对称密码体制中，语义安全和 IND-CPA 安全是两种不同的安全级别。

严格来说，语义安全、IND-CPA 安全与 IND-CCA 安全的定义就是通过挑战者（Challenger，$\mathcal{C}$）和敌手（$\mathcal{A}$）之间的攻击博弈（attack game）来定义的，简述如下。

5.2.5.1 语义安全模型

敌手选择两个（等长度的）明文，挑战者随机加密其中一个，并将密文发送给敌手，让敌手猜测对应的明文，若敌手猜中的概率不大于 1/2，则称该方案满足语义安全性。

5.2.5.2 IND-CPA 安全模型

IND-CPA 所表达的含义为：在敌手能自行选择明文，并查询对应的密文这一场景（安全性设定、模型）下，加密算法是否还能实现密文不可区分。在这一安全模型中，敌手可以查询任意一条明文消息对应的密文，这需要将算法的加密功能开放给敌手，即敌手拥有“访问加密预言机”（access to the encryption oracle）的能力，以保证敌手可成功实施选择明文攻击。IND-CPA game 的示意图如图5–7所示。

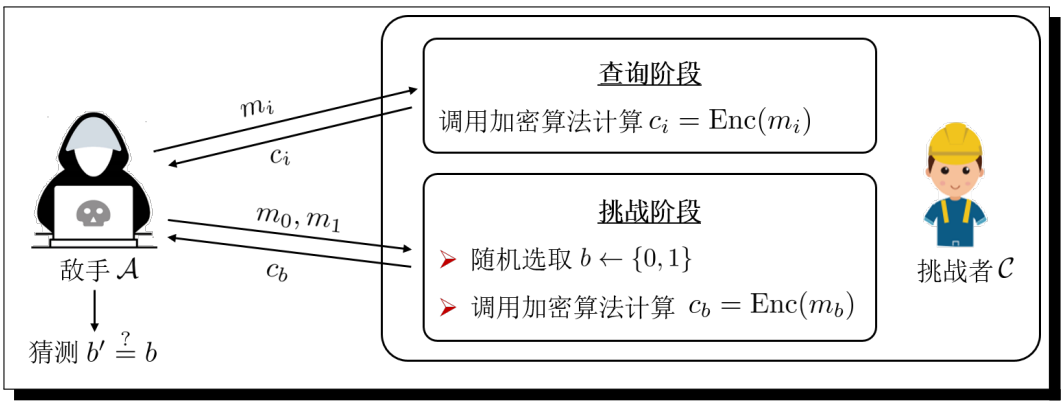

图 5–7 IND-CPA game 示意图

该 game 中，敌手 $\mathcal{A}$ 是 PPT 敌手，挑战者 C 能自由调用加密预言机 E。结合图5–7，IND-CPA game 的过程如下：

（1）挑战者 C 随机选取 $k \in_R \{0,1\}^n$，作为加密算法的密钥；
（2）敌手 $\mathcal{A}$ 向 C 提交明文 m_i，C 计算 m_i 对应的密文 c_i；
（3）重复步骤（2）q 次（敌手查询 q 次不同消息的密文）；
（4）敌手 $\mathcal{A}$ 向 C 提交明文 m_0 和 m_1；
（5）C 随机选择 $b \leftarrow \{0,1\}$（比如以抛硬币（flip a coin）的方式），然后加密并返回挑战密文 c_b；
（6）敌手根据 c_b 输出猜测比特 b'（即敌手猜测 C 加密的是 m_0 还是 m_1）。如果 $b' = b$，敌手 $\mathcal{A}$ 获胜。

由此可以看出，IND-CPA 安全实际要表达的含义是：**如果密文足够随机的话，密文是不会泄露任何关于明文的信息的**。对于 IND-CPA 的敌手 $\mathcal{A}$ 来说，其赢得该 game 的优势（advantage）概率为 $\mathrm{Adv}_E^{\mathrm{IND-CPA}}(\mathcal{A}) = |\Pr[\mathcal{A}\ wins] - 1/2|$，其中，$\mathrm{Adv}_E^{\mathrm{IND-CPA}}(\mathcal{A})$ 称为 $\mathcal{A}$ 面对 E 时在 IND-CPA game 中获胜的优势概率。如果该概率是可忽略的（negligible），则称该方案是 IND-CPA 安全的。一个安全的加密算法至少要能达到 IND-CPA 的安全性。而在上面这个模型中，只关心了算法的加密功能和密文本身的随机性，而实际中的敌手也有可能访问解密算法来实现攻击。因此，需要一种更强的安全模型来保护解密算法和密文。

5.2.5.3 IND-CCA 安全模型

IND-CCA 安全保证了密文的不可伪造性，即：给定一个密文，敌手不能知道该密文对应的明文。在这个安全模型中，敌手不但可使用加密问询（即访问 encryption oracle，加密预言机），也可以使用解密问询（即访问 decryption oracle，解密预言机），即敌手可以自行构造密文并提交给 C，C 执行解密算法并返回解密结果，从而保证敌手可实施选择密文攻击，而敌手的最终目标与 IND-CPA game 中相同。在公钥密码体制中，IND-CCA 的 find-then-guess game 如图5–8所示。

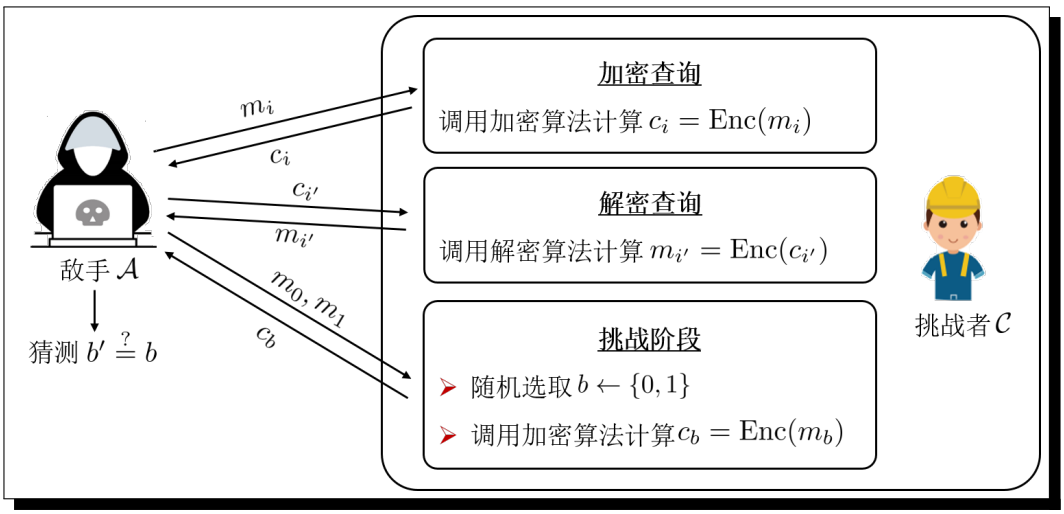

图 5–8 IND-CCA game 示意图

该 game 中，敌手 $\mathcal{A}$ 仍是 PPT 敌手，挑战者 C 能自由调用加密预言机 E 和解密预言机 D。结合图5–8，IND-CCA Game 的步骤如下：

（1）挑战者 C 随机选取 $k \in_R \{0,1\}^n$，作为加解密算法的密钥；
（2）敌手 $\mathcal{A}$ 向 C 提交任意明文 m_i，C 计算 m_i 对应的密文 c_i；
（3）敌手 $\mathcal{A}$ 向 C 提交任意密文 $c_{i'}$，C 计算 $c_{i'}$ 对应的明文 $m_{i'}$；
（4）重复步骤（2）q 次，步骤（3）q' 次（敌手查询 q 次不同消息的密文，查询 q' 次不同密文的解密结果）；
（5）敌手 $\mathcal{A}$ 向 C 提交明文 m_0 和 m_1；
（6）C 随机选择 $b \leftarrow \{0,1\}$（比如以抛硬币（flip a coin）的方式），然后加密并返回挑战密文 c_b；
（7）敌手 $\mathcal{A}$ 根据 c_b 输出猜测比特 b'（即敌手猜测 C 加密的是 m_0 还是 m_1）。如果 $b' = b$，敌手 $\mathcal{A}$ 获胜，即敌手 $\mathcal{A}$ 猜出了密文 c 对应的明文 m_b。

此时，敌手 $\mathcal{A}$ 的优势概率可表示为 $\mathrm{Adv}_{E,D}^{\mathrm{IND-CCA}}(\mathcal{A}) = |\Pr[\mathcal{A}\ wins] - 1/2|$，如果该概率是可忽略的，则称该方案是 IND-CCA 安全的。

在此模型中，由于敌手没有密钥，因此其很难生成合法的密文。因此，$\mathcal{A}$ 每次提交的密文大概率会解密出一些无意义的结果，但开放解密预言机的目的在于：让敌手能在拥有密文并可篡改密文的条件下，依然做到对明文的保护。因为一个加密算法如果仅能支持 CPA 安全性，那么中间人敌手可能会截获密文并对其进行有针对性的篡改，从而使这种篡改也被施加到解密后的明文中，敌手进而能通过篡改后的明文来攻破加密结果（密文）的不可区分性。因此，满足 IND-CCA 安全可使得敌手无法通过修改密文来观测解密算法的行为。

另外，考虑到敌手在提交了明文 m_0 和 m_1 后，可能依然能进行若干次解密查询，以继续学习，这相当于将 C 提供的预言机永久开放给敌手了，此时对应的安全模型被称为 IND-CCA2，相应地，前述 IND-CCA game 也被称为 IND-CCA1。

5.3 密码学基于的困难性问题

有些密码学方法是利用数学上可证明非常困难的问题来保证安全性的，称为“可证明安全性”。现代密码学所基于的困难性问题主要包括大整数分解困难问题、离散对数问题、二次剩余问题，或者判定 n 阶剩余类问题（DCR）等。

5.3.1 大整数分解困难问题

大整数分解（Integer Factoring）困难问题是指：若已知两个整数 p 和 q，求其乘积 $p \cdot q = n$ 是容易的。但若已知两个整数的乘积 n，n 为大整数，求其是由哪两个整数 p

和 q 相乘得到的却是非常困难的。

大整数分解困难问题是著名的非对称加密算法 RSA 的安全性基础。但这一安全性问题正面临安全性挑战，一方面，数学家一直在研究质数分布的问题，企图找到这一问题的高效算法；另一方面，随着量子计算技术的发展，目前 Google、微软等已经构建出量子计算原型机。特别是，如果两者结合，将极大提升破解这一难题的可能性，比如，数学家 Shor 已于 1994 年提出了针对大整数分解难题的量子算法，即 Shor 算法。

5.3.2 离散对数困难性问题

离散对数困难性问题（Discrete Logarithm Problem，DLP）是指：给定一个大质数 p 和一个 p 阶有限循环群 $\mathbb{G}$，g 是群 $\mathbb{G}$ 上的一个生成元。若已知 $x \in \mathbb{Z}_p^*$，求解 $y = g^x \mod p$ 是容易的，但若已知 $y \in \mathbb{Z}_p^*$，求解 $y = g^x \mod p$ 中的 x 却是非常困难的，其困难性主要来源于模数运算。

离散对数困难性问题是 ElGamal 公钥加密算法、椭圆曲线加密算法（Elliptic Curve Cryptography，ECC，椭圆曲线密码学）、DSA 数字签名算法、ECDSA 签名算法、SM3 签名算法、基于身份的密码算法（IBE、IBS）、SM9 标识密码算法（Identity-Based Cryptography，IBC，基于标识的密码算法）等的安全性基础。

注解 5.1 SM3 是我国政府采用的一种哈希函数标准，由国家密码管理局于 2010 年 12 月 17 日发布，相关标准为 GM/T 0004-2012《SM3 密码杂凑算法》。该哈希算法是在 SHA-256 基础上进行改进而实现的，适用于商用密码应用中的数字签名和验证、消息认证码生成及验证、随机数生成等，其算法公开。

SM9 是我国政府采用的一种标识密码标准，也是我国首个全体系纳入 ISO/IEC 标准的非对称密码算法，由国家密码管理局于 2016 年 3 月 28 日发布，相关标准为 GM/T 0044-2016《SM9 标识密码算法》。在商用密码体系中，SM9 主要用于用户的身份认证。据称，SM9 的加密强度等同于 3072 位密钥的 RSA 加密算法。

与离散对数问题相关的一个概念是“离散对数假设”（Discrete Logarithm Assumption，DLA）：用 $P_{\mathcal{A}}^{DLP} = Pr[x \leftarrow \mathcal{A}(g, g^x)]$ 表示算法 $\mathcal{A}$ 能够成功解决 DLP 问题的概率，对于任意 PPT 算法 $\mathcal{A}$，$P_{\mathcal{A}}^{DLP}$ 是可忽略的。离散对数假设可以看作是离散对数问题的另一种表述方法。

5.3.3 二次剩余困难性问题

二次剩余困难性问题（The Quadratic Residuosity Problem）是指：如果存在一个算法 $\mathcal{A}$ 能以多项式时间 t，以概率 $P \geq \xi$ 寻找到模 $n = pq$ 的二次剩余，则存在算法 $\mathcal{A}^*$ 能够

以时间 $t+O(\log N)^{O(1)}$，以概率 $P \geq \frac{\xi}{2}$ 分解 n，其中 p 和 q 是两个不同的奇质数，n 是一个 Blum 整数（Blum Integer），所谓“Blum 整数”是一个半素数，半素数是两个素数的乘积。Blum 整数不能是偶数，也不能是 20 以下的数，因此，当需要寻找 Blum 整数时，只需要检查大于 20 的奇整数是否是 Blum 整数。

由于大整数分解是一个困难性问题，因此，在多项式时间内寻找到模 $n = pq$ 的二次剩余也是一个困难性问题。简单来说，二次剩余困难性问题是指：给定互质的两个整数 a 和 n（否则，$a \equiv 0 \pmod{n}$），判断 a 是否是模 n 的二次剩余（或是否存在模平方根 x）的问题。如果 a 是模 n 的二次剩余，找出满足 $x^2 \equiv a \mod n$ 的 x 值在计算上是困难的。该问题的难度等价于大整数分解困难问题，当 n 是质数时，存在多项式时间算法，但 n 是合数时，则找不到多项式时间算法。二次剩余问题是一个 NP 问题。

5.3.4 判定 n 阶剩余类问题

判定 n 阶剩余类问题也称为“合数剩余类困难问题”、“复合剩余类难题”。给定两个大质数 p 和 q，则乘积 $n = pq$ 为合数。如果整数 z 是模 n^2 的 n 阶剩余类中的元素，那么存在整数 $y \in \mathbb{Z}_{n^2}^*$，使得 $z = y^n \bmod n^2$，但若给定一个整数 z，判定 z 是不是模 n^2 的 n 阶剩余类中的元素是困难的。

除了上述的几个困难性问题，密码学中常用的困难问题还包括近似最大公因子（Greatest Common Divisor，GCD，也称为“最大公约数”）难题和离散子集求和难题等。需要注意，密码学中所说的“困难”并不是指相应的问题是不可求解的，而是指随着问题规模增大，求解该问题的时间复杂度将会很高，比如是指数级的时间复杂度 $O(2^n)$。

5.3.5 LPN 困难性问题

LPN（Learning Parity with Noise，带有噪声的奇偶性学习）困难性问题是一个在学习理论和密码学领域都很热门的问题，也是 Learning With Errors（LWE，带误差学习，参见附录5.14）问题在二元域上的版本，具体表述如下：给定一组向量，其中部分向量经过了随机噪声的干扰，任务是找出这些向量以及生成它们的随机线性码。这一问题是极其困难的，具有抗量子计算的能力。

LPN 问题本身在最坏情况下是 NP-Complete 问题（即 NP 完全问题，或简称为 NPC 问题，所有的 NP 问题都可以规约成它，参见附录2.3），而且也是少数被认为是在量子计算机模型下仍能保持安全的计算复杂问题。LPN 问题与编码理论中一个著名的悬而未决的公开问题，即随机线性码纠错问题（Decoding a random linear code problem）等价。由于该问题的困难性，基于 LPN 的困难假设构造 CPA（选择明文攻击）或 CCA（选择

密文攻击）安全的公钥加密方案是极具挑战性的任务。对 CPA 和 CCA 的介绍，参见附录5.1。基于 LPN 困难性问题可以构造 VOLE（向量不经意线性计算）等密码学应用。

5.4 对称加密算法

对称加密算法是一种应用较早的加密算法，技术成熟。在对称加密算法中，消息发送方将明文（原始数据）和加密密钥一起经过特殊加密算法（对称加密算法）处理后，使其变成复杂的加密密文发送出去。消息接收方收到密文后，若想解读原文，则需要使用加密用过的密钥及相同算法的逆算法对密文进行解密，才能使其恢复成可读明文。以 Alice 和 Bob 之间进行对称加密通信为例，如图5–9所示。

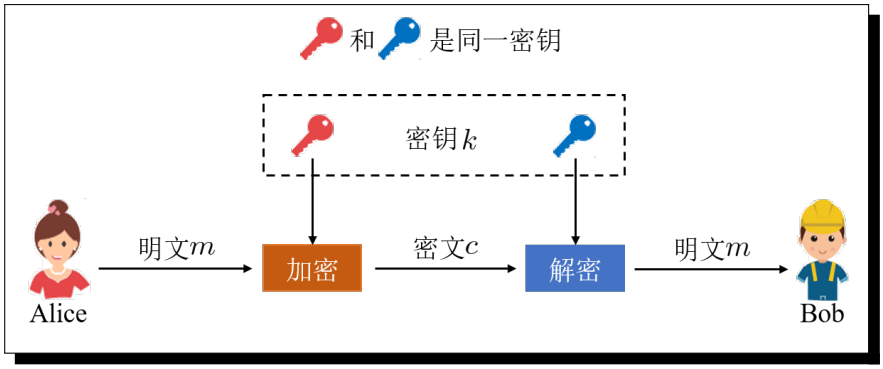

图 5–9 对称加密算法示意

在对称加密算法中，使用的密钥只有一个，消息收发双方都使用这个密钥对数据进行加密和解密，这就要求消息接收方事先必须知道加密密钥。有时这类加密算法又叫做传统密码算法，要求消息发送方和消息接收方在安全通信之前，商定一个密钥。

目前常见的对称加密算法有：DES、3DES、AES、Blowfish、RC2、RC4、RC5、IDEA、SKIPJACK 等。这些算法中，DES（数据加密标准）是一种常见的分组加密算法，3DES（或称为 TDEA）是三重数据加密算法块密码的通称，AES（高级加密标准）则是下一代的加密算法标准，速度快，安全级别高。

5.5 异或运算

异或运算通常用“XOR”表示，是 eXclusive OR（独有的“或”运算）的缩写，在密码学中经常使用。为便于与 OR（或）运算进行对比，OR 运算和 XOR 运算的真值表如表5–2和5–3所示。

表 5–2 OR 真值表

x	y	OR 结果
0	0	0
0	1	1
1	0	1
1	1	1

表 5–3 XOR 真值表

x	y	XOR 结果
0	0	0
0	1	1
1	0	1
1	1	0

由此可见，XOR 可以看作更单纯的 OR 运算，主要用于判断两个值是否不同（相异），相异则运算结果为 1，相同则为 0。通常用符号“⊕”来表示 XOR 运算，XOR 运算的性质如下（其中 a，b，c 为某个值）：

- $a \oplus a = 0$
- $a \oplus 0 = a$
- 可交换性：$a \oplus b = b \oplus a$
- 结合性：$a \oplus b \oplus c = a \oplus (b \oplus c) = (a \oplus b) \oplus c$

异或运算等价于二进制下的模 2 加法运算（先求和再模 2）。模 2 加法运算是一种常见的数学运算方法，在计算机科学和信息技术领域中经常使用，其特点是：当两个二进制数的对应位不同时，运算结果为 1；当两个二进制数的对应位相同时，运算结果为 0，这与异或运算的运算规则是一致的。模 2 加法运算的一个常见的应用场景是计算校验和（check sum）。在数据通信过程中，发送方可以通过对数据进行模 2 加法运算得到一个校验和，然后将这个校验和附加到数据包的末尾发给接收方。接收方接收到数据包后，同样对数据进行模 2 加法运算，并将计算结果与接收到的校验和进行比较，如果两者相同，则认为数据没有出错；如果两者不同，则认为数据有误，需要重传。

与异或运算相对应的是同或运算，通常用符号“⊙”表示，主要用于判断两个值是否相同，相同则运算结果为 1，否则为 0。

5.6 Diffie-Hellman 密钥协商

Diffie-Hellman 密钥协商算法早于 RSA 非对称加密算法，因此，我们首先来看 Diffie-Hellman 密钥协商算法，然后再介绍公钥加密算法。

5.6.1 Diffie-Hellman 密钥协商算法

密钥协商（key negotiation/agreement）也称为密钥交换（key exchange）。Diffie-Hellman 密钥协商算法（简称为 D-H 密钥协商算法，或 D-H 算法）的思想是密钥协商双方通过

交换一些信息，协商出一个只有双方知道的密钥。

以 Alice 和 Bob 进行密钥协商为例。假设系统公开参数包含一个大质数 p 和一个阶为 p 的有限循环群 $\mathbb{G}$（例如，$\mathbb{G} = \mathbb{Z}_p^*$），$\mathbb{G}$ 的生成元为 g。D-H 密钥协商算法可描述如下：

（1）Alice 选择随机数 $a \in_R \mathbb{Z}_p^*$，计算 $y_A = g^a \pmod p$，并公开 y_A，这里，$\mathbb{Z}_p^*$ 为定义在大质数 p 上的循环群（$\mathbb{Z}_p$ 是有限域，$\mathbb{Z}_p^*$ 由 $\mathbb{Z}_p$ 中的非零元素组成）；
（2）Bob 选择一个随机数 $b \in_R \mathbb{Z}_p^*$，计算 $y_B = g^b \pmod p$，并公开 y_B；
（3）Alice 获取到 y_B 后，计算 $y_B^a \pmod p$；
（4）Bob 获取到 y_A 后，计算 $y_A^b \pmod p$。

显然，有：

$$y_B^a \pmod p = y_A^b \pmod p = g^{ab} \pmod p = Key_{AB}, \tag{5.1}$$

Key_{AB} 就是 Alice 和 Bob 协商出的只有双方知道的密钥[1]。

可以将 D-H 密钥协商算法与具体的非对称加密算法结合，得到 D-H 密钥协商算法的具体形式。比如将 D-H 密钥协商算法与椭圆曲线加密算法（ECC）结合，得到椭圆曲线 D-H 密钥协商（ECDH）算法，比如 ECDH（P256）、ECDH（P384）、ECDH（P521）等，这时，Alice 和 Bob 选择的随机数 a 和 b 即为双方的私钥，g 对应于具体椭圆曲线上的基点 G，y_A 和 y_B 对应于双方的公钥，这里不再赘诉。另外，作为密钥协商算法的应用，有一种基于 ECDH 的对称加密算法，也称为椭圆曲线集成加密方案（Elliptic Curve Integrated Encryption Scheme，ECIES）[2]，具体实现过程如图5–10所示。

由于 D-H 密钥协商算法中缺少对密钥协商双方的身份认证，因此可能会遭受中间人攻击，即 A 和 B 都与中间人 C 协商了一个密钥（在密钥协商过程中，A 和 B 并不知情），然后 C 就可以仿冒 A 和 B 的身份，监听双方的通信，而 A 和 B 都不知道他们在和 C 共享通信。另外，D-H 密钥协商过程还可能遭受阻塞性攻击，即敌手请求大量的密钥，使得被攻击方花费相对较多的计算资源来回应敌手的请求，而无暇做真正的工作。

关于 D-H 算法，有两个困难性假设：一是计算 D-H 困难假设（Computational Diffie-Hellman Assumption，CDH），简称为 CDH 假设；二是决策 D-H 困难假设（Decisional Diffie-Hellman Assumption，DDH），简称 DDH 假设。

[1]因为密钥协商过程中用到了 Alice 和 Bob 选择的随机数，而该随机数是其他方不知道的。

[2]https://medium.com/asecuritysite-when-bob-met-alice/elliptic-curve-integrated-encryption-scheme-ecies-encrypting-using-elliptic-curves-dc8d0b87eaa.

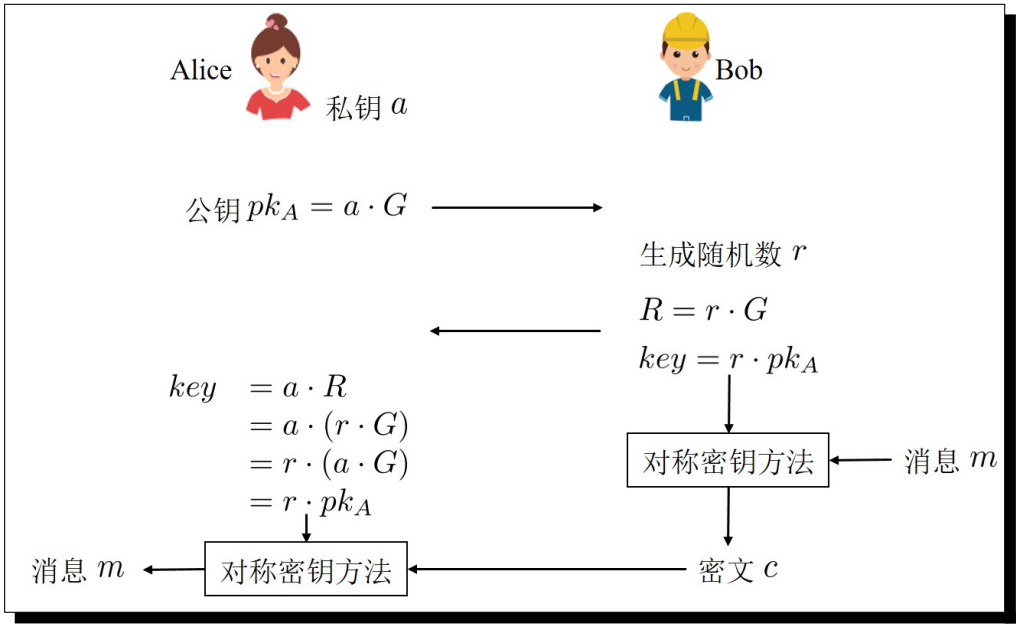

图 5–10 ECIES 的实现过程

5.6.2 CDH 假设

假设有人窃听了 Alice 和 Bob 之间进行的 D-H 密钥协商过程，并获得了 $\mathbb{G}$，g，g^a 和 g^b，CDH 假设是指：基于 $\mathbb{G}$，g，g^a 和 g^b，在多项式时间内计算得到 g^{ab} 是困难的。

5.6.3 DDH 假设

直观上来说，DDH 问题是指敌手是否能够从 $\mathbb{G}$ 中的元素中区分出 Alice 和 Bob 协商出的密钥 Key_{AB}。DDH 假设是指：要区分密钥 Key_{AB} 与 $\mathbb{G}$ 中的随机元素是困难的。

如果 DDH 假设成立，那么对于所有 PPT 算法 $\mathcal{A}$，都应存在可忽略的函数 negl()，满足：

$$|Pr[\mathcal{A}(\mathbb{G}, p, g, g^a, g^b, g^c) = 1] - Pr[\mathcal{A}(\mathbb{G}, p, g, g^a, g^b, g^{ab}) = 1]| \leq \varepsilon(n), \tag{5.2}$$

该式中的 a，b，c 是从 $\mathbb{Z}_p^*$ 中随机选取的。显然，DDH 假设强于 CDH 假设。

5.7 公钥加密算法

公钥加密算法也称为非对称加密算法。在 MPC 领域，常用的公钥加密算法包括 RSA、ElGamal、Lifted ElGamal、Okamoto-Uchiyama、Paillier 和 ECC 算法等。其中，RSA 和 ElGamal 满足乘法同态性，Paillier 算法具有加法同态性。一个加密算法具有同态性质

是指：对于两个明文 a 和 b，该算法满足 $\mathrm{Dec}(\mathrm{Enc}(a) \odot \mathrm{Enc}(b)) = a \oplus b$，$\odot$ 和 $\oplus$ 分别对应明文域和密文域上的运算。当 $\oplus$ 代表加法时，称该加密算法具有加法同态性，当 $\oplus$ 代表乘法时，称该加密算法具有乘法同态性，而不管 $\odot$ 代表的是加法还是乘法。

5.7.1 RSA

RSA 算法是 1977 年由 Ronald L. Rivest、Adi Shamir 和 Leonard Adleman 共同提出的 [2]，是最为经典的公钥加密算法，至今已有 40 多年的历史了，其安全性基于大整数分解困难问题。

5.7.1.1 RSA 算法描述

RSA 公钥加密算法基于欧拉定理和大整数分解困难性问题（即分解两个大质数的乘积 $n = pq$ 的困难性），其中欧拉定理是该算法的核心，大整数分解困难性问题是该算法的安全保证。RSA 公钥加密算法的密钥生成和加解密过程如下：

（1）密钥生成

- 随机选择两个不相等的大质数 p 和 q。
- 计算 $n = pq$（这是保证算法安全性的基础）。
- 计算欧拉函数 $\phi(n) = (p-1)(q-1)$。
- 随机选择一个整数 e，满足 $1 < e < \phi(n)$，且 e 与 $\phi(n)$ 互质（即：$\gcd(e, \phi(n)) = 1$，其中 gcd 表示求最大公约数，greatest common divisor）。
- 计算 e 对于 $\phi(n)$ 的模反元素 d。也就是要求解 $ed \equiv 1 \pmod{\phi(n)}$，即求解 $ed - 1 = k\phi(n)$，若令 $d = x$，$-k = y$，则有：$ex + \phi(n)y = 1$。
- 私钥 $\mathsf{sk} = d$，将 n 和 e 封装成公钥 $\mathsf{pk} = (n, e)$。

（2）加密过程

假设消息明文为 m。当加密消息 m 时，需要使用公钥进行如下计算：

$$\mathrm{Enc}(n, e, m) : m^e \equiv c \pmod{n},$$

c 即为消息密文。

（3）解密过程

使用私钥解密，即计算

$$\mathrm{Dec}(d, c) : c^d \equiv m \pmod{n}.$$

在 RSA 算法中，消息 m 必须是整数，且满足 $m < n$，如果是字符串，可以取其

ASCII 值或 Unicode 值。如果要加密大于 n 的整数，有两种解决方法：一是把长消息分割成若干段短消息，每段分别加密；二是先选择一种对称加密算法（比如数据加密标准 DES 等），用该算法加密消息后，再用 RSA 公钥加密算法加密 DES 加密后的消息密文。

安全性方面，由上述加解密过程可见，如果不知道 d，就没有办法从密文 c 中求解出明文 m。而要知道 d 就必须分解大整数 n，根据大整数分解困难性问题，这是极难做到的，因为目前还不存在一个多项式复杂度的算法可以解决该问题，所以，一般认为，当选择的 n 足够大（通常 ≥ 2048 位）时，RSA 算法是能够保证通信安全的。

5.7.1.2 RSA 算法的安全风险分析

在学术研究中，未对明文进行填充（padding）操作的 RSA 算法也称为“Textbook RSA”（直译为“教科书式 RSA”），后续研究表明其存在安全性隐患，这主要体现在如下两方面：

（1）Textbook RSA 是一个确定性的算法（加密过程中未引入随机数），也就是说，对于确定的明文和密钥，总是会计算出同样的密文。假如攻击者知道明文大概会是什么样的，比如 11 位的电话号码，他就可以用少得多的代价，尝试出所有的明文-密文组合，从而可以在下一次拿到密文的时候，通过对比找出相应的明文。这种在攻击者可以选择明文分布的前提下，如何保证密文不被破解，就构成了密码学中的“**选择明文攻击下的语义安全性**”（semantic security under chosen plain-text attack）问题。显然，在类似上面的场景中，RSA 算法存在安全性问题，易受重放攻击和统计分析。

（2）Textbook RSA 算法还存在“密文可构造”的问题。考虑这样一个场景：攻击者不仅可以获得前面场景中提到的信息，还可以向解密方发送密文并让解密方帮他解密，这类似于选择密文攻击。比如，攻击者可计算 $c' \equiv c \cdot 2^e \pmod{n}$，并发送给解密方，解密方解密之后，攻击者就得到了 $2m$，从而得到明文消息 m。密文的可构造性显然容易引起很严重的后果，例如对 AES-CBC（CBC 模式下的 AES 加密算法）的攻击。上面的场景构成了一个更强的安全性假设，即选择密文攻击下（CCA）的安全性。该场景听起来似乎是不切实际的，但对于 C/S（Client-Server）方式的应用，如果 Client 也掌握在攻击者手中，攻击者就可以通过某些手段来获取 Client 解密之后的明文。

密码学协议的安全性往往并不是绝对的。在真实的应用环境中，对密码学协议实施任何一种攻击都是相对困难的，攻击者更倾向于去关注代码层面的漏洞。但即使如此，设计者在设计协议时依然有必要去考虑上述问题。对于 Textbook RSA 算法，一种有效的手段是通过“填充”（padding），就是将明文与某个随机数组合起来，即对明文进行“加盐”操作。这样，明文就被转移到了一个更大的明文空间上，这可在一定程度上防止选择明文攻击。但这样还是不够的，随着选择密文攻击方式被提出，密码学界提出了

更安全的填充模式，即 RSA-PSS 和 RSA-OAEP：

- RSA-PSS 是 RSA 签名的两种主要方法之一，全称为 Probabilistic Signature Scheme，即概率签名方案。它是私钥签名过程的一种填充模式，与 RSA-PKCS#1 v1.5 版共同构成目前主流的 RSA 签名方式。相对于 PKCS（Public-Key Cryptography Standards，公钥密码标准），PSS 不能从签名中恢复原始签名。在 Openssl-1.1.x 版本之后，默认使用更安全的 RSA-PSS 签名模式。
- RSA-OAEP 有时也写作 RSA-OA-EP，是一种公钥密码算法，其中的 OAEP（Optimal Asymmetric Encryption Padding，最优非对称加密填充）是一种填充算法，通过在待加密的数据上附加随机填充来增加数据的随机性，从而增加了加密结果的安全性。在 RSA-OAEP 中，RSA 用于加密和解密数据，而 OAEP 用于增加数据的安全性。这种算法结合了 RSA 的稳定性和 OAEP 的安全性，可提供更强大的加密和解密安全性。

5.7.1.3 RSA 算法的乘法同态性

假设有两个明文 m_1 和 m_2，私钥为 $\mathsf{sk} = (n, d)$，公钥为 $\mathsf{pk} = (n, e)$，公钥用于加密，私钥用于解密。使用 RSA 算法加密后，得到

$$\begin{aligned} c_1 &= \mathrm{Enc}(e, m_1) = m_1^e \bmod n, \\ c_2 &= \mathrm{Enc}(e, m_2) = m_2^e \bmod n. \end{aligned} \tag{5.3}$$

将上述两个密文相乘，可以得到

$$\begin{aligned} c_1 \cdot c_2 &= m_1^e m_2^e \bmod n \\ &= (m_1 m_2)^e \bmod n \\ &= \mathrm{Enc}(e, m_1 m_2). \end{aligned} \tag{5.4}$$

显然，解密 $c_1 \cdot c_2$ 时，$\mathrm{Dec}(d, c_1 \cdot c_2) = (c_1 \cdot c_2)^d \bmod n = m_1 m_2$，即：将两个消息相乘后加密，所得的结果等价于将两个密文相乘后的结果，因此，RSA 算法具有乘法同态性。

如前所述，在实际应用中，为了保证强安全性，RSA 算法可采用填充模式（比如，RSA_PKCS1_Padding、RSA_PKCS1_OAEP_Padding 等），根据密钥长度（常用 1024 位或 2048 位）对明文分组进行填充，而**只有不对明文进行填充的原始 RSA 算法才能满足乘法同态特性**。因此，利用 RSA 的乘法同态性实现同态加密（HE）运算会存在安全弱点，攻击者可能通过选择明文攻击等攻击方式得到原始数据。

5.7.2 ElGamal

ElGamal 公钥加密算法是由埃及密码学家和企业家 Taher ElGamal 在 1985 年提出的一种基于 Diffie-Hellman 离散对数困难问题的公钥密码算法 [3]。请注意，这里之所以拼成 ElGamal，而不是 Elgamal 或者 El Gamal，是因为该算法的提出者本人就是这样拼写的，详见英文版维基百科中的“Taher ElGamal”词条。ElGamal 公钥加密算法可实现公钥加密和数字签名功能，并满足乘法同态特性。目前，离散对数困难性问题仍然没有高效的求解方法，因此该加密算法的安全性还是有保障的。

5.7.2.1 ElGamal 加密算法描述

ElGamal 公钥加密算法的加解密过程如下：

（1）密钥生成

- 随机选择一个大质数 p，且要求 $p-1$ 有大质数因子。再选择一个模 p 的本原元 a，将 p 和 a 公开。
- 随机选择一个整数 d，作为私钥，$2 \le d \le p-2$。
- 计算 $y = a^d \mod p$，取 y 为公钥。

（2）加密过程

用 m 表示明文，则 Elgamal 算法的加密过程为：

- 随机选取一个整数 k，满足 $2 \le k \le p-2$；
- 计算 $C_1 = a^k \mod p$；
- 计算 $C_2 = my^k \mod p$。

则密文为：

$$c = \mathrm{Enc}(y, m) = (C_1, C_2).$$

（3）解密过程

使用私钥 d，由密文 c 可得明文 m：

$$\mathrm{Dec}(d, c) = \frac{C_2}{C_1^d} \mod p = \frac{my^k}{(a^k)^d} \mod p = m\frac{(a^d)^k}{(a^k)^d} \mod p = m.$$

附录4.4.2已给出本原元的定义，这里再次说明如下：使得 $a^n \equiv 1 \mod p$ 成立的最小正幂 n 若满足 $n = \phi(p) = p-1$，则称 a 是 p 的本原元（也称为本原根或原根），其中函数 ϕ 为欧拉函数。

ElGamal 算法是一种随机化加密算法（也称为“非确定性加密算法”），随机化加密是指密文由明文和所选的随机数来确定，即使每次使用相同的密钥加密相同的明文，

得到的密文结果也不相同，因此不存在与 RSA 算法类似的选择明文攻击问题。但是，Elgamal 算法的安全密钥长度和其他一些加密算法相比还是较长，因此其运算速度有待提高，而且由于 Elgamal 密码系统加密和解密的数学算法完全不同，造成签名和验证的时间开销不同，加密和解密过程花费的时间也不同。

ElGamel 算法有门限 ElGamal 加密算法的版本。

5.7.2.2 ElGamal 算法的乘法同态性

对于明文 m_1 和 m_2，使用 ElGamal 算法加密后，对应的密文分别为

$$\begin{aligned} c_1 &= (C_1^{m_1}, C_2^{m_1}), \\ c_2 &= (C_1^{m_2}, C_2^{m_2}), \end{aligned} \tag{5.5}$$

其中，

$$\begin{aligned} C_1^{m_1} &= a^{k_1} \bmod p, \\ C_2^{m_1} &= m_1 y^{k_1} \bmod p, \\ C_1^{m_2} &= a^{k_2} \bmod p, \\ C_2^{m_2} &= m_2 y^{k_2} \bmod p, \end{aligned} \tag{5.6}$$

y 为公钥，k_1 和 k_2 为随机选择的两个整数。所以，

$$\begin{aligned} c_1 \cdot c_2 &= \mathrm{Enc}(k_1, k_2, y, m_1, m_2) \\ &= \left(C_1^{m_1} \cdot C_1^{m_2}, C_2^{m_1} \cdot C_2^{m_2}\right) \\ &= \left(a^{k_1+k_2} \bmod p, m_1 m_2 y^{k_1+k_2} \bmod p\right) \\ &= \left(C_1^{m_1 m_2}, C_2^{m_1 m_2}\right). \end{aligned} \tag{5.7}$$

用 d 表示私钥，则使用 ElGamal 算法解密时，可得到:

$$\mathrm{Dec}\left(d, C_1^{m_1 m_2}, C_2^{m_1 m_2}\right) = \frac{C_2^{m_1 m_2}}{\left(C_1^{m_1 m_2}\right)^d} \bmod p = m_1 m_2, \tag{5.8}$$

即：密文相乘并解密后，等价于直接将明文相乘，因此，ElGamal 公钥加密算法满足乘法同态性。

Elgamal 公钥加密算法是 ISO 同态加密国际标准中唯一指定的乘法同态加密算法。

5.7.3 Lifted ElGamal 门限加密算法

在 MPC 中，门限密码体制是对抗合谋攻击的一个重要工具。在门限密码体制中，公钥由 n 个参与方联合生成，可以直接用于加密消息；解密密钥由 n 个参与方联合持有，但需要 n 个参与方中一定数量的参与方参与才能正确解密。类似于秘密分享，如果至少需要 n 个参与方中的 t 个参与方参与才能解密，少于 t 个参与方时无法获取到明文的任何信息，这样的密码体制称为“(t,n)-门限密码体制”。

Lifted ElGamal 是一种结合 ElGamal 加密与同态加密的门限加密算法 [3]，通过同态加密技术实现密文运算，支持对加密数据的直接计算，而无需解密。这一特性使其在隐私计算和多方协议中尤为重要，可保护数据隐私并实现安全计算。

5.7.3.1 Lifted ElGamal 门限加密算法描述

Lifted ElGamal 门限加密算法的密钥生成和加解密过程如下：

（1）密钥生成

给定安全参数 λ，密钥生成算法生成一个 λ 比特的大质数 p，以及 $\mathbb{Z}_p^*$ 的一个生成元 g。n 个参与方 $P_1, P_2, \cdots, P_n$ 共同协商一个小质数 ρ。参与方 P_i 选取私钥 $k_i \in \mathbb{Z}_p^*$，然后计算 $K_i = g^{k_i} \mod p$，并联合生成公钥

$$h = \prod_{i=1}^{n} K_i \mod p = \prod_{i=1}^{n} g^{k_i} \mod p = g^{\sum_{i=1}^{n} k_i} \mod p. \tag{5.9}$$

（2）加密过程

对于要加密的明文 $m \in \mathbb{Z}_p^*$，选择随机数 r，然后计算密文

$$C = (C_1, C_2) = (g^r \mod p,\ \rho^m h^r \mod p). \tag{5.10}$$

（3）联合解密过程

首先，每个参与方 P_i $(i \in [n])$ 计算 $t_i = C_1^{k_i} \mod p$，并公布 t_i。然后，n 个参与方进一步计算

$$t = \prod_{i=1}^{n} t_i \mod p = \prod_{i=1}^{n} C_1^{k_i} \mod p = C_1^{\sum_{i=1}^{n} k_i} \mod p, \tag{5.11}$$

最后，计算 $\rho^m = \frac{C_2}{t} \mod p$，通过查询构造的对数表[a]得到解密结果 m。

[a] Lifted ElGamal 门限加密算法解密结果为 ρ^m，此时，需要进一步计算离散对数，为简化计算，可以通过离散对数表获得离散对数计算结果。

5.7.3.2 Lifted ElGamal 门限加密算法加法同态性

对于明文消息 $m_1, m_2 \in \mathbb{Z}_p^*$，使用 Lifted ElGamal 门限加密算法可得：

$$\begin{aligned} E(m_1) &= (g^{r_1} \mod p,\ \rho^{m_1} h^{r_1} \mod p), \\ E(m_2) &= (g^{r_2} \mod p,\ \rho^{m_2} h^{r_2} \mod p), \\ E(m_1)E(m_2) &= (g^{r_1+r_2} \mod p,\ \rho^{m_1+m_2} h^{r_1+r_2} \mod p) \\ &= (g^{r} \mod p,\ \rho^{m_1+m_2} h^{r} \mod p) \\ &= E(m_1+m_2). \end{aligned} \tag{5.12}$$

因此，Lifted ElGamal 门限加密算法具有加法同态性。

5.7.3.3 Lifted ElGamal 门限加密算法的应用场景

Lifted ElGamal 门限加密算法常用于以下场景：

- 隐私保护协议：通过加密数据保护用户隐私，同时支持密文运算。
- MPC：在多个参与方的计算场景中，确保数据隐私和计算准确性。

2021 年，浙江大学等机构开展了 Lifted ElGamal 门限加密算法的相关研究，并应用于隐私集合求交协议设计等场景。

另外，已有开源库[1]实现了 Lifted ElGamal 门限加密算法，该开源库也实现了对受限明文空间的零知识证明。

5.7.4 Okamoto-Uchiyama 加密算法

Okamoto-Uchiyama（OU）加密算法是一种基于复合剩余类困难问题的公钥加密方案 [4]，其安全性依赖于大整数分解的困难性。该算法在隐私计算领域被广泛应用，尤其在同态加密场景中表现突出。

5.7.4.1 OU 加密算法描述

OU 加密算法的密钥生成和加解密过程如下：

（1）密钥生成

给定安全参数 λ，选取两个 λ 比特的大质数 p 和 q，令 $n = p^2 q$。在 $\mathbb{Z}_{p^2}^*$ 中随机选取一个生成元 g，使得 $g_p = g^{p-1} \mod p^2$ 的阶为 p，则有 $\gcd(p, q-1) = 1$，

[1] https://github.com/aistcrypt/Lifted-ElGamal.

$\gcd(p-1,q)=1$。计算 $h=g^n \mod n$，定义函数 $L(x)=\frac{x-1}{p}$，则公钥为 $\mathsf{pk}=(g,n,h)$，私钥为 $\mathsf{sk}=(p,q)$。

（2）加密过程

对于明文 $m\in\mathbb{Z}_p$，选取随机数 $r\in\mathbb{Z}_n^*$，然后计算密文

$$c=g^m h^r \mod n. \tag{5.13}$$

（3）解密过程

对于密文 $c\in\mathbb{Z}_n^*$，解密得到明文

$$m=\frac{L(c^{p-1} \mod p^2)}{L(g^{p-1} \mod p^2)} \mod p. \tag{5.14}$$

5.7.4.2 OU 加密算法的加法同态性

对于明文消息 $m_1,m_2\in\mathbb{Z}_p$，使用 OU 加密算法可得：

$$\begin{aligned}
E(m_1)&=g^{m_1}h^{r_1} \mod n,\\
E(m_2)&=g^{m_2}h^{r_2} \mod n,\\
E(m_1)E(m_2)&=g^{m_1+m_2}h^{r_1+r_2} \mod n\\
&=g^{m_1+m_2}h^{r} \mod n\\
&=E(m_1+m_2),\\
E(m_1)^{m_2}&=g^{m_1m_2}h^{r_1m_2} \mod n\\
&=g^{m_1m_2}h^{r} \mod n\\
&=E(m_1m_2).
\end{aligned} \tag{5.15}$$

因此，OU 加密算法具有加法同态性。

5.7.4.3 OU 加密算法的应用

在 OU 加密算法的实际工程实现中，采用先取模再计算的方式优化数值稳定性，例如，对明文和密文，先进行模运算，再执行幂运算，可既防止溢出又提升计算效率。此外，通过向量化处理和并行计算可进一步提升大规模数据处理性能。

OU 加密方案在 HEU 隐私计算库中被用于同态加密场景，支持密文加法、乘法等操作。测试显示，其密文加法性能为 57.16 毫秒/万次，但通过 Python 接口调用时存在约 3

倍性能损耗（实测与理论值对比）。

5.7.5 Paillier

Paillier 公钥加密算法是 Pascal Paillier 在 1999 年提出的概率公钥加密算法 [5]，其安全性基于合数剩余类困难问题（即判定 n 阶剩余类难题），是一种满足加法同态的加密算法，已经广泛应用在加密信号处理或第三方数据处理领域。

5.7.5.1 Paillier 加密算法描述

Paillier 公钥加密算法的密钥生成和加解密过程如下：

（1）密钥生成

- 选择两个质数 p 和 q，使得 $|p| = |q| = \lambda$（或者令随机选择的两个质数 p 和 q 满足 $\gcd(pq, (p-1)(q-1)) = 1$，其中 gcd 表示求最大公约数，这一条件也能保证 p 和 q 的长度相等），其中 λ 是系统安全参数。
- 令 $n = pq$，$\tau = \mathrm{lcm}(p-1, q-1)$，其中 lcm 表示求最小公倍数（least common multiple）。
- 定义 $L(x) = (x-1)/n$，随机选择 $g \in_R \mathbb{Z}_{n^2}^*$，满足 $\gcd(L(g^\tau \mod n^2), n) = 1$。
- 公钥为 (n, g)，私钥为 τ。

（2）加密过程

对于消息 $m \in \mathbb{Z}_n$，选择任意随机数 $r \in_R \mathbb{Z}_n^*$，对 m 加密得到密文：

$$c = \mathrm{Enc}(n, g, r, m) = g^m r^n \mod n^2.$$

（3）解密过程

使用私钥 τ，由密文 $c \in \mathbb{Z}_{n^2}^*$，解密 c 可得到明文：

$$\mathrm{Dec}(\tau, c) = \frac{L(c^\tau \mod n^2)}{L(g^\tau \mod n^2)} \mod n = m.$$

5.7.5.2 Paillier 算法的加法同态性

对于任意两个明文消息 $m_1, m_2 \in \mathbb{Z}_n$，以及两个随机数 $r_1, r_2 \in_R \mathbb{Z}_n^*$，由 Paillier 公钥加密算法可得：$c_1 = \mathrm{Enc}(n, g, r_1, m_1) = g^{m_1} r_1^n \mod n^2$，$c_2 = \mathrm{Enc}(n, g, r_2, m_2) = g^{m_2} r_2^n \mod n^2$，

则有:

$$\begin{aligned} c_1c_2 &= \mathrm{Enc}(n,g,r_1,m_1)\mathrm{Enc}(n,g,r_2,m_2) \\ &= g^{(m_1+m_2)}(r_1r_2)^n \bmod n^2. \end{aligned} \tag{5.16}$$

解密密文 c_1c_2，可得

$$\mathrm{Dec}(\tau, c_1c_2) = \frac{L((c_1c_2)^\tau \bmod n^2)}{L(g^\tau \bmod n^2)} \bmod n = m_1 + m_2, \tag{5.17}$$

因此，“密文乘”经解密后得到“明文加”。一般来说，我们希望密文计算得到的结果和明文计算的结果相同，所以对密文是如何计算的不做要求，因此我们称 Paillier 公钥加密算法具有加法同态性（即：要得到“明文加”的结果，只需要将密文相乘，然后解密就可以了），而不是乘法同态性。

5.7.6 ECC

5.7.6.1 什么是椭圆曲线

椭圆曲线（Elliptic Curve，EC）是由维尔斯特斯拉（Weierstrass）方程 $E: y^2+axy+by = x^3+cx^2+dx+e$ 所确定的平面曲线，其中 a、b、c、d、e 属于域 $\mathbb{F}$，域 $\mathbb{F}$ 可以是有理数域 $\mathbb{Q}$、复数域 $\mathbb{C}$，也可以是有限域 $GF(p)$。

并非所有的椭圆曲线都适合用来构建加密算法，可以用来加密的椭圆曲线的表达式为：

$$y^2 = x^3 + ax + b, \tag{5.18}$$

若取 $a = -2$、$b = 4$，则该椭圆曲线的几何形状如图5–11所示。

如图5–11，对于椭圆曲线上的两个点 P 和 Q，$P+Q$ 对应的是 P、Q 两点连线与椭圆曲线的交点关于 x 轴的对称点。

在密码学中，通常对椭圆曲线进行模 p 处理，此时对应的曲线 $\mathrm{Ep}(a,b)$ 表示为：

$$y^2 = x^3 + ax + b \pmod p \tag{5.19}$$

其中，p 为质数，$x, y \in [0, p]$。

ECC 加密算法的安全性基于离散对数困难性问题，具体来说：已知椭圆曲线上的一个点 G，对于给定的数值 d，计算 $P = dG$（P 也是椭圆曲线上的点）是容易的，但若给定 G 和 $P = dG$，求 d 是非常困难的。该安全性主要是模数运算来保障的。

5.7.6.2 ECC 加密算法描述

基于以上基础知识，ECC 加密算法可描述如下：

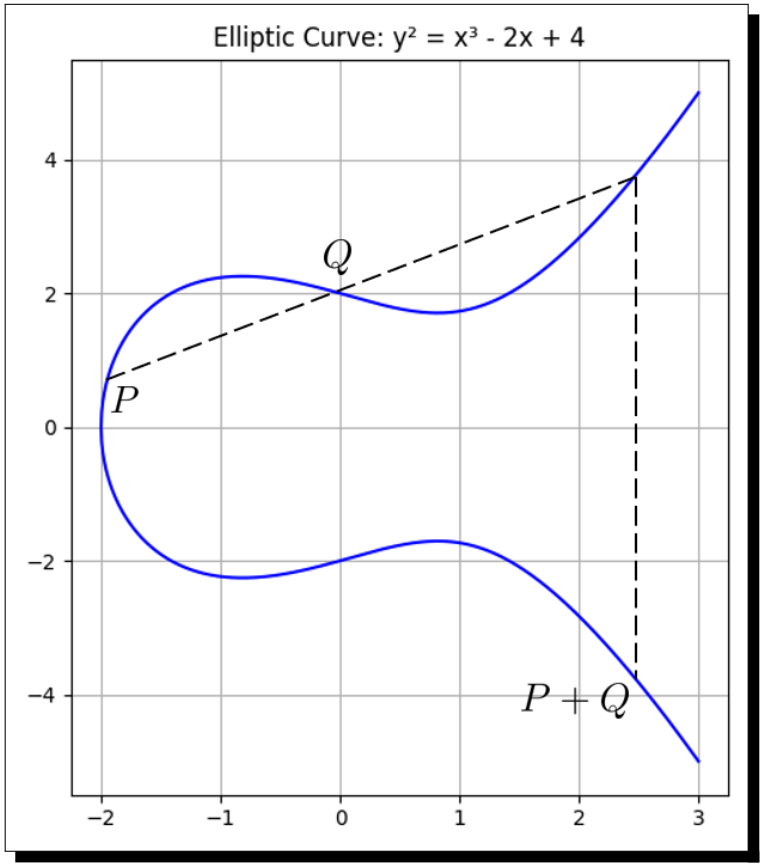

图 5–11 椭圆曲线示意

（1）密钥生成

选择椭圆曲线上的点 G 作为基点，假设私钥为 $\mathsf{sk} \in_R \mathbb{Z}_p^*$，则公钥为 $\mathsf{pk} = \mathsf{sk} \cdot G$。根据上述困难性问题，给定 pk，求解 sk 是非常困难的。

（2）加密过程

选择一个随机数 $r \in_R \mathbb{Z}_p^*$，生成消息 m 对应的密文 c，该密文是一个点对：

$$c = \mathrm{Enc}(\mathsf{pk}, m) = \{r \cdot G, m + r \cdot \mathsf{pk}\}. \tag{5.20}$$

（3）解密过程

使用私钥 sk 解密 c 得到明文 m：

$$\begin{aligned} \mathrm{Dec}(\mathsf{sk}, c) &= (m + r \cdot \mathsf{pk}) - \mathsf{sk} \cdot (r \cdot G) \\ &= m + r \cdot \mathsf{sk} \cdot G - \mathsf{sk} \cdot (r \cdot G) = m. \end{aligned} \tag{5.21}$$

5.8 单向函数（OWF）

单向函数（One-way Functions，OWF）是一种正向易于计算（可在多项式时间内完成计算），而逆向难以计算（即逆向计算无多项式时间算法）的函数，也就是说，给定一个输入，可以很容易计算出输出，但是给定输出，却很难计算出输入。单向函数的存在性对密码学意义重大，可认为单向函数的存在性等价于所有（有意义的）公钥密码算法的存在性。

单向函数的定义如下：称函数 $f:\{0,1\}^* \to \{0,1\}^*$ 是单向函数，当且仅当（iff）其满足：

- 易于计算：不需要使用原本的函数，只要结果相同即可，即存在 PPT 算法 M_f，使得 $\forall x \in \mathrm{Dom}(f)$，有 $M_f(x) = f(x)$。这里的 $\mathrm{Dom}(f)$ 表示函数 f 的定义域。
- 难以求逆：对于随机均匀选取的定义域中的比特串，将其映射过后再求它的任意一个原象是很困难的，即对于任意的 PPT 算法 $\mathcal{A}$（也表示攻击者），能找到可忽略函数 $\varepsilon()$，使得

$$\Pr[\mathrm{Invert}_{\mathcal{A},f}(n)] \le \varepsilon(n) \tag{5.22}$$

  或者

$$\Pr_{x\leftarrow\{0,1\}^n}[\mathcal{A}(f(x)) \in f^{-1}(f(x))] \le \varepsilon(n), \tag{5.23}$$

  上式中的 $x \leftarrow \{0,1\}^n$ 表示随机产生输入。

存在单向函数是可构造安全加密方案的充要条件，即：假设存在单向函数，则存在伪随机数生成器（PRNG，参见5.10.1）、伪随机函数（PRF，参见5.11.1）、伪随机置换（PRP，参见5.11.2）和选择密文攻击（CCA，参见5.1.3）安全的公钥加密方案；如果存在窃听者不可区分的公钥加密方案，则存在一个单向函数。

目前，是否存在完美的单向函数仍然是未知的，因此，存在单向函数本身就是一个密码学假设。但从实践上来看，这种假设是没有太大问题的。密码学安全的哈希函数（cryptographically secure hash functions）就是单向函数的候选者之一。

5.9 密码学中的认证方法

5.9.1 哈希算法

哈希（hash）算法又称摘要（digest）算法、散列算法、杂凑算法，也称为哈希函数，其作用是对任意输入数据（key）进行计算后，都能得到一个固定长度的数据摘要（或称消息摘要，hash value），这是易于计算的，但是反过来，给定一个消息摘要，很难得到原始输入数据，这有时候甚至是不可能的。

5.9.1.1 哈希算法的特点

一个优秀的哈希算法应该具备如下特点：

- 正向快速：哈希算法的执行效率要尽量高，针对较长的文本，也能快速地计算出其哈希值。
- 单向性：从哈希值不能反向推导出原始数据，所以哈希算法也叫单向哈希算法。
- 对输入数据非常敏感：相同的输入一定得到相同的输出，哪怕原始数据只修改了一个比特（bit），最后得到的哈希值也大不相同；
- 发生哈希冲突的概率很小：对于不同的原始数据，大概率得到不同的输出，哈希值相同的概率非常小。

注解 5.2 哈希冲突又称为哈希碰撞。哈希函数的抗碰撞性可进一步分为强抗碰撞性和弱抗碰撞性。

弱抗碰撞性是指：给定一个数据 d_1 的哈希值 $H(d_1)$，找到另一个数据 d_2，使得其哈希值 $H(d_2)$ 与已给定数据的哈希值 $H(d_1)$ 相同，这在计算上是不可行的。

强抗碰撞性是指：找到两个不同的数据 d_1 和 d_2，使得它们的哈希值相同，即 $H(d_1) = H(d_2)$，这在计算上是不可行的。

所谓“计算上不可行”是基于计算复杂性理论来说的，是指对于该运算，不存在一个多项式时间算法。这是哈希函数安全性的基础。

5.9.1.2 典型的哈希算法

典型的哈希算法主要包括 MD4、MD5（已被证明不够安全）和 SHA-1、SHA-2 和 SHA-3 等。MD5 和 SHA-1 均是以 MD4 为基础进行设计的。

MD4（RFC 1320）是 MIT 的 Ronald L. Rivest 于 1990 年设计的，其中的 MD 是 Message Digest（消息摘要）的缩写，其输出为 128 位。MD4 并不足够安全。MD5（RFC 1321）是 Rivest 于 1991 年对 MD4 的改进版本。它对输入仍以 512 位分组，其输出是 128 位。MD5 比 MD4 复杂，并且计算速度要慢一点，但更安全，但已证明 MD5 已不足够安全，因为其强抗碰撞性已经被攻破。

注解 5.3 Request For Comments（RFC）文档是一系列以编号排定的文件。RFC 文档由 Internet Society（ISOC）赞助发行。RFC 文档用于对互联网通信协议进行详细说明。此外，RFC 文档中还额外加入了许多在标准内的论题，例如对于互联网新开发的协议及其发展过程中的所有记录，因此，几乎所有的互联网相关标准都有相应的 RFC 文档。

SHA-1（Secure Hash Algorithm-1）是由 NIST（National Institute of Standards and Technology，美国国家标准技术研究所）和 NSA（National Security Agency，美国国家安全局）设计的，它的输出长度为 160 位，抗碰撞性比 MD5 更好。设计 SHA-1 时基于和 MD4 相同的原理，并且模仿了 MD4 算法。SHA-1 的强抗碰撞性已于 2005 年由王小云教授团队攻破，因此已不被推荐使用。

为了提高安全性，NIST 和 NSA 还设计出了 SHA-224（将 SHA-256 的结果截掉 32 比特）、SHA-256、SHA-512/224（将 SHA-512 的结果截掉 288 比特）、SHA-512/256（将 SHA-512 的结果截掉 256 比特）、SHA-384（将 SHA-512 的结果截掉 128 比特）、SHA-512 算法，统称为 SHA-2，这些算法跟 SHA-1 算法原理类似，也是一起发布的。SHA-2 尚未被攻破，因此仍可使用。

SHA-3 是 2005 年在 NIST 主导下，通过公开竞争的方式进行标准化的，最终于 2012 年将 Keccak 算法确定为 SHA-3 标准。不同于前面的哈希算法，Keccak 算法中采用的是海绵结构，其对输入长度没有限制。

除了上面介绍的哈希算法，典型的哈希算法还包括 RIPEMD-160。RIPEMD-160 是 1996 年由 Hans Dobbertin、Antoon Bosselaers 和 Bart Preneel 设计的，能够产生 160 比特的哈希值输出。这一系列哈希函数还包括 RIPEMD-128、RIPEMD-256、RIPEMD-320 等其他版本。比特币中使用的哈希算法就是 RIPEMD-160。

5.9.1.3 全域哈希函数

全域哈希（full-domain hash 或 universal hashing）函数是一种哈希函数的选择方法，旨在通过随机性来减少哈希碰撞的概率。全域哈希函数集合的定义如下：

定义 5.2 设 U 为键的全域，$\mathcal{H}$ 是哈希函数的有限集合。这些哈希函数将 U 中的键映射到哈希表的 m 个槽（$0, 1, \cdots, m-1$）中。如果对于任意不相等的 $x, y \in U$，满足条件 $H(x) = H(y)$ 的哈希函数 H 的数目等于 $|\mathcal{H}|/m$（即 $\mathcal{H}$ 中哈希函数总数的 $1/m$），则称 $\mathcal{H}$ 为全域哈希函数集合。

全域哈希函数通过随机选择哈希函数来减少碰撞的可能性，即全域哈希函数具有随机性。对于任意两个不相等的键 x 和 y，从 $\mathcal{H}$ 中随机选择一个哈希函数 H，x 和 y 的哈希值发生碰撞的概率是 $1/m$。碰撞概率方面，如果将 n 个键放入具有 m 个槽的哈希表中，并随机从 $\mathcal{H}$ 中选择哈希函数，那么对于给定的键 x，它与其它键发生碰撞的期望次数小于 n/m（该比值也称为负载因子 α）。

构造全域哈希的关键是创建一个哈希函数集合 $\mathcal{H}$，使得对于任意两个不相等的键 x 和 y，满足 $H(x) = H(y)$ 的哈希函数的数目等于 $|\mathcal{H}|/m$。一种常见的构造方法是使用多

项式哈希函数，并结合随机数生成器来确保哈希函数的随机性。

全域哈希在哈希表和数据存储领域有广泛应用，特别是在需要处理大量数据且希望减少碰撞概率的场景中。通过随机选择哈希函数，全域哈希可以提供比传统固定哈希函数更好的性能，因为传统哈希函数在某些情况下可能会遇到大量碰撞，导致性能下降。

与全域哈希相关的一个概念是完全哈希（Perfect Hashing），完全哈希是另一种哈希方法，它旨在构建一个静态哈希表，使得在最坏情况下查找时间也是 $O(1)$。完全哈希通常使用两级结构，每一级都应用全域哈希来减少碰撞。与全域哈希相比，完全哈希更注重于构建静态哈希表时的最坏情况的性能。

5.9.2 消息认证码

在开放的计算和通信环境（例如 Internet）中，可能会使用不可靠的媒介传输和存储信息。因此，对信息完整性（integrity）的校验在某些情况下就显得十分重要。基于密钥作完整性校验的方法常称为 MAC（Message Authentication Code，消息认证码）。通常，MAC 用于在共享密钥的双方之间校验相互传递的信息。

MAC 的具体实现方面，主要有两种方法：一是基于单向哈希函数来实现，即 HMAC（Hash-based Message Authentication Code，基于哈希的消息验证码）；二是基于分组密钥（如 AES）的实现，分组加密的工作模式有 ECB（Electronic CodeBook，电子密码本模式），CBC（Cipher Block Chaining，密文分组链接模式），CFB（Cipher Feedback，密文反馈模式），OFB（Output Feedback，输出反馈模式）四种 [6]，其中的 CBC 和 ECB 是两种比较常用的模式，相应的 MAC 分别称为 CBC-MAC 和 ECB-MAC。

在生成和验证 MAC 的过程中，消息收发双发需要使用一个双方共知的密钥，而其他第三方在未获知该密钥的情况下，无法计算出 MAC 值，因而无法篡改消息，因此，MAC 可用于保证消息完整性，同时对消息来源方进行身份验证。MPC 协议中通常使用两种信息论安全的消息认证码（IT-MAC），即 BDOZ 类型的 IT-MAC 和 SPDZ 类型的 IT-MAC。

5.9.3 数字签名算法

数字签名算法（Digital Signature Algorithm，DSA）是一种安全协议，数字签名类似于写在纸上的普通物理签名，但其使用公钥加密算法等技术来实现，用于鉴别数字信息的完整性，确保信息在传输过程中没有被篡改或伪造，同时也可以验证信息发送方的身份。数字签名算法的发展历程，以及对面向特定应用场景的签名方案的简介可参阅文献 [7]。由于数字签名方案非常多，这里仅简要介绍基于非对称加密算法的签名方案，和

盲签名、多重签名、群签名、环签名等面向特定应用场景的签名方案，以及 2004 年提出来的 Camenisch-Lysyanskay 签名方案（C-L 签名方案）。

数字签名的主要用途如下：

- 保证数据完整性：数字签名可以用来确保数据在传输过程中没有被篡改，从而保证数据的完整性。
- 验证身份：数字签名可以用于验证通信方的身份。在互联网通信中，公钥/私钥对的使用使得双方可以在不安全的通信通道中确认对方的身份。
- 防止重放攻击：数字签名可以防止未经授权的第三方在数据传输过程中重复使用或复制数据。
- 防止数据被伪造：数字签名可以防止数据被伪造，因为在基于公钥加密算法的数字签名方案中，只有知道私钥的人才能创建有效的数字签名。
- 防止签名方抵赖签名行为：数字签名可以防止抵赖，因为数字签名的创建者无法否认自己的签名。

5.9.3.1 基于非对称加密算法的签名方案

签名和验签的基本过程：基于 RSA 等非对称加密算法的数字签名生成和验证的基本过程如图5–12和5–13所示。

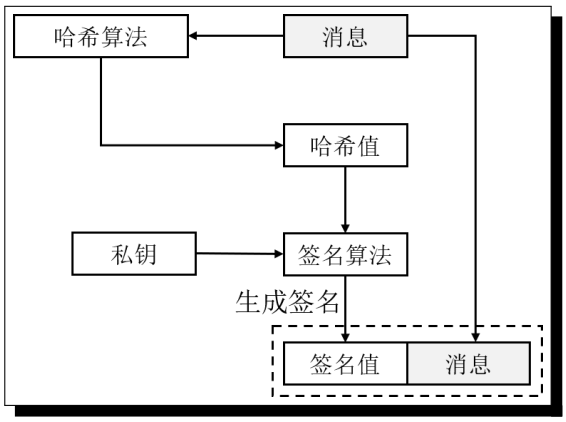

图 5–12 生成数字签名的基本过程

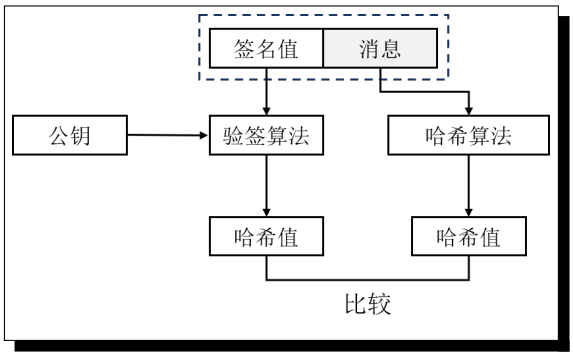

图 5–13 验证数字签名的基本过程

图5–12所示的数字签名的基本原理可概括为：消息发送方用哈希函数生成消息的哈希值，然后用自己的私钥对这个哈希值进行加密，加密后的哈希值将作为数字签名和消息一起发送给消息接收方。消息接收方接收到消息及其签名后，首先用与发送方一样的哈希函数从接收到的原始消息中计算出哈希值，再用发送方的公钥来对数字签名进行解密，如果解密出来的哈希值与计算出来的哈希值相同，那么接收方就能确认该消息是由

发送方进行数字签名的，并且消息在传输过程中没有被篡改。

椭圆曲线数字签名算法（ECDSA）：假设签名方的私钥为 sk，公钥为 $\mathsf{pk} = \mathsf{sk} \cdot G$，$G$ 为椭圆曲线上的基点，则椭圆曲线数字签名算法如下：

（1）签名过程

- 签名方选择随机数 $r \in_R \mathbb{Z}_p^*$，计算 $r \cdot G = (x, y)$。
- 签名方计算 $s = (H(m) + \mathsf{sk} \cdot x)/r$，其中 $H(m)$ 为消息 m 的哈希值（比如，ECDSA256 使用 SHA256 来生成 256 比特长的消息摘要）。
- 签名方将消息 m 和数字签名 $\{r \cdot G, s\}$ 发送给消息接收方。

（2）验签过程

- 消息接收方收到消息 m 及其签名后，计算消息的哈希值 $H(m)$。
- 使用发送方的公钥 pk 验证 $H(m) \cdot G/s + x \cdot \mathsf{pk}/s \stackrel{?}{=} r \cdot G$，如果两者相等，则验证通过，消息接收方接受该消息。

在上述验签过程中，利用了如下式所示的推导过程：

$$\begin{aligned} H(m) \cdot G/s + x \cdot \mathsf{pk}/s &= H(m) \cdot G/s + x \cdot (\mathsf{sk} \cdot G)/s \\ &= (H(m) + x \cdot \mathsf{sk}) \cdot G/s \\ &= r \cdot (H(m) + x \cdot \mathsf{sk}) \cdot G/(H(m) + x \cdot \mathsf{sk}) \\ &= r \cdot G. \end{aligned} \tag{5.24}$$

5.9.3.2 面向特定应用场景的签名方案

盲签名（Blind Signature）是由 David Chaum 于 1983 年提出来的 [8]，可以满足签名方对发送方的消息进行签名，却不知道消息的具体内容这一应用需求。一般的签名方案相当于在合同文本等材料上直接签名，而盲签名则相当于将文件放入信封，签名方在信封上签名，且并不知道文件的具体内容，签名的有效性可以在消息去盲化后被公开验证。盲签名也可被认为是一种特殊的隐形认证技术，使得发送方能够保证消息的完整性、真实性和隐私性。

5.9.3.2.1 盲签名的一般流程

盲签名一般只运行在两个实体之间，称为 clint 和 server，或者签名方（signer）和用户（user），盲签名的一般流程如图5–14所示。

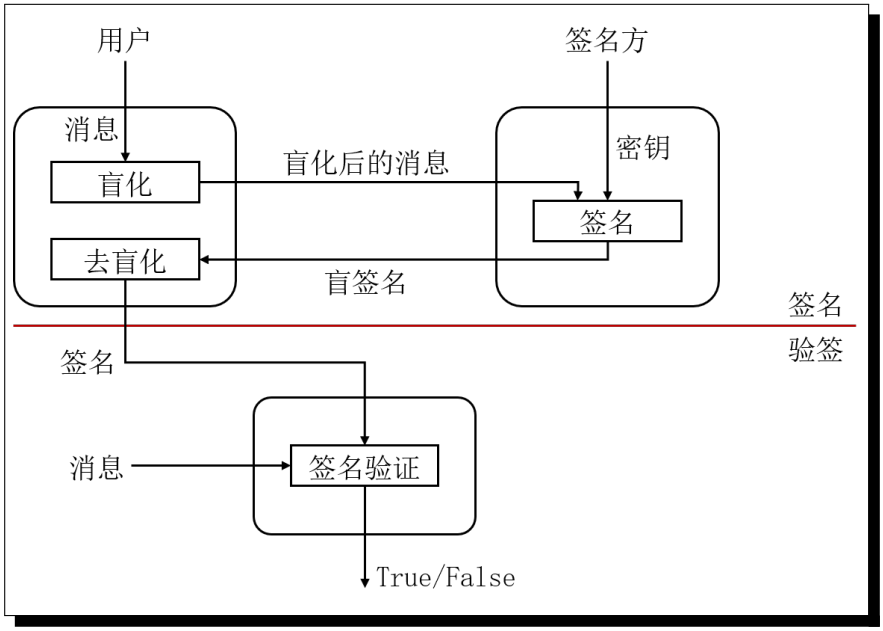

图 5–14 盲签名的基本流程

5.9.3.2.2 基于 RSA 算法的盲签名

目前，盲签名已经有多种实现方式，比如基于 RSA 算法的盲签名方案、基于 Schnorr 的盲签名 [9]、基于 DSA 的具有消息恢复功能的盲签名方案 [10]、基于离散对数问题的盲签名 [11]、基于身份的盲签名 [12]、BLS 盲签名方案 [13]、基于 SM9 数字签名的盲签名等。其中，基于 Schnorr 的盲签名是一个短签名方案，是 ElGamal 签名方案的变形，其安全性基于离散对数困难性和哈希函数的单向性。BLS 盲签名方案由 Boneh、Lynn、Shacham 于 2004 年提出，允许验证方用接收方提供的公钥对消息进行盲签名。

下面以基于 RSA 算法的盲签名（一般简称为：RSA 盲签名）方案为例，假设 A 是签名接收方，B 是签名方，A 让 B 在消息 m 上签名，B 的私钥为 (n, d)，并公开其 RSA 公钥 (n, e)。具体的签名流程为：

（1）对消息 m 进行盲化（相当于把文件装进信封）：A 选择盲因子 r，并计算 $m' = m \cdot r^e \mod n$。
（2）B 对 m' 进行签名（相当于在信封上签名）：$m'^d = (m \cdot r^e)^d \mod n$。
（3）A 去盲化，得到原始签名（相当于原本在信封上的签名渗透信封，落在信封里的文件上）：$s = m'^d \cdot r^{-1} = (m \cdot r^e)^d \cdot r^{-1} \mod n = m^d \cdot r^{ed} \cdot r^{-1} \mod n = m^d \mod n$，这样，B 就在不知道原始消息的情况下对原始消息进行了签名。

一般来说，盲签名方案有两个步骤：加密（encryption）和解密（decryption）。该方案实际上通过将信息加密，然后由签名方进行有效签名，最后使用解密来获得签名信息，使得签名方既不知道签名信息的内容及签名接收方的实际身份，也不可能改变签名的内容，而消息的真实拥有方（即上述方案中的 A）能够轻松获取其签名信息，更加安全。但也需要注意，基于 RSA 的盲签名存在的一个问题是：通过 RSA blinding attack，可能会利用盲签名将一个消息解密为另一个消息。

此外，相比传统的签名方案，盲签名具有明显的优势：

- 保护隐私：传统签名方案需要签名方知道被签名消息的内容，这可能导致隐私泄露。而盲签名方案中，签名方无法知道被签名消息的内容，从而有效地保护了数据隐私。
- 提高安全性：由于盲签名的不可追踪性，使得它比传统签名方案更难以被恶意攻击者获取或篡改，这使得盲签名在保护数据安全方面更具优势。
- 支持广泛的应用场景：盲签名可以应用于许多需要保护用户隐私和数据安全的领域，例如电子投票、数字货币等。

多重签名（multisignature）是指多个用户同时对一个消息签名，表现形式是 m/n，也就是说：共有 n 个私钥有签名权限，若有其中的 m 个私钥进行了签名，则该签名就是有效的，且生成的签名比各个签名人独立签名生成的签名值集合更加简短。比如，在电商领域，该值常取为 2/3。当表现形式为 1/1 时，对应的就是传统的用私钥签名、用公钥验签的签名模式。多重签名机制可用于区块链等对多方签名有需求且对签名长度敏感的应用场景。

5.9.3.2.3 多重签名的分类

多重数字签名一般分为有序多重数字签名方案和广播多重数字签名方案。和多重签名机制紧密相关的签名压缩机制还有聚合签名（aggregate signature）。聚合签名可以将多个签名压缩为一个签名。聚合签名进一步可分为通用聚合签名和顺序聚合签名等。顺序聚合签名中签名和聚合操作根据已经签名的消息集、已经聚合的签名以及待签名新消息，利用私钥进行签名并完成聚合操作。

（1）有序多重签名

消息发送方预先设计一种签名顺序，并将这种签名顺序发送到每一位签名方。然后将待签消息 m 发送到第一个签名方，第一个签名方签名后，将消息发送给第二个签名方。从第二个签名方开始，每一位签名方收到签名消息后，首先验证上一签名的有效性，如果签名有效，则继续签名，并将签名消息发送到下一个签名方；如果签名无效，则拒

绝对消息签名，终止整个签名过程。当签名验证方收到签名消息后，验证签名的有效性，如果有效，多重签名有效；否则多重签名无效。

（2）广播多重签名

消息发送方同时将待签消息 m 发送给每一位签名方进行签名，然后将签名的结果发送到签名收集者，由收集者签名对消息进行整理并发送给签名验证方。签名验证方验证多重签名的有效性。由于广播多重签名不用考虑签名的顺序，因此其应用更为广泛。

5.9.3.2.4 基于 RSA 的广播多重签名

RSA 算法的参数约定参见5.7.1节（第73页），此外，用 $H(\cdot)$ 表示一个理想的哈希函数，m 表示待签的消息，ID_i 表示签名方的身份并且是公开的。假设签名方的数量为 t，则基于 RSA 的广播多重签名的生成和验证过程如下 [14]：

（1）每个签名方 i 计算 $h_i = H(ID_i)$ 和 $s_i = h_i^{-d} \mod n$，然后选择一个随机数 r_i，并计算 $R_i = r_i^e \mod n$，然后把 R_i 发送给接收方。
（2）接收方计算 $T = R_1 * \cdots * R_t \mod n$，并把 T 广播出去。
（3）每个签名方 i 计算 $hm = H(T, m)$ 和 $D_i = r_i s_i^{hm} \mod n$，并把 D_i 发送给签名收集方。
（4）签名收集方计算 $D = D_1 D_2 \cdots D_t$，则广播多重签名为 (D, T, hm)，签名收集方将多重签名发送给接收方。
（5）接收方收到消息 m' 和多重签名后，首先计算 $h_i = H(ID_i)$，$i \in [t]$，然后计算 $T' = D^e (h_1 h_2 \cdots h_t)^{hm}$ 和 $hm' = H(T', m')$，最后验证 $hm' = hm$ 是否成立，如果成立，则接受该签名。

上述多重签名生成和验证过程中，签名方、接收方和签名收集方之间存在较多交互，且过程中涉及较多计算。

5.9.3.2.5 Bellare-Neven（BN）多重签名

BN 多重签名方案是由 Bellare 和 Neven 于 2006 年提出来的 [15]。用 sk_i 表示私钥，与之对应的公钥为 $\mathsf{pk}_i = \mathsf{sk}_i \cdot G$，$G$ 为椭圆曲线上的基点，用 m 表示待签名的消息，并选定一个理想的哈希函数 $H(\cdot)$，设签名方的数量为 t，则 BN 多重签名方案如下：

（1）令 $L = H(\mathsf{pk}_1, \mathsf{pk}_2, \cdots)$，$PK = \sum_{i=1}^{t} \mathsf{pk}_i$。
（2）每一个签名方 i 选择一个随机数 r_i，计算 $R_i = r_i \cdot G$，并把 R_i 发送给其他签名方。令 $R = \sum_{i=1}^{t} R_i$。
（3）每一个签名方 i 计算 $s_i = r_i + H(L, pk_i, R, m) \cdot sk_i$，并把 s_i 发送给签名收集方。
（4）签名收集方计算 $s = \sum_{i=1}^{t} s_i$，得到多重签名为 $\sigma = (R, s)$，并把该多重签名发送给接收方。
（5）接收方验证等式 $s \cdot G = R + H(L, pk_1, R, m) \cdot PK + H(L, pk_2, R, m) \cdot PK + \cdots + H(L, pk_t, R, m) \cdot PK$ 是否成立，如果成立，则接受该签名。

从步骤（5）中可以看出，BN 多重签名方案不具有密钥聚合性，因此在验签过程中会涉及到较多计算量。

5.9.3.2.6 基于 Schnorr 签名的多重签名

目前已经有多种基于 Schnorr 签名的多重签名方案，比如 MuSig [16] 等。MuSig 在组合方面很新颖：支持密钥聚合。该方案有两个版本，即两轮 MuSig 和三轮 MuSig，两者都可以证明是安全的，但三轮 MuSig 仅依赖于离散对数 (DL) 假设，而两轮 MuSig 依赖于稍强的 One-More Discrete Logarithm (OMDL) 假设。

Schnorr 签名具有可证明安全性、线性、效率高、占用空间小等优点。这里沿用上面的参数定义，介绍一种简单的 Schnorr 多重签名（Simple Schnorr Multi-Signatures，SSMS）方案 MuSig：

（1）令 $L = H(pk_1, pk_2, \cdots)$，$PK = \sum_{i=1}^{t} H(L, pk_i) pk_i$。
（2）每一个签名方 i 选择一个随机数 r_i，计算 $R_i = r_i \cdot G$，并把 R_i 发送给其他签名方。令 $R = \sum_{i=1}^{t} R_i$。
（3）每一个签名方 i 计算 $s_i = r_i + H(PK, R, m) H(L, pk_i) \cdot \mathsf{sk}_i$，并把 s_i 发送给签名收集方。
（4）签名收集方计算 $s = \sum_{i=1}^{t} s_i$，得到多重签名为 $\sigma = (R, s)$，并把该多重签名发送给接收方。
（5）接收方验证等式 $s \cdot G = R + H(PK, R, m) \cdot PK$ 是否成立，如果成立，则接受该签名。

与 BN 多重签名方案不同，SSMS 方案中将 PK 定义成为这些公钥的倍数之和，其中乘数取决于所有参与密钥的哈希。SSMS 在普通的公钥模型中可使用密钥聚合，并具

备应有的安全性，其与标准的 Schnorr 签名具有相同的密钥和签名大小。

新近进展方面，2020 年，Nick 等人 [17] 提出一种具有可验证确定性随机数的 Schnorr 多重签名，称为 MuSig-DN；2021 年，Nick 等人 [18] 进一步提出一种称为 MuSig2 的两轮简单多重签名方案；2022 年，Boschini 等人 [19] 提出一种基于格的具有单轮在线阶段的多重签名方案，称为 MuSig-L。

5.9.3.2.7 多重签名应用示例

多重签名可以用于多个应用场景，举例如下：

- 加密数字货币：因为在加密数字货币领域，如何保证交易安全是一大难题，因此，加密数字货币（比如比特币、以太坊、门罗币等）成为多重签名的一个重要应用领域–不论是加密货币交易平台、在线钱包，还是第三方货币托管运营机构，大多都会采用多重签名的方式来保障资金安全，降低黑客攻击风险。
- 办公自动化：当企业需要进行决策或一份文件需要几个单位或部门分别签字才有效时，可以使用多重签名来实现快速办公，签名方可以通过签署或拒签来表达是否认可该决策或文件。
- 电子金融：当出现一笔交易或转账时，使用多重签名可以有效的避免出现错误，其他私钥持有者可以在发现错误的第一时间通过拒绝签名的方式阻止该错误交易。
- CA（Certificate Authority，认证中心）认证：CA 认证即电子认证服务，是由负责发放和管理数字证书的认证中心为信任背书的认证服务。在此场景中，多重签名可以有效的避免认证错误，使认证更具权威性。

群签名（group signature）是 1991 年由 Chaum 和 Van Heyst [20] 提出来的，其允许多个签名人形成一个签名人群组，群组中的任意一个成员可代表整个群组匿名地生成对某个消息的签名（“匿名”表示验签人无法判断生成签名的具体群成员的身份），可以验证签名来自于该群组，但无法准确追踪到签名方是群组中的哪个成员。为了添加新的群成员，需要一个群管理员，群管理员负责维护群组中成员的群组资格，并在必要时识别生成某个签名的签名人身份，因此存在群管理员能够追踪到签名方身份的风险。

5.9.3.2.8 群签名的安全性要求

群签名的安全性要求包括：

- 完整性（soundness and completeness）：有效的签名能够被正确验证；

- 不可伪造性（unforgeability）：只有群成员才能产生有效的群签名，其他任何人包括群管理员都不能伪造一个合法的签名。
- 匿名性（anonymity）：给定一个群签名后，对除了唯一的群管理员以外的任何人来说，确定签名方的身份是不可行的，至少在计算上是困难的。
- 可跟踪性（traceability）：在发生纠纷的情况下，群管理员可以打开一个签名来确定出签名方的身份，而且任何人都不能阻止一个合法的签名被打开。
- 不关联性（unlinkability）：在不打开签名的情况下，确定两个不同的签名是否为同一个群成员所签的是不可行的，至少在计算上是困难的。
- 无框架（no framing）：即便群成员和群管理员串通，也无法创造出非本群组成员创建的新签名（即不在群组中的签名方无法生成群签名）；
- 不可伪造的跟踪验证（unforgeable tracing verification）：群管理员不能错误地指责签名方创建他没有创建的签名。
- 抵抗联合攻击（coalition-resistance）：即使一些群成员串通在一起也不能产生一个合法的不能被跟踪的群签名。

群签名机制的变体包括：

- 可追踪群签名（traceable group signature），其允许授权方追踪某个成员的签名而不暴露其他成员的身份信息及其生成的签名；
- 可追责的追踪群签名（accountable tracing signature），其可限制群组管理员滥用追踪群成员签名的能力。该群签名机制可用于有隐私保护需求的诸多签名应用中。

5.9.3.2.9 群签名的一般流程

群签名的一般流程如图5–15所示。

图5–15中的流程可描述如下：

（1）初始化：群管理员建立群资源，生成对应的群公钥（Group Public Key，GPK）和群主私钥（Group Main Secret Key，GMSK），群公钥对整个系统中的所有用户（比如群成员、验证方等）公开。

（2）群成员加入：当有用户要加入群时，群管理者颁发群证书（group certificate）给该用户，使其成为群成员。

（3）生成群签名：群成员利用获得的群证书签署文件，生成群签名。

（4）验证群签名：验证方利用群公钥仅可以验证所得群签名的正确性，但不能确定群签名的正式签署方是哪些群成员。

（5）打开（或追踪）签名方信息：群管理元利用群主私钥可以对群成员生成的群签

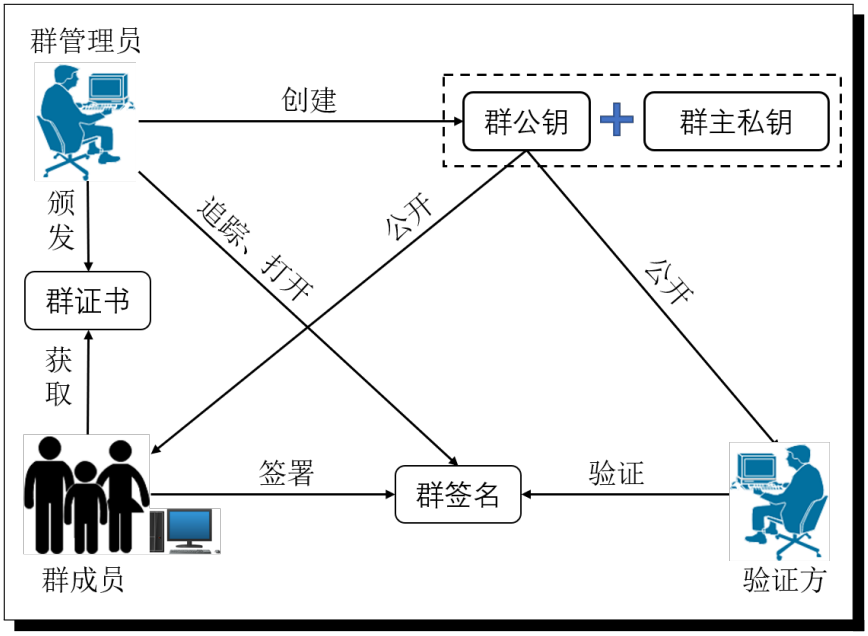

图 5–15 群签名的一般流程

名进行追踪，并在需要时揭露签名方的身份。

环签名（Ring Signature）是与群签名类似的一种数字签名方案，由 Rivest、Shamir 和 Tauman 于 2001 年最先提出 [21]。环签名是一种简化的群签名，和群签名不同的是，环签名机制中没有环成员管理机制，因此也就没有环管理员，只有环成员，且不需要环成员之间的协作，环上任意成员可以匿名地代表整个环成员完成签名操作，从而不能确认某个签名的具体签名人的身份。这意味着环签名比群签名在签名人隐私保护方面更为彻底。

环签名可以实现签名方无条件签名，其主要特性包括：

- 完全匿名性：其他人仅可验证环签名的有效性，无法获得签名方的身份信息；
- 不可追踪性：无法追踪签名对应的签名方的信息。

5.9.3.2.10 环签名的算法组成

假设有 n 个参与方，则环签名主要以下几个算法组成：

（1）密钥生成算法 $\{(\mathsf{sk}_i, \mathsf{pk}_i)\} \leftarrow \mathrm{Gen}(1^\lambda)$：一个 PPT 算法，输入为系统安全参数 λ，输出为公私钥对。假定 Gen 函数为每一个参与方 P_i 生成的私钥为 sk_i，与之对应的公钥为 pk_i，不同参与方的公私钥可能来自不同的公钥密码体制。

（2）签名算法 $\sigma \leftarrow \mathrm{Sign}(m; \mathsf{pk}_1, \cdots, \mathsf{pk}_n; \mathsf{sk}_s)$：一个 PPT 算法，输入为消息 m 和 n

个环成员的公钥以及其中一个成员的私钥 sk_s，输出为对消息 m 的签名 σ，σ 中的某个参数根据一定的规则呈环状。具体实现方面，签名方首先选定一个临时的签名方集合，集合中包含签名方自身，然后签名方使用自己的私钥和签名集合中的其他成员的公钥就可以独立地生成签名，而无需其他成员的帮助。

（3）验证算法 $\text{True/False} \leftarrow \text{Verify}(m, \sigma)$：一个确定性算法，输入 (m, σ) 后，若 σ 为 m 的环签名，则输出“True”，否则输出“False”。

环签名因为其签名隐含的某个参数按照一定的规则组成环状而得名，但之后提出的许多方案中不要求签名的构成结构呈环形，只要签名的形成满足自发性、匿名性和群特性，也称之为环签名。

5.9.3.2.11 与群签名的比较

与群签名相比，两者的异同如表5–4所示：

表 5–4 环签名与群签名的对比

比较项	群签名	环签名
匿名性	两种方案都是个体代表群体签名的体制，验证方能验证签名为群体中某个成员所签，但并不能知道是哪个成员，从而可达到签名方匿名的效果。	
可追踪性	存在群管理员，因此可保证签名的可追踪性。群管理员可以撤销签名，揭露真正的签名方。	本身无法揭示签名方，除非签名方本身想暴露或者在签名中添加额外的信息。
签名管理	由群管理员管理。	不需要管理，签名方只有选择一个可能的签名方集合，获得其公钥，然后公布这个集合即可，所有成员平等。

环签名机制有诸多变体，比如：

- 门限环签名（threshold ring signature）：结合环签名和门限签名的功能，既保护签名方的隐私又防止小部分环中成员滥用签名能力；
- 可关联环签名（linkable ring signature）：可以确定两个签名是否由同一个签名方生成（但不能确定具体签名方的身份）；
- 可追踪环签名（traceable ring signature）：可以确定生成了两次或两次以上签名的某个签名方的身份。

和环签名相近的还有网签名（mesh signature），该机制允许在无公钥的情况下，将某人变成环上的成员。环签名机制有众多应用场景，包括隐私保护、电子现金、电子选举等。

5.9.3.3 Camenisch-Lysyanskay 签名方案

Camenisch-Lysyanskaya 签名简称为 C-L 签名，于 2004 年提出 [22]，由提出者的名字来联合命名。C-L 签名可用于群签名或聚合签名的场景中，提高签名的匿名性，并降低签名的计算复杂度。C-L 签名也是一种适用于零知识证明的签名方案，能够对一组数据进行签名，并且能够体现这些被证明组件的关系，这样的性质恰好与零知识证明所需的性质相契合。

C-L 签名中用到了双线性映射的概念：

定义 5.3 设 $\mathbb{G}_1$ 和 $\mathbb{G}_2$ 是阶为质数 p 的循环群，g 是 $\mathbb{G}_1$ 的一个生成元，则双线性群满足映射关系 $e:\mathbb{G}_1*\mathbb{G}_1\to\mathbb{G}_2$，且该映射满足如下性质：

- 双线性性：对于 $\forall a,b\in\mathbb{Z}_p$，有 $e(g^a,g^b)=e(g,g)^{ab}$；
- 非退化性：$e(g,g)\neq 1$；
- 可计算性：对于 $\forall u,v\in\mathbb{G}_1$，存在有效的算法来计算 $e(u,v)$。

这种具有双线性性质的二元映射也称为双线性对（Bilinear Pairing），是密码学算法的基本构造工具之一，零知识证明、聚合签名等技术方案大多基于双线性对来构造。

C-L 签名包括如下三个算法：

（1）密钥生成算法 $\mathrm{KeyGen}(1^\lambda)$：设 $\mathbb{G}_1$ 和 $\mathbb{G}_2$ 是阶为质数 p 的双线性群，$e:\mathbb{G}_1*\mathbb{G}_1\to\mathbb{G}_2$，$\mathbb{G}_1=\langle g\rangle$。随机选择 $x,y\in\mathbb{Z}_p$，令 $X=g^x$，$Y=g^y$，输出私钥 $\mathsf{sk}=(x,y)$，公钥 $\mathsf{pk}=(p,\mathbb{G}_1,\mathbb{G}_2,e,g,X,Y)$。

（2）签名算法 $\mathrm{Sign}(m,\mathsf{sk})$：输入消息 $m\in\mathbb{Z}_p^*$ 和私钥 sk，选择一个随机数 $a\in\mathbb{G}_1$，计算 $b=a^y,c=a^xb^{xm}$，输出签名 $\sigma=(a,b,c)$。

（3）验证算法 $\mathrm{Verify}(\sigma,m,\mathsf{pk})$：输入对消息 m 的签名 σ，使用公钥进行如下验证：如果 $e(a,Y)=e(b,g)$ 且 $e(c,g)=e(a,x)\cdot e(b,x)^m$，则输出 1（表示通过验证），否则输出 0（表示未通过验证）。

5.10 伪随机生成器

在密码学中，使用随机数来保持随机性是保证系统安全的重要手段。随机数可分为真随机数和伪随机数。真随机是指自然界真实发生的随机事件，真随机数就是描述自然

界真实发生的随机事件的数字。与之相对应的，伪随机数是指计算机模拟自然界产生随机数的机理产生的随机数。

伪随机生成器（Pseudo-random generator，PRG）是一种确定性的算法或函数，它能够根据一个初始的种子值（seed）生成一个看似随机的数字序列。虽然这些序列并非真正的随机，因为它们是由确定的算法和种子决定的，但在许多应用场景中，它们具有与真正随机序列相似的统计特性，足以满足各种需要随机数的任务。

在密码学应用中，伪随机生成器的安全性至关重要。一个安全的伪随机生成器应该能够抵抗各种攻击，如预测攻击、统计攻击等。即攻击者在不知道种子的情况下，难以从生成的伪随机序列中推断出后续的数字或种子本身。对随机数的性质可分为如下三个逐步加强的层次 [6]，这也是伪随机生成器的评估指标：

- **随机性**（randomness）：生成的随机数列看起来是完全杂乱无章的数列，不存在统计学上的偏差/规律性。满足该性质的随机数可称为“弱伪随机数”。
- **不可预测性**（unpredictability）：不能通过已生成的随机数列推测出下一个随机数。满足该性质的随机数可称为“强伪随机数”，即：攻击者在知道过去生成的伪随机数列的前提下，仍然无法预测出下一个要生成出来的伪随机数。
- **不可重现性**（non-reproducibility）：不能重现出相同的随机数列，除非将已生成的随机数列保存下来。满足该性质的随机数可称为“真随机数”。仅依靠软件是无法生成具备不可重现性的随机数列的，需要结合从硬件中获取的信息，比如：热噪声、传感器收集的热量变化、声音的变化、键盘输入的时间间隔等。

密码学中使用的随机数，至少要满足不可预测性，仅满足随机性是不够的。

5.10.1 伪随机数生成器（PRNG）

通过特定的硬件设备可基于无法预测和重现的自然现象的信息生成随机数列，这种硬件设备称为随机数生成器（Random Number Generator，RNG）。而在绝大多数实际应用中是通过软件来生成随机数的，这种软件称为伪随机数生成器（Pseudorandom Number Generator，PRNG），其通过一系列种子值计算出伪随机数。伪随机数只是尽可能地接近其应具有的随机性，但是因为有“种子值”，所以伪随机数在一定程度上是可控可预测的。

伪随机数生成器的结构如图5–16所示。

构建伪随机数生成器的主要方法包括：杂乱法、线性同余法、哈希函数法、密码法、基于 ANSI X9.17 的方法等，对这些方法的具体介绍参见文献：结城浩，《图解密码技术（第 3 版）》，周自恒译，人民邮电出版社，2016.6。

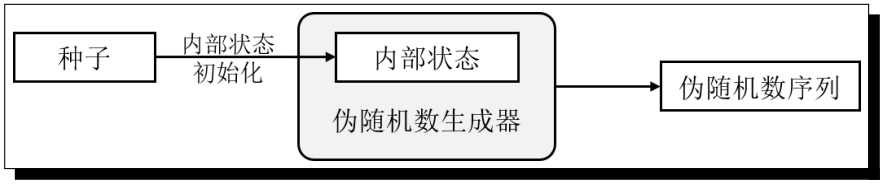

图 5–16 伪随机数生成器结构

5.10.2 伪随机相关生成器（PCG）

伪随机相关生成器（Pseudorandom Correlation Generator，PCG）是 E. Boyle 等人 [23] 提出来的，其本质是通过“少量的通信”使参与方得到“短的种子”，随后参与方使用这些种子生成“长的随机串”（即相关随机数）。在 MPC 应用中，使用 PCG 可使得“离线阶段”不再需要存储大量的相关随机数，只需要存储少量的 PCG 种子即可。当“在线阶段”需要消耗相关随机数时，各方不需要任何通信交互，可以直接通过 PCG 种子产生所需的相关随机数。

两方的伪随机相关生成器由 PCG.Gen 和 PCG.Extend 两个算法构成（如图5–17所示）：

（1）PCG.Gen(1^λ)：随机算法，给定系统安全参数 1^λ，输出一对种子 $(\text{seed}_0, \text{seed}_1)$。PCG.Gen 可以由可信第三方或通过两方协议来实现。

（2）PCG.Extend(b, seed_b)：确定性算法，给定 $b \in \{0, 1\}$ 和种子 seed_b，输出满足 $|\text{seed}_b| < |R_b|$ 的伪随机串 R_b。

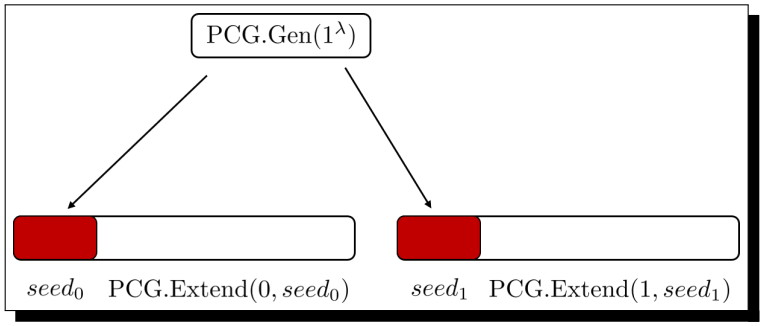

图 5–17 伪随机相关生成器构成示意

5.11 伪随机函数与伪随机置换

5.11.1 伪随机函数（PRF）

伪随机函数（Pseudorandom Function，PRF）使用一个确定性算法，计算出看似随机的数据序列。如前所述，相比于真随机，伪随机是指函数的输出看起来是随机的，但是如果输入的初始值（随机数种子）是相同的，那么输出的“随机数”的数值也是相同的。

与伪随机数生成器相比，伪随机函数中增加了对输入输出的描述，这里使用 K 表示密钥空间，X 表示输入空间，Y 表示输出空间，若对于随机选定的密钥 $k \in_R K$ 和输入 $x \in X$，函数 f 的输出 $y = f(k,x) \in Y$ 看上去“像”是随机的，则称函数 f 是一个定义在 X 到 Y 上的伪随机函数。

5.11.2 伪随机置换（PRP）

伪随机置换（Pseudorandom Permutation，PRP）是一种在密码学中常见的随机函数，经常用在加密方案中。在伪随机函数的基础上，伪随机置换中引入了“映射可逆”的要求，即：对于伪随机置换函数 f_P，对于给定的输入 x，通过计算 $f_P(k,x)$ 可以获得输出 y。同时，给定同样的密钥 k 和 y，可以通过计算 $f_P^{-1}(k,y)$ 得出 x。

伪随机置换函数具有高效性，因为其正向计算是多项式级别的，同时其逆向计算也是多项式级别的。因此，即使存在一个多项式资源的敌手或区分器，也无法猜出或建模出某个伪随机置换函数 f_P 的映射方式。

在应用中，伪随机置换可以保护原始数据的隐私性和安全性。例如，在公钥密码体制中，接收方可以通过使用伪随机置换来对发送方的公钥进行加密，从而使得只有拥有私钥的发送方才能解密得到原始数据。

5.11.3 可编程伪随机函数（PPRF）

可编程伪随机函数（Programmable PRF）可将一组点集 $(\mathbf{x},\mathbf{y})$ 编码到伪随机函数中，输入 $\mathbf{x}$，可得到 $\mathbf{y}$。PPRF 的算法组成如下：

（1）$\mathrm{KeyGen}(1^\lambda, P) \to (k, \mathrm{hint})$：给定系统安全参数 1^λ 和一个点集 P，生成 PRF 密钥 k 和公开的辅助信息 hint。

（2）$f(k, \mathrm{hint}, \mathbf{x}) \to \mathbf{y}$：在输入的 $\mathbf{x}$ 上计算 PRF 值，给出输出 $\mathbf{y}$。

其中的 hint 可看作是一个与输入有关系的随机数。

5.12 过滤器（Filter）

除了作为生成消息摘要的强有力工具外，哈希算法还展现出了在优化查询处理流程中的独特价值，尤其是在对数据库查询进行高效过滤方面。以查询数据库为例，当我们从数据库中检索数据时，要查询的数据可能在数据库中，也可能不在数据库中。面对海量的查询请求，若其中相当一部分是针对数据库中根本不存在的数据项发起的，那么这些请求无疑会迫使数据库执行大量不必要的 I/O 操作，造成资源的极大浪费。特别值得注意的是，当此类无效查询的数量激增时，数据库的 I/O 性能将承受巨大的压力，绝大部分处理能力被耗费在了响应这些徒劳无功的查询上，严重制约了系统整体的响应速度和效率。

为了缓解这一困境，引入基于哈希算法的查询过滤机制显得尤为重要。该类机制能够巧妙地预先评估查询请求的有效性，通过计算请求数据项的哈希值，并与数据库中现有数据的哈希索引进行快速比对，从而迅速识别并过滤掉那些针对不存在的数据项发起的查询。这样一来，数据库便能专注于处理那些真正有意义的、针对已存在数据的查询请求，显著减少不必要的 I/O 负担，极大地降低系统的运行压力，提升整体性能和用户体验。因此，使用过滤器无疑是提升数据库管理效率和响应速度的关键策略之一。

5.12.1 布隆过滤器

布隆过滤器（Bloom Filter，BF）是 1970 年由 Burton Howard Bloom [24] 提出来的，它实际上是一个很长的二进制向量（bitmap，位图，由二进制位组成的数组）和一系列随机映射函数（哈希函数）。其大致思路是：当需要检索一个元素是否在一个数据集合中时，首先查询布隆过滤器中是否存有该元素的信息，有的话，再进一步去查询数据集合，没有的话，则直接返回元素不存在的信息，这样就过滤掉了无意义的检索请求。

5.12.1.1 技术原理

在判断一个元素是否存在于某个数据集合（比如数据库）中，传统方法是将该元素与数据集合中的所有元素进行比较来确定。这一策略在链表、树结构、哈希表等数据结构中都有所体现。然而，随着集合规模的扩大，元素的累积不仅意味着存储需求的显著增加，还直接导致了检索效率的下降，因为每一次查询都可能需要遍历更多的元素。为了解决这个问题，布隆过滤器巧妙地运用了哈希映射技术，以一种间接且高效的方式表示集合中的元素，从而快速响应元素存在性的查询请求。

但是，布隆过滤器在提供高效查询能力的同时，也存在一定的局限性，因为一般来说，它无法直接返回被查询元素的具体信息或值，仅能告知该元素是否“可能”存在于

数据集合中。

布隆过滤器的工作过程如下：

（1）初始化时，需要一个长度为 n 比特的数组（或者向量），每个比特位初始化为 0，如图5–18所示。

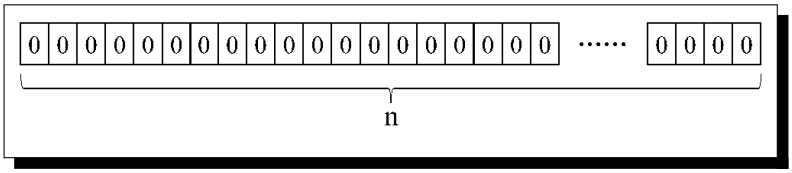

图 5–18 布隆过滤器初始化

（2）然后准备 k 个理想的哈希函数 $H_1, \cdots, H_k : key \rightarrow [n]$，这些哈希函数可以把 key 值映射为 $[n]$ 中的一个整数，n 为布隆过滤器的比特位长度。

（3）当想要把某个 key 值加入数据集合中时，使用 k 个哈希函数计算出 k 个哈希值，并把比特数组中对应的位置上的值由 0 变为 1。作为示意，下面以三个哈希函数 Hash1、Hash2 和 Hash3 为例，先后将“Top”和“Security”这两个 key 值加入数据集合中，如图5–19所示。

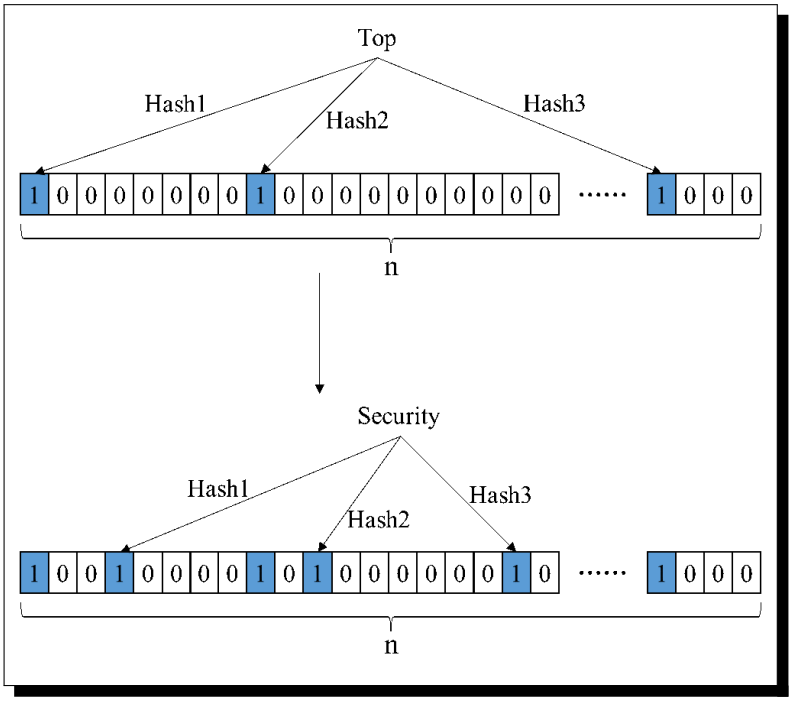

图 5–19 将 Key 值加入数据集合时的布隆过滤器操作

（4）判断某个 key 值是否在数据集合中时，用 k 个哈希函数计算出 k 个哈希值，并

查询比特数组中对应的比特位，如果全部比特位的值都为 1，则该 key 值**很可能**在数据集合中，如果有比特位是 0，则该 key 值一定不在该数据集合中。如图5–20所示，当判断 safety 是否在数据集合中时，发现有一个比特位的值为 0，于是可以断定 safety 不在数据集合中。

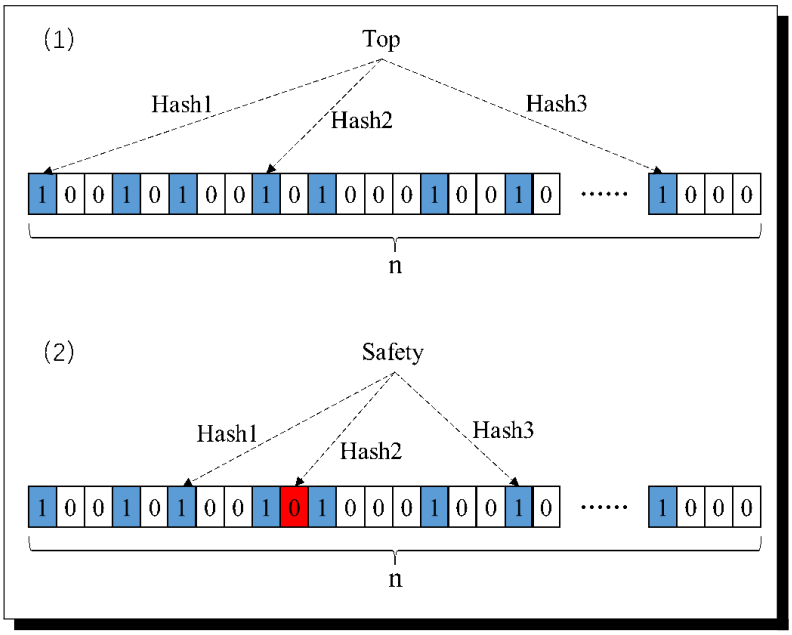

图 5–20 判断 Key 值是否在数据集合中时的布隆过滤器操作

布隆过滤器的优点是空间效率和查询时间都比一般的算法好得多，缺点是随着加入集合的 key 值越来越多，被置为 1 的比特位也会越来越多，可能存在这样的情况：一个 key 值并不在集合中，但全部 k 个哈希函数返回的比特位都已被其他 key 值置为 1，这样就起不到过滤的作用了。因此，使用布隆过滤器时，可能会出现误报。显然，布隆过滤器的比特位长度会直接影响误报率，布隆过滤器越长，误报率会越低，因此，在实际应用中，比特数组长度 n 的值要尽量设置大一些。

另外，布隆过滤器还存在“删除困难”的问题，因为布隆过滤器使用了有限大小的比特位向量（比特数组），当要加入的 key 值较多时，哈希映射过程几乎必然会出现哈希碰撞，即比特数组中的一位可能被映射了两次甚至多次，所以，即使存储的元素已经从数据集合中删除，也不能从布隆过滤器中删除该元素（即不能将该元素映射到的位置上的值由 1 变为 0），否则可能会出现查询错误。

5.12.1.2 典型应用

在海量数据查询中，布隆过滤器有优异的空间效率和低误判率，因此有着非常广泛的应用。除了对数据查询进行优化外，布隆过滤器的其它典型应用还包括：

- 数据去重：在用户提交表单时，可以使用布隆过滤器快速判断该用户是否已经提交过相同的数据，从而防止重复提交；在数据同步过程中，利用布隆过滤器检测重复数据，避免不必要的数据冗余。
- 缓存穿透防护：当缓存中不存在某个数据，而用户频繁查询该数据时，可能会导致缓存穿透，即所有请求都直接穿透到数据库层，给数据库造成巨大压力。布隆过滤器可以在缓存层之前迅速过滤掉不存在的数据，从而减轻数据库的压力。
- 垃圾邮件过滤：垃圾邮件过滤系统可以使用布隆过滤器来存储已知的垃圾邮件发送者的地址或特征，当新邮件到达时，先通过布隆过滤器快速判断该邮件是否可能来自垃圾邮件发送者，从而提高垃圾邮件的识别效率。
- 搜索爬虫 URL 去重：在网络爬虫应用中，布隆过滤器可以帮助爬虫快速判断某个 URL 是否已经被抓取过，从而避免重复请求同一 URL，提高爬虫的效率和效果。
- IP 黑名单/白名单维护：在 Web 应用中，可以使用布隆过滤器维护一份 IP 黑名单或白名单，快速拦截恶意请求或放行正常请求，提高系统的安全性和稳定性。
- 系统启动加速：在系统启动时，通过布隆过滤器判断某些热门数据是否在缓存中，可以提前加载这些数据到缓存中，从而减少冷启动时的缓存穿透问题，加速系统的启动过程。

实际应用案例包括：

- Google 著名的分布式数据库 Bigtable 中使用了布隆过滤器来查找不存在的行或列，以减少磁盘查找的 I/O 次数；
- Squid 网页代理缓存服务器在 cache digests 中使用了布隆过滤器；
- Venti 文档存储系统采用布隆过滤器来检测先前存储的数据；
- SPIN 模型检测器使用布隆过滤器在大规模验证问题时跟踪可达状态空间；
- Google Chrome 浏览器使用布隆过滤器来加速安全浏览服务。

5.12.2 混淆布隆过滤器

5.12.2.1 Dong 等人的混淆布隆过滤器

混淆布隆过滤器（Garbled Bloom Filter，GBF）是 2013 年由 Dong 等人 [25] 提出来的，作为对传统布隆过滤器的改进。与传统布隆过滤器相比，Dong 等人设计的混淆布

隆过滤器的每一个位置 $GBF[i]$ $(i \in [m])$ 不再是存放单个比特，而是一个 λ 比特长的字符串，m 是混淆布隆过滤器的长度，λ 应取较大的数值，比如取为系统安全参数（计算安全参数）。

具体来说，对于一个数据集合 $X = \{x_1, \cdots, x_n\}$，Dong 等人设计的混淆布隆过滤器的工作过程可表示为：对每一个元素 x_i $(i \in [n])$，使用 k 个哈希函数 $H_1, \cdots, H_k$ 将其映射到混淆布隆过滤器的 k 个位置 $\mathrm{GBF}[H_1(x_i)], \cdots, \mathrm{GBF}[H_k(x_i)]$，在这 k 个位置上都存放 λ 比特的字符串（即：首先计算 $H_j(x_i)$，并将该哈希值映射到 GBF 的一个位置上，并在该位置上存放一个长度为 λ 比特的字符串），且满足条件 $x_i = \oplus_j \mathrm{GBF}[H_j(x_i)]$，其中，$i \in [n]$，$j \in [k]$。所有未被映射到的位置将会被设置为随机的比特字符串，以保证每个位置上存储的数据满足均匀分布性和彼此之间的不可区分性。上述过程的示意图如图5–21所示。

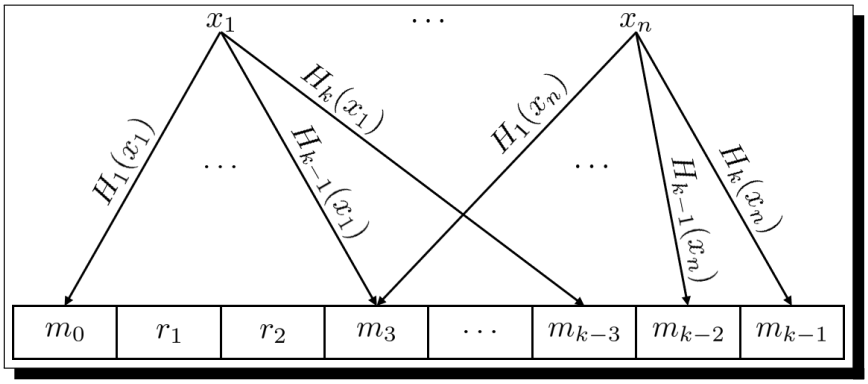

图 5–21 混淆布隆过滤器工作过程示意图

由于混淆布隆过滤器使用字符串数组和哈希函数进行操作，其插入和查询操作的时间复杂度通常与哈希函数的数量 k 成正比，即 $O(k)$。另外，相比于存储元素本身的数据结构（如哈希表、集合等），混淆布隆过滤器在空间效率上通常具有优势，因为它只需要存储少量的共享字符串即可表示大量的元素。

5.12.2.2 面向批量检索的混淆布隆过滤器

需要注意的是，混淆布隆过滤器具有灵活性和可扩展性。灵活性方面，可以根据实际应用的需求调整和优化上述混淆布隆过滤器，如通过改变哈希函数的数量、字符串的长度等参数来提高其性能或安全性，甚至设计不同数据结构的混淆布隆过滤器。实际上，Dong 等人设计的上述混淆布隆过滤器是为在半诚实敌手模型下实现两方的隐私集合求交协议而设计的；可扩展性方面，随着元素数量的增加，混淆布隆过滤器可能需要重新

构建或调整其参数以保持较低的误报率和较高的性能，这种调整通常相对简单且成本较低。

下面结合6.2节介绍的等重码，介绍一种基于关键词、面向批量匿踪查询（Private Information Retrieval，隐私信息检索）的混淆布隆过滤器的构造方法。首先将数据集转换为键值对（key-value）的形式，假设关键词（keyword，简写为 kw）的数量为 M，每个关键词（kw）对应的数据条目 d 至多为 $s(s \geq 1)$ 个，若某个关键词对应的数据条目不足 s 个，不足的部分使用 0 或 null 来填充。转换后的数据集记为

$$S = \{[kw_1, (d_1^1, \cdots, d_s^1)], [kw_2, (d_1^2, \cdots, d_s^2)], [kw_M, (d_1^M, \cdots, d_s^M)]\},$$

然后使用等重码映射算法将所有的关键词映射为等重码，同时，使用数据编码算法对各关键词对应的数据元素进行编码。于是，可将数据集 S 进一步转换成基于等重码的格式：

$$S_C = \{[CW_{kw_1}, (cd_1^1, \cdots, cd_s^1)], [CW_{kw_2}, (cd_1^2, \cdots, cd_s^2)], [CW_{kw_M}, (cd_1^M, \cdots, cd_s^M)]\},$$

其中，CW_{kw_i} 为关键词 kw_i 对应的等重码，cd_j^i 为数据 d_j^i 的编码值，$i \in [M]$，$j \in [s]$。

之后，将数据集映射到混淆布隆过滤器上。这里以将某个等重码 CW_{kw_i} 对应的 s 个数据元素映射到混淆布隆过滤器上为例，构造方式如下：

（1）使用 CW_{kw_i} 和哈希函数 $H_1, H_2, \cdots, H_s$ 进行计算（得到的哈希值记为 $h_1^i, \cdots, h_s^i \in \{0,1\}^m$），将等重码 CW_{kw_i} 映射到可验证布隆过滤器的 s 个位置，并将编码后的数据 $cd_1^i, \cdots, cd_s^i$ 存储在可验证布隆过滤器的这 s 个位置上。
（2）使用 CW_{kw_i} 和哈希函数 $H_{s+1} : \{0,1\}^m \to [N]$ 进行计算，得到 CW_{kw_i} 在该可验证布隆过滤器上对应的第 $s+1$ 个位置，在该位置上存入一个二进制串 b_{s+1}^i，用于对检索结果进行校验，该二进制串满足 $h_1^i \oplus \cdots \oplus h_s^i \oplus b_{s+1}^i = CW_{kw_i}$。
（3）重复进行上述步骤，从而将数据集映射到该混淆布隆过滤器上。
（4）对于可验证布隆过滤器上未被映射到的位置，使用 m 位长的随机值填充。

使用上述方法构建的混淆布隆过滤器具有可验证性，因此也可称为“可验证布隆过滤器”。

5.12.3 布谷鸟过滤器

5.12.3.1 布谷鸟哈希算法

布谷鸟哈希（Cuckoo Hashing）由 Rasmus Pagh 和 Flemming Friche Rodler 于 2001 年提出 [26]，其名称源于该哈希方法对应的行为与布谷鸟（大杜鹃）在别的鸟巢中下蛋，

孵出的小布谷鸟将别的鸟蛋挤出鸟巢的行为相类似。布谷鸟哈希算法并不特指一种哈希算法，而是一种哈希值的放置方法，旨在解决哈希表中的哈希值冲突问题，从而用较少的计算换取较大的哈希值存储空间。

布谷鸟哈希具有占用空间小、查询迅速等特性，可用于构建布隆过滤器和进行内存管理等使用场景中。目前，布谷鸟哈希算法常与普通哈希算法相结合，用于构造隐私集合求交（PSI）等 MPC 协议。

在布谷鸟哈希算法中，“桶”（Bucket 或 Stash）是一个核心概念，它是布谷鸟哈希表中的一个基本存储单元，用于存储数据的哈希值或相关键值对，一个桶可以看作是一个数组或列表。在布谷鸟哈希算法中，通常会有多个桶，这些桶共同构成了一个哈希表。布谷鸟哈希算法的目标是使用 k 个哈希函数 $H_1, H_2, \cdots, H_k : \{0,1\}^* \to \{C_i\}_{i\in[m]}$，将某数据方持有的元素放入 m 个桶（记为 $B[1], \cdots, B[m]$）中某一个桶的某个位置上。假设集合 X 的大小为 n，布谷鸟哈希的工作过程可以概括为：

（1）对于集合 X 中的某个元素 q，使用预设的 k 个哈希函数计算该元素的哈希值 $H_1(q), H_2(q), \cdots, H_k(q)$。
（2）这些哈希值对应各个桶中的位置，这些位置记为 $C = (C_1, C_2, \cdots, C_m)$，其中，$C_i$ 表示将该元素映射到第 i 个桶 $B[i]$ 的一个位置上。
（3）如果这些桶中有空位置，就将 q 放入其中一个桶的空位置中，结束插入；如果各桶中的相应位置上都有元素，则从中随机选择一个桶，踢出桶中相应位置上的元素 $\hat{q}$，并把 q 放入该位置上。
（4）循环将 $\hat{q}$ 插入到其它位置上，直到找到空位置，或者达到循环次数的上限。

研究 [27] 表明，当 $k = 3, 4, 5$ 时，桶的数量 m 分别取 $1.27n$，$1.09n$，$1.05n$ 能够使得布谷鸟哈希的插入失败概率不大于 2^{-40}。

下面以两个哈希函数、一个桶为例，进一步说明布谷鸟哈希的工作过程。此时，布谷鸟哈希算法可简记为：$C \leftarrow \text{Cuckoo}^{m=1}_{H_A,H_B}(X)$，具体工作过程如下：

（1）若插入 key_1 的两个位置（对应于用两个哈希函数计算出的两个哈希结果）均为空（$null$），则插入两者中序号较小的那个位置上，如图5–22所示。

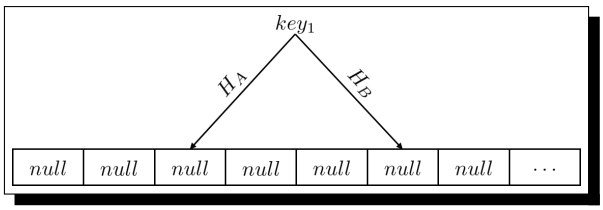

图 5–22 插入 key_1

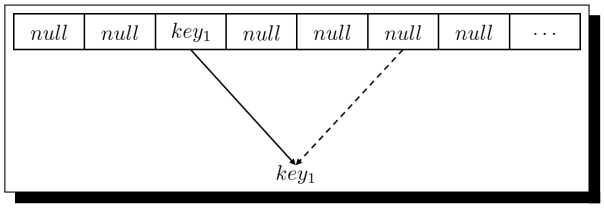

图 5–23 插入 key_1 后

（2）插入 key_1 后，效果如图5–23所示。

（3）此后，插入 key_2 时，若两个位置中有一个位置为空，则插入到空的位置中，如图5–24所示。

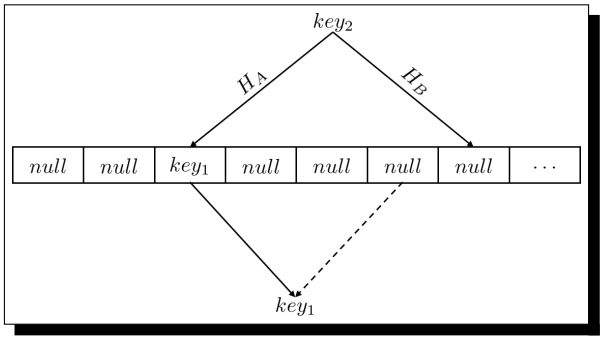

图 5–24 插入 key_2

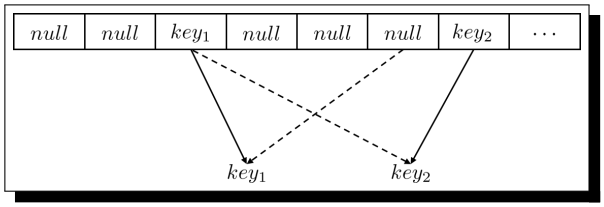

图 5–25 插入 key_2 后

（4）插入 key_2 后，效果如图5–25所示。

（5）当需要插入新 key 值 key_i 时，若发现对应的两个位置均被占用，如图5–26所示。此时，随机选择一个位置，踢出该位置上的 key 值（图5–27中以踢出 key_1 为例），将踢出的 key 值放置在 key_1 的另一个哈希结果对应的位置上。

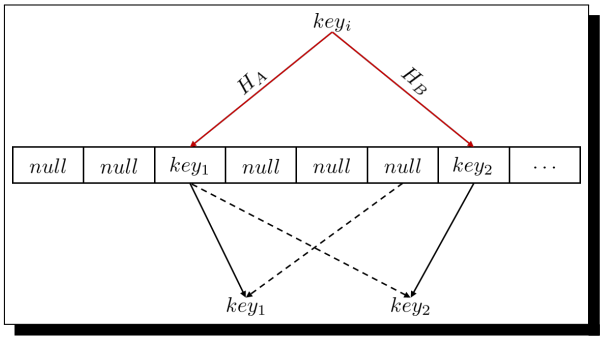

图 5–26 插入新 key 值 key_i

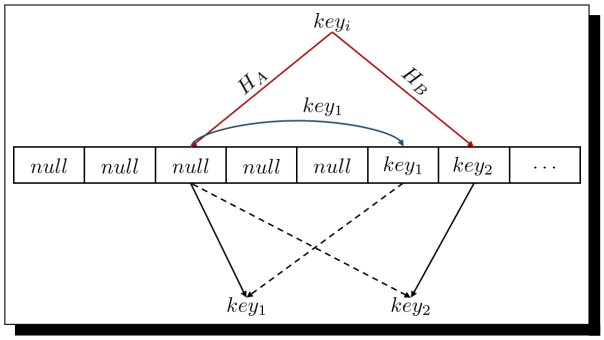

图 5–27 踢出 key 值

（6）如果 key_1 的另一个哈希结果对应的位置也被占用，则继续踢出该位置上的 key 值，将 key_1 放置在此位置上，并将被踢出的 key 值放置在其对应的另一个哈希结果对应的位置上，如图5–28所示。反复执行这一过程，直到将相应的 key 值放置在为空的位置上，或者达到某个循环次数（门限值）时，结束此次布谷鸟哈希过程。

对于达到循环次数门限值的元素，布谷鸟哈希的不同变体有不同的处理方式，主要有如下两种处理方式：一是增加哈希函数的数量，进一步提高空间利用率；二是增加数组或列表（桶）的数量，每个哈希函数对应一个数组或列表（一个桶），每次选择多个

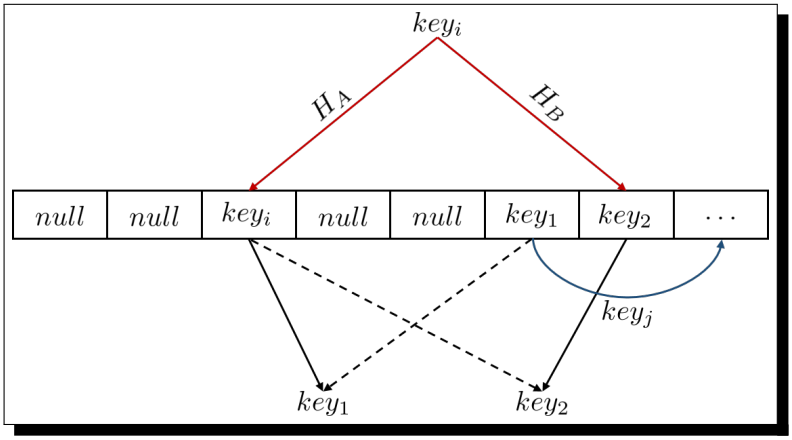

图 5–28 继续踢出 key 值

数组或列表中的空位置进行放置，这种根据 k 的大小来调节桶的个数的方法可以使得插入失败概率很低。

以增加桶的数量为例，此时的放置过程如下：

（1）当新插入一个值 key_i 时，若 H_A 在上面桶中的位置和 H_B 在下面桶中的位置分别被 key_1 和 key_x 占据，则任选一个 key 踢出（这里以踢出 key_1 为例，如图5–29所示）。

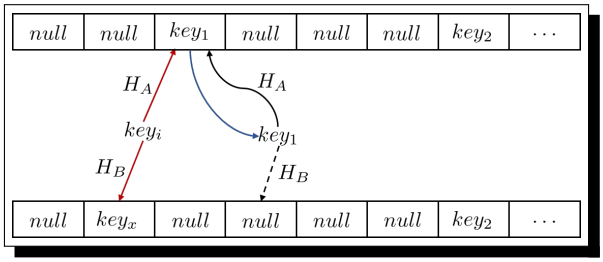

图 5–29 两个桶时

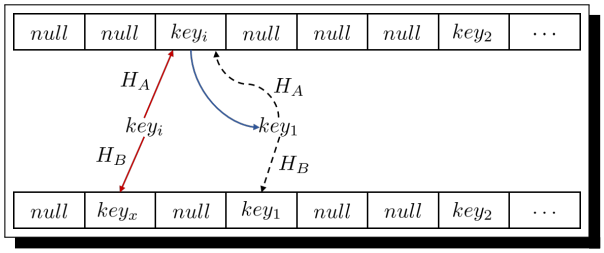

图 5–30 插入新 key 值

（2）计算 key_1 的 H_B 值，然后插入到下面的 H_B 对应的桶中，如图5–30所示。

在实际应用时，也可采用主副哈希表策略，即在主哈希表的基础上，增加若干个副哈希表，即使用额外的桶，当一个元素在主表中达到插入循环次数门限值时，将其插入到某个副表（额外增加的桶）中。这里指出，对于隐私集合求交（PSI）协议来说，使用桶会产生很多额外的开销。因此，在使用布谷鸟哈希算法构造 PSI 协议时，考虑到算法性能，一般不使用桶（stash-less）。

根据布谷鸟哈希算法的元素插入规则，可得到这样一个重要的性质：对于某个元素 $q \in X$，其所在布谷鸟哈希表上的位置必然是 $H_1(q), H_2(q), \cdots, H_k(q)$ 所映射到的位置中的一个。

5.12.3.2 布谷鸟哈希表

由上述的布谷鸟哈希算法将元素插入各个桶中后形成的哈希表被称为布谷鸟哈希表，按照上一节中所述的方法，其形成过程如图5–31所示。

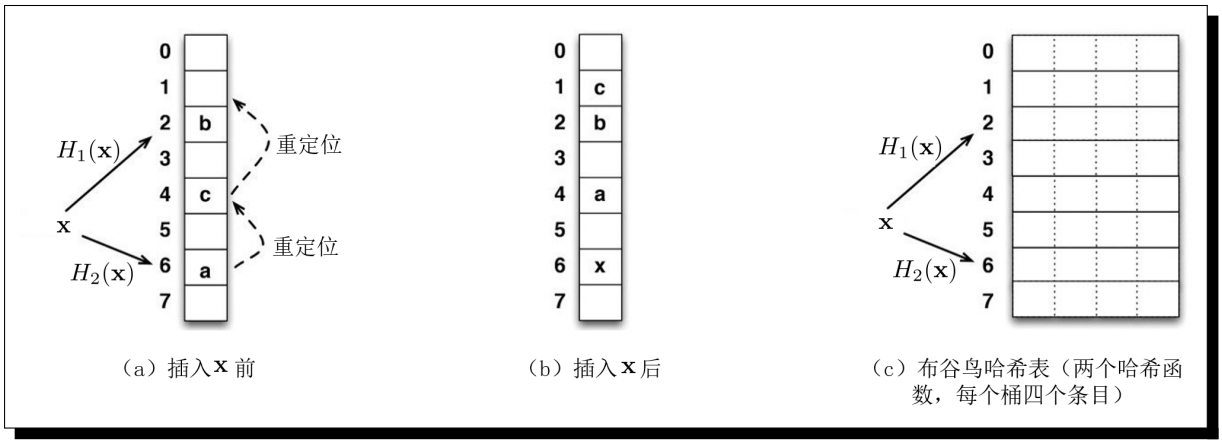

图 5–31 传统布谷鸟哈希表

图图5–31中的（a）和（b）展示了一个基本的布谷鸟哈希表的插入操作，是由一个桶数组组成，每个插入项都有由哈希函数 $H_1(x)$ 和 $H_2(x)$ 确定的两个候选位置。

布谷鸟哈希表由一个桶数组组成，其中一个桶可以有多个条目（比如上述图（c）中有四个条目）。而每个桶中有四个指纹（Fingerprint，这里是指使用一个哈希函数生成的 n 位比特位，n 的具体大小由所能接受的误判率来设置）位置，意味着一次哈希计算后布谷鸟有四个“巢”可用，而且四个巢的位置是连续的，可以更好的利用 CPU 进行高速缓存。

除了传统布谷鸟哈希表，还有一种被称为混淆布谷鸟表（Garbled Cuckoo Table）的布谷鸟哈希表 [28]。混淆布谷鸟表是对布谷鸟哈希表的概念的拓展。

5.12.3.3 基本的布谷鸟过滤器

2014 年，CMU 大学的 Bin Fan 等人基于布谷鸟哈希算法提出了布谷鸟过滤器 [29]，基本的布谷鸟过滤器是由两个或者多个哈希函数构成的。布谷鸟过滤器的布谷鸟哈希表的基本单位称为条目（entry）。每个条目存储一个指纹。

（1）元素插入：布谷鸟过滤器的插入是重点，与朴素的布谷鸟哈希不同，布谷鸟过滤器采取了两个并不独立的哈希函数，具体来说，

$$\begin{aligned} i_1 &= \text{hash}(x), \\ i_2 &= i_1 \oplus \text{hash}(f). \end{aligned} \tag{5.25}$$

i_1、i_2 即是计算出来的两个桶上的索引，其中第一个桶上的索引是通过某个哈希函数计算出来的，第二个是使用第一个的索引和指纹的哈希进行异或操作后得到的。根据异或操作的特性：同为 0，不同为 1，且 0 和任何数异或是这个数本身，那么 i_1 也可以通过 i_2 和指纹的哈希值进行异或来计算。换句话说，当需要从桶中迁走一个键（即某条目上存储的指纹）时，可以直接用当前桶上的索引 i 和存储在桶中的指纹的哈希值计算它在备用桶上的位置。

指纹 f 是通过哈希函数取一定量的比特位得到的，即 $f = \text{fingerprint}(x)$。为什么不直接用 i_1 和指纹做异或操作呢？因为指纹一般只是 key 映射出来的少量 bit 位，当指纹的比特位与整个桶数组相比很小时，如果不进行哈希操作，而是使用“$i\oplus$ 指纹”来计算备用位置，元素将被放置到离桶上索引 i_1 很近的位置，比如使用八位的指纹大小，最多只能改变 i_1 的低八位，所以也就是两个候选通的位置最多相差 256，不利于均匀分配桶上的位置。

（2）元素查找：布谷鸟过滤器的查找过程很简单，给定一个元素 x，算法首先根据上述插入公式，计算 x 的指纹和两个候选桶上的索引。然后读取这两个桶，如果两个桶中存储的现有指纹均匹配，则布谷鸟过滤器返回 True，否则过滤器返回 False。此时，只要不发生桶溢出，就可以确保没有假阴性（即元素的指纹与存储在桶上的索引位置上的指纹匹配，但返回的是 False）。

（3）元素删除：标准的布隆过滤器不支持删除操作，因此删除单个元素需要重建整个过滤器，而计数布隆过滤器（参见5.12.3.4节）需要更多的空间。布谷鸟过滤器和计数布隆过滤器类似，可以通过从哈希表中删除相应的指纹来删除插入的元素，而其他具有类似删除过程的过滤器比布谷鸟过滤器更为复杂。

具体删除的过程也很简单：检查给定元素在两个候选桶上的位置，如果与任何一个桶中的指纹相匹配，则从该桶中删除匹配指纹的一份副本。

布谷鸟过滤器用更低的空间开销解决了布隆过滤器不能删除元素的问题，达到了更好的效果。具体来说，布谷鸟过滤器的优势包括：

- 支持动态添加和删除元素；
- 即使在桶数组接近已满（比如空间利用率达到 95% 时）的情况下，也能提供比布隆过滤器更高的查找性能；
- 比布隆过滤器更容易实现；
- 在要求满足的误判率低于 3% 的情况下，比布隆过滤器的空间开销更低。

为了达到上述效果，布谷鸟过滤器对布谷鸟哈希算法进行了如下改变：

（1）为了提高桶的利用率，使用多路哈希桶；

（2）为了减少内存的使用，只存储 key 的指纹（哈希值）。

布谷鸟过滤器的缺点包括：

- 删除不完美，存在误删的可能性。删除的时候，只是删除了一份指纹副本，并不能确定此指纹副本是要删除的 key 的指纹。同时，这也导致了假阳性（False Positive，FP，即过滤器判定某元素存在于数据集合中，但实际上并不存在）的情况。
- 插入复杂度比较高。随着插入元素的增多，复杂度会越来越高，因为存在桶满、踢出的操作，所以需要重新计算，但综合来讲复杂度还是常数级别。
- 存储空间的大小必须为 2 的指数，这一限制影响了空间利用率。
- 同一个元素最多插入 kb 次，这里，k 指哈希函数的个数，b 指的是桶中能装入的指纹的个数，也就是桶的大小。如果布谷鸟过滤器支持删除，则必须存储同一项的多个副本。插入同一项 $kb+1$ 次将导致插入失败。这类似于计数布隆过滤器，其中重复插入会导致计数器溢出。

除了上面介绍的几种常用的过滤器，还有计数布隆过滤器、动态计数过滤器等。

5.12.3.4 计数布隆过滤器

计数布隆过滤器（Counting Bloom Filter，CBF）是标准 BF 的一种扩展。标准 BF 使用一个比特数组（Bit Array），每个位置只能是 0 或 1。当插入元素时，通过多个哈希函数将对应的位置置为 1；查询时，如果所有哈希位置都为 1，则判定元素“可能存在”。因此，标准 BF 对于需要精确检测结果的场景将不再适用，而带计数器的布隆过滤器的出现解决了这个问题。CBF 的改进在于（如图5–32所示）：在标准 BF 的每一个位上都额外对应的增加了一个计数器（Counter），

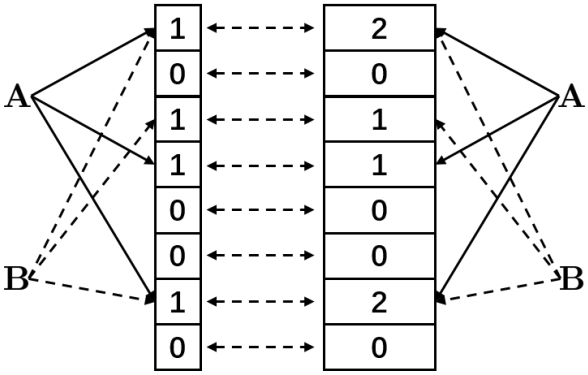

图 5–32 计数布隆过滤器生成过程

- 插入元素时，给对应的 k（k 为哈希函数的个数）个计数器的值分别加 1；
- 删除元素时给对应的 k 个计数器的值分别减 1；
- 查询时，检查元素哈希到的所有位置的计数器是否都 > 0。

CBF 的优点是：（1）支持删除，这使其适用于需要动态更新集合的场景；（2）相比维护一个完整的元素列表，CBF 依然保持了极高的空间效率。

CBF 的缺点包括：

（1）空间开销增大：CBF 的元素删除功能是以多占用几倍的存储空间为代价的。这

其中最关键的问题是 CBF 需要增加多少存储量。假设计数器数组的长度为 m（对应 BF 的位数组），C_i 表示计数器数组中第 i 个计数器的大小，即哈希函数映射到第 i 位的次数，则每个计数器最少位数 N 为

$$N = \sum_{i=1}^{m} [\log_2 C_i]. \tag{5.26}$$

CBF 的空间占用通常是标准 BF 的 4 ~ 8 倍。

（2）计数溢出（Counter Overflow）：如果计数器位数有限（如 4 位，最大值为 15），当插入操作过于频繁时，计数器可能溢出，导致数据错误。

（3）假阳性（False Positive）：与标准布隆过滤器一样，CBF 也存在假阳性概率（即误报），且概率通常略高于标准版。

作为 CBF 的一种实现，SBF（Spectral Bloom Filter，光谱布隆过滤器）将所有计数器排成一个位串（形似“光谱”），计数器之间完全不留空隙，然后通过建立索引结构来访问计数器，并达到了只使用 $O(N)+O(m)$ 位的存储目标，以及 $O(m)$ 的构建时间。虽然 SBF 解决了动态计数器的存储问题，但其引入了复杂的索引结构，这让对每个计数器的访问变得复杂而耗时。

CBF 主要用于需要动态维护集合且对空间敏感的场景。

- 网络路由与转发：在软件定义网络（SDN）或内容中心网络（CCN）中，用于快速判断数据包是否属于某个流（Flow）。当流结束时，可以从过滤器中删除对应的条目，释放资源。
- 分布式缓存系统：在 Memcached 或 Redis 集群中，用于判断数据是否在缓存中。当缓存项因过期或被淘汰而删除时，CBF 可以同步删除对应的指纹，避免缓存穿透。
- 垃圾邮件过滤：用于维护垃圾邮件发件人列表。当某个发件人从黑名单中移除时，可以从过滤器中删除，而无需重建整个过滤器。
- 数据库查询优化：在 LSM-Tree 结构的数据库（如 LevelDB、RocksDB）中，用于快速判断某个键是否存在于某个 SSTable 文件中。当文件被合并或删除时，对应的过滤器条目也需要被清理。

5.12.3.5 动态计数过滤器

为改进前述的 SBF 的缺陷（CBF 在动态数据集下空间效率低和重建开销大），研究人员又发明了动态计数过滤器（Dynamic Count Filter，DCF），其核心思想是采用了一种可动态调整大小的数据结构，能够根据数据集中元素数量的变化，灵活地增加或减少过滤器的容量，而无需完全重建。

具体来说，DCF 使用两个数组来存储所有的计数器，它们的长度都为 m（即 BF 的位数组长度）。第一个数组是一个基本的 CBF（即图5–33中的 CBFV，counting bloom filter vector，计数布隆过滤器向量），计数器的长度固定为 $x = \log(M/n)$，其中 M 是集合中所有元素的个数，n 为集合中不同元素的个数；第二个数组用来处理计数器的溢出（即图5–33中的 OFV，overflow vector，溢出向量），数组每一项的长度并不固定，根据计数器的溢出情况进行动态调整。

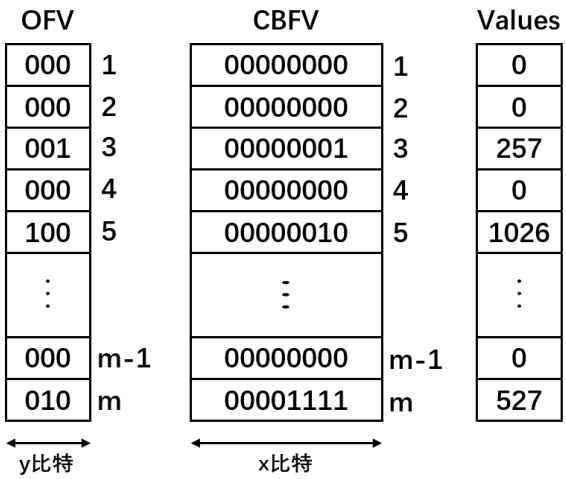

图 5–33 动态计数布隆过滤器生成过程

在查询一个计数器时，DCF 要求两次内存访问。假设想查询位置为 j 的计数器的值，先读出 CBFV 和 OFV 的值，分别记为 C_j 和 OF_j，那么该计数器的值可以表示为 $V_j = (2x \times OF_j + C_j)$。

在向集合中增加元素时，如果 OFV 的最大值从 $2x-1$ 增加到 $2x$，OFV 就需要给每一项增加 1 位，否则就会溢出。对应地，当 OFV 的最大值从 $2x$ 减少到 $2x-1$ 时，OFV 就需要减少 1 位。每次 OFV 大小改变的时候都需要重新创建一个 OFV 数组，然后把旧 OFV 数组的值拷贝到新建的 OFV 数组中，最后把旧 OFV 数组的空间释放掉。对于减少的情况，可以采用一些策略延迟 OFV 的重建，以避免一些临时性的减少导致 OFV 反复重建。

相比静态的 CBF，DCF 通过避免为可能出现的最大数据量预先分配固定大小的计数器数组，显著提高了内存使用效率，并能更好地应对数据流中元素数量的突发性增长。应用场景方面，DCF 主要用于数据流处理中的重复项检测和频繁项挖掘等任务。

5.13 可验证随机函数（VRF）

可验证随机函数（Verifiable Random Function，VRF）是一种加密函数，由 Micali、Rabin 和 Vadhan [30] 首先提出，它可以将输入映射为可验证的伪随机输出。具体来说，VRF 生成一个随机数，并且包含生成者的私钥签名，因此验证方可以通过公钥确定随机数的合法性。VRF 具有可验证性、唯一性和随机性等特性。

为便于理解，可验证随机函数可以表达为 $(r, \text{proof}) \leftarrow \text{VRF}(\text{sk}, \text{seed})$，以私钥 sk 和随机种子 seed 作为输入，输出伪随机数 r 和该伪随机数的证明 proof，其要实现的目标

是：任何人都可以使用 sk 对应的公钥 pk 和证明 proof 来验证伪随机数 r 是否是由私钥 sk 的持有者算出来的，并且伪随机数、私钥和随机种子都是唯一确定的，任何人都不能根据随机种子和伪随机数推导出私钥。

VRF 的具体操作流程如图5–34所示。

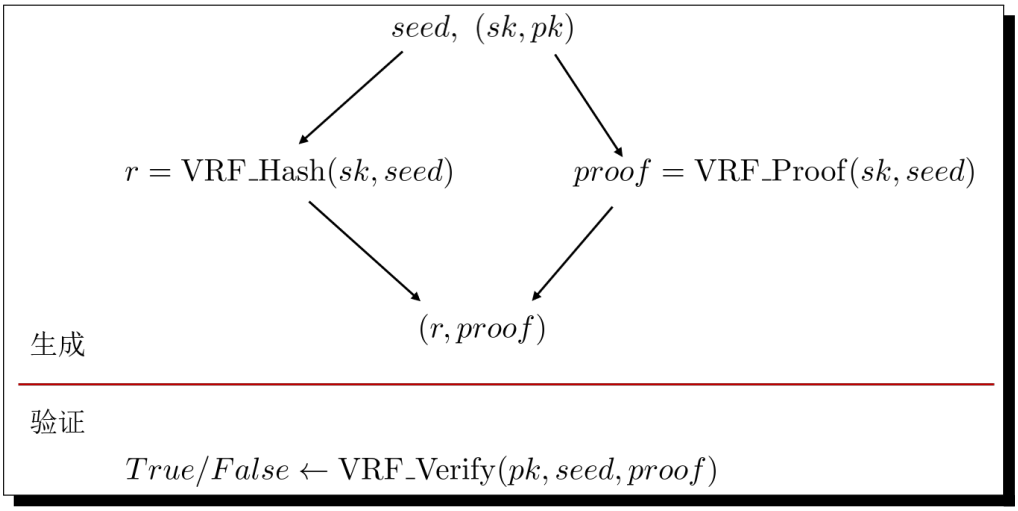

图 5–34 VRF 的具体工作流程

结合图5–34，VRF 的工作流程可以表述为：

（1）证明方生成一个公私钥对 (sk, pk) 和一个随机种子 seed。
（2）证明方计算 r = VRF_Hash(sk, seed) 和 proof = VRF_Proof(sk, seed)。
（3）证明方把 seed、proof 和 pk 发送给验证方。
（4）验证方计算 True/False ← VRF_Verify(pk, seed, proof)，True 表示验证通过，False 表示验证不通过。

可验证随机函数的关键性质包括：

- 可验证性：任何人都可以验证 VRF 生成的随机数是否有效，只需要查看证明并验证哈希计算结果是否正确即可。只有持有 VRF 私钥的人才能进行哈希运算，而任何持有公钥的人都可以验证哈希运算结果。
- 随机性：对于不知道 seed 或不持有私钥的人来说，VRF 输出的结果完全不可预测且概率均匀分布，没有任何规律可循。VRF 输出的每个结果都拥有同样的概率，并使用 seed 和私钥以独特的方式生成随机数。
- 唯一性：VRF 使用数学算法来生成唯一的随机数以及证明。VRF 将 seed 隐藏起来，以保障其不可预测性；同时，将随机数证明公开，以供所有人查看和验证。

5.14 格密码学基础

格密码学是一种基于格理论的公钥密码系统，它利用了格的困难问题来实现加密和解密的操作。格密码学的研究涉及到的密码数学问题很多，学科交叉特色明显，研究方法趋于多元化，是当前密码学的热门研究方向。虽然格论和格密码学是不同的领域，但它们之间存在一定的联系。格密码学利用了格论中的一些概念和性质，如格的代数结构、格的子空间等，来设计安全的密码系统。同时，格密码学的发展也推动了格论的研究，促进了数学学科的发展。

在过去的 20 年中，格密码学已经成为密码学领域中的一个非常吸引人的研究方向。基于格的密码学原语之所以吸引人，是因为其安全性通常可以基于最坏情况下的难度（或困难性）假设（Worst-case Hardness Assumptions），而且它们似乎即使在量子计算上也能够保证安全性。

5.14.1 什么是格

格（lattice）是一种数学结构，是指一组线性无关的非 0 向量（称为格基）的整系数线性组合。给定 n 个线性无关的向量 $\mathbf{x}_1, \cdots, \mathbf{x}_n \in \mathbb{R}^m$，对于任意一组整数 $c_1, \cdots, c_n$，由它们生成的格被定义为：$\mathcal{L}(\mathbf{x}_1, \cdots, \mathbf{x}_n) = \{\sum_i c_i \mathbf{x}_i | c_i \in \mathbb{Z}\}$，$c_1\mathbf{x}_1 + \cdots c_n\mathbf{x}_n$ 都是属于这个格的向量，$\mathbf{x}_1, \cdots, \mathbf{x}_n$ 即为格基，n 称为格的维数。

一个格的格基可以不是唯一的。即使是同样的格的两组格基，其长度也可能相差很大。数学家和密码学家们普遍认为：对于一个维数足够高的格，通过一组随机选取的格基找到一组短格基，或得到一组线性无关的短格向量是困难的。这个问题称作“最短独立向量问题”（Shortest Independent Vector Problem，SIVP）。除此之外，还有一些其它的基于格的困难问题，如 gap-SVP、BDD 等。

5.14.2 格密码学简介

在密码学的实际应用中，格密码算法所基于的困难问题多采用容错学习 LWE（Learning with Error，容错学习）问题。LWE 问题与 SIS（Short Integer Solution，小整数方案）问题密切相关，因此，我们首先看一下 SIS 问题。SIS 问题是指：给定由多个随机并独立的 $\mathbf{a}_i \in \mathbb{Z}_q^n$ 组成的多项式，可以找到它们中的一个“小”整数 $\mathbf{x}$，使得他们的和为 $0 \in \mathbb{Z}_q^n$，其中，n 为整数维数（integer dimension），$n \geq 1$，q 为整数模数（integer modulus），$q \geq 2$，通常取为较大的质数。

LWE 可以简洁地描述为区分“有噪声线性方程”和真正随机方程的任务。LWE 问题是指：给定一个秘密向量 $\mathbf{s} \in \mathbb{Z}_q^n$，作为一个均匀随机密钥，给定多组 $(\mathbf{a}_i, b_i \approx \langle \mathbf{a}_i, \mathbf{s} \rangle) \in$

$\mathbb{Z}_q^n \times \mathbb{Z}_q$，其中，每个 $\mathbf{a}_i \in \mathbb{Z}_q^n$ 是均匀随机并独立的向量，$\mathbf{A} = (a_i)$ 为给定的随机矩阵，$b_i = \langle \mathbf{a}_i, \mathbf{s} \rangle + e$，矩阵 $\mathbf{b} = (b_i)$，e 是从特定的高斯分布中选择的误差值（也称为“噪声”），$e \leftarrow \chi$，通过 $(\mathbf{a}_i, b_i)$ 推出 $\mathbf{s}$ 是困难的。我们称 $(\mathbf{A}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \mod p)$ 为 LWE 样本，$\mathbf{s}$ 为 LWE 秘密。

LWE 的目标是将 $(\mathbf{a}_i, b_i \approx \langle \mathbf{a}_i, \mathbf{s} \rangle) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$ 形式的多项式与均匀随机并独立的多项式区分开来，每个内积 $\langle \mathbf{a}_i, \mathbf{s} \rangle \in \mathbb{Z}_q$ 被一个相对集中在 $0 \pmod q$ 附近的新的随机误差项所扰动。容易看到，如果不存在误差项 $\mathbf{e}$，这一问题即为求解线性方程组，可通过高斯消元法轻易解决。然而，当引入误差（噪声）$\mathbf{e}$ 之后，问题的复杂度骤然增加，LWE 问题可归约到 SIVP 等格上的困难问题，即求解 LWE 问题的难度不低于求解格上的困难问题，这使得 LWE 问题成为构建安全密码学系统的重要基础。

近年来，围绕 SIS 和 LWE 问题提出了多种密码方案。在应用于密码算法时，LWE 问题存在一个很大的优势：存在“最坏情况到平均情况的归约”，即求解平均情况下的 LWE 问题，其难度不低于最难的 SIVP 问题实例。对于一些早期的公钥密码算法，当参数选取不恰当时，密码算法易受攻击。而对于基于 LWE 问题的格密码来说，由于存在最坏情况到平均情况的归约，因而可以避免这种攻击的产生。这为基于 LWE 问题设计的密码算法带来了很大安全性上的优势。

直接通过 LWE 问题构造的密码学方案效率并不是很高。即使是最简单的原语，例如简单的单向函数，在主要安全参数中也至少具有二次方的密钥大小。更多的时候，我们需要将整数向量用多项式代替，得到多项式 LWE 或环 LWE（Ring-LWE, Ring Learning with Error，简称 RLWE）。

为了更容易理解 RLWE 问题，首先介绍需要用到的代数知识。设 $f(x) = x^\lambda + 1 \in \mathbb{Z}[x]$，其中，系统安全参数 λ 是 2 的幂，$f(x)$ 在有理数上不可约。用 $R = \mathbb{Z}[x]/\langle f(x) \rangle$ 表示一个整数系数模 $f(x)$ 的多项式环。R 中的元素通常用阶小于 λ 的整数多项式表示，比如，$R = \mathbb{Z}[x]\langle \Phi_m(x) \rangle$，此时，$R$ 也被称为“m 次分圆多项式（cyclotomic polynomial）的代数整数环”，其中，第 m 个分圆多项式 $\Phi_m(x) = \prod_{i \in Z_m^*}(x - \omega_m^i) \in \mathbb{Z}[x]$，$\omega_m = \exp(2\pi/m) \in \mathbb{C}$ 是 m 次的首一（principal）单位复根，$\omega_m^i = \exp(2\pi\sqrt{-1}/m) \in \mathbb{C}$ 在所有的第 m 个本原（primitive）单位复根上进行迭代。

比如，当 $m \geq 2$ 且 m 是 2 的幂时，分圆多项式可表示为 $\Phi_m(x) = x^{m/2} + 1$。注意，$\Phi_m(x)$ 的阶等于 $\phi(m)$，这使得 R 成为阶为 $N = \phi(m)$ 的域扩展。定义 $R_q = R/qR \cong R/\langle (\phi_m(x), q) \rangle$，其中的 q 不一定为质数。约等号后面的部分将用作密文的模数（modulus）。

在上述环中，RLWE 问题可以描述为：设 $s = s(x) \in R_q$ 是一个均匀随机的环元素，它是需要保密的。给定 $(a, b \approx a \cdot s) \in R_q \times R_q$，其中 $a \in R_q$ 是均匀随机的多项式，

$b = as + e$，e 是从特定的随机分布中选择误差值，$e \in \sigma^{\lambda}$，通过 (a, b) 推出 s 是困难的。RLWE 的样本为 $(a, b = as + e \mod q)$，其中 a、s 和 e 均为多项式。

类似于标准的 LWE，攻击者的目标是区分任意多个（独立的）“随机噪声环多项式”和真正均匀的环多项式。更具体地说，有噪声多项式是 $(a, b \approx a \cdot s) \in R_q^2$，乘积 $a \cdot s$ 被一些“小”的随机误差项所扰动，这些误差项是从 R 上的某一分布中选择的。

环 LWE 的安全性建立在理想格中相应数学问题困难性的基础之上。尽管这些问题在困难性上面被认为不如格问题更可靠，但目前还没有发现可以有效求解这些问题的算法。此外，还有可靠性介于 LWE 和环 LWE 问题之间的模 LWE 问题，以及这些问题的变种 LWR、环 LWR、模 LWR 问题。格密码的安全性基本上都依赖于这些问题的困难性。

5.14.3 NTRU 公钥加密算法

对于公钥密码算法的替代性问题，主要有以下技术解决方案：NTRU（Number Theory Research Unit，数论研究单元）公钥密码体制、McEliece 公钥密码体制、MQ 公钥密码体制。McEliece 公钥密码体制的安全性基于纠错码问题，安全性强，但计算效率低。MQ 公钥密码体制，即多变元二次多项式公钥密码体制（Multivariate Quadratic Polynomials in Public Key Cryptosystem），基于有限域上的多变元二次多项式方程组的难解性，在安全性方面的缺点比较明显。相比之下，NTRU 公钥加密体制具有算法简洁、计算速度快、占用存储空间小的优势。

NTRU 算法是 1996 年由美国布朗大学三位数学教授提出的公钥加密体制，其密钥生成方法相对容易，且加密、解密的速度比 RSA 等著名算法快得多。NTRU 是一个带有专利保护的公钥加密系统[1]，使用基于格的加密算法来加密数据。它主要包括两部分算法：NTRUEncrypt 用于加密，NTRUSign 用于数字签名。

NTRU 算法是基于格和多项式环的密码体制，其主要思想包括：选择一个合适的格结构，如循环格、椭圆格等；生成一个随机数矩阵作为加密密钥；将对称加密算法（如 AES）的密文转换为矩阵形式，并与其他矩阵相乘得到加密结果；解密过程为加密过程的逆运算。最初的 NTRU 方案 [31] 可简述如下：

（1）生成公共参数：选择一个次数 n 为素数的多项式 $f(x) = x^n - 1$。也可以选择多项式 $f(x) = x^n + 1$，但此时要求次数 n 为 2 的幂次（a power of two）。基于该多项式可形成如下多项式环（polynomial ring）：$R = \mathbb{Z}[x]/f(x)$，这是一个卷积多项式环。选择一个足够大的素数 q，于是可得到如下商环（quotient ring）：$R_q = R/qR$。

[1]这是该算法尚未得到普遍应用的主要原因之一。

（2）生成公钥：从多项式环 R 中选择两个环元素（多项式）$g, s \in R$，要求这两个环元素的系数相对较小，接着计算 $h = 2g \cdot s^{-1} \in R_q$，$h$ 即为公钥。
（3）生成私钥：多项式 s 即为私钥，要求 s 在模 2 和模 q 的情况下可逆。显然，已知两个多项式相乘的结果，推导任何一个多项式都是困难的，即已知公钥推导私钥是困难的。
（4）加密过程：将需要加密的明文编程进一个短误差项（Short error term）$e \in R$，然后从多项式环 R 中随机选择一个元素 $r \in R$，计算密文 $c = h \cdot r + e \in R_q$。
（5）解密过程：计算 $c \cdot s = 2g \cdot r + e \cdot s \in R_q$，接着对该结果进行模 2 运算，便可以得到 $e \cdot s$，其中 s 为私钥，是已知的，接着便可以恢复出 e，也就是恢复出明文比特。

Hoffstein 和 Silverman 对最初的 NTRU 算法进行了优化 [32]。文献 [33] 中探讨了 NTRU 算法的安全性问题，并使其与理想格上的最坏情况问题一样安全。

NTRU 算法在密码学领域具有广泛应用，主要用于保护数据的机密性，如电子商务、通信、嵌入式等领域。与其他流行的公钥加密系统不同，NTRU 算法的安全性依赖于格中最短向量问题（SVP），可防止被 Shor 算法破解，从而显著提升性能。此外，NTRU 算法还具有抗量子计算攻击的能力[1]，而 RSA 和 ECC 算法是无法抵抗量子计算的。

5.14.4 格密码的独特优势

基于格论构建的密码系统，具备一些传统密码学（如 RSA、ECC）难以比拟的优势，其中最引人注目的就是抗量子计算攻击的能力。目前主流公钥密码所依赖的大数分解、离散对数等问题，在量子计算机的 Shor 算法面前显得非常脆弱。而格密码依赖的 SVP/CVP 问题，目前尚未发现有效的量子算法能够破解，因此被公认为后量子密码的主要候选者。

此外，格密码通常具有运算效率高（基于线性运算）、功能强大（可支持全同态加密等高级功能）等优点。

5.15 后量子密码算法

MPC 协议基于传统密码体制构建。传统公钥加密算法使用经典计算机在多项式时间内无法破解的已证明困难问题作为安全基础，与此类似地，当代密码学探索量子计算机无法有效求解的困难问题作为安全基础来设计密码体制，这些密码体制被称为后

[1]这一点还需要学界进行进一步验证，但目前仍未找到能够破解该算法的弱点。

量子密码（Post-Quantum Cryptography，POC）或抗量子计算密码（Quantum-Resistant Cryptography，QRC）。顾名思义，后量子密码算法是指能够抵御未来大规模量子计算机攻击的密码学算法。它并非直接利用量子特性，而是基于传统计算机平台，依赖那些目前被认为量子计算机也无法高效解决的数学难题。

后量子密码是我们迈向未来数字安全的关键一步。它并非单一技术，而是一个多元发展的算法生态，旨在构建能够抵御量子计算冲击的新一代安全防线。

5.15.1 诞生的背景：量子计算的威胁

当前我们广泛使用的公钥密码体系（如 RSA、ECC 等）的安全性，依赖于大整数分解或离散对数等数学问题的计算复杂性。然而，量子计算机搭载的 Shor 算法能将这些难题在多项式时间内破解，从而构成根本性威胁。一旦大规模量子计算机成为现实，现有的网络安全基石将面临崩塌的风险。后量子密码正是在这一背景下，为应对“量子世界”的挑战而展开的研究。

5.15.2 主要算法与标准化

当前，设计后量子密码的技术路线主要有如下 5 种：

- 基于格的后量子密码算法：安全性基于格上最短向量问题等数学难题，其核心特点是结构灵活，效率较高，适用于加密和签名。代表性算法包括 ML-KEM (CRYSTALS-Kyber)、ML-DSA、FALCON 等。
- 基于编码的后量子密码算法：安全性基于一般线性码的解码问题，核心特点是研究历史久，公钥尺寸大。代表性算法包括 Classic McEliece 公钥加密算法等。
- 基于多变量的后量子密码算法：安全性基于有限域上非线性方程组的求解困难性，核心特点是签名速度快，但公钥大。代表性算法包括 GeMSS、Rainbow 签名算法。
- 基于哈希函数的数字签名算法：基于哈希函数的抗碰撞性，核心特点是方案简单，安全性理解直观，但签名较大。代表性算法包括 XMSS（eXtended Merkle Signature Scheme，可扩展 Merkle 签名方案）和 SLH-DSA (SPHINCS+) 等。
- 基于曲线同源的后量子密码算法。

其中，基于格的密码学因其在安全性和效率之间的良好平衡，被认为是前景最广阔的方向之一。

为了推动后量子密码的实际应用，美国国家标准与技术研究院（NIST）自 2016 年起主导了后量子密码算法的标准化工作。

- 已标准化算法：2022 年至 2024 年间，NIST 已公布首批标准化算法，包括基于格的密钥封装机制 ML-KEM（原 CRYSTALS-Kyber）和数字签名算法 ML-DSA（原 CRYSTALS-Dilithium），以及基于哈希的数字签名算法 SLH-DSA（原 SPHINCS+）。这意味着这些算法将作为推荐标准在全球范围内推广。
- 产业应用：这些标准化算法正被快速集成到主流产品中。例如，微软已在 Windows 11 中引入后量子算法 API，谷歌 Chrome 浏览器也默认启用了支持 ML-KEM 的混合加密机制。

5.15.3 当前的挑战与发展

尽管后量子密码前景广阔，但其发展仍面临一些挑战：

- 算法成熟度：许多算法仍处于研究和完善阶段，其长期安全性需要更多时间和实践来检验。
- 性能与兼容性：部分后量子密码算法在密钥尺寸、签名长度或计算开销上可能大于传统算法，与现有系统的集成和迁移是一项复杂且漫长的工程。专家估计，全面迁移可能需要 10-15 年时间。
- 持续的安全分析：新的密码分析攻击（包括侧信道攻击和新型量子算法等）可能不断出现，算法需要持续评估和更新。

5.16 开源密码学库

密码学领域有很多开源的密码学库，一些值得学习的开源密码学库的链接及简介如下：

- Bcrypt: 一个跨平台的文件加密工具，加密文件可以移植到所有可支持的操作系统和处理器中。
- BeeCrypt: 快速的加密图形库，功能强大，接口方便。
- Botan: C++ 加密库。
- Crypto++: 一个有关加密方案的免费的 C++ 库。
- GnuPG: OpenPGP（广泛采用的邮件加密标准）标准的完整实现。
- GnuTLS: 实现了 SSL，TLS 和 DTLS 协议的安全通信库。尽管目前仍有系统使用 SSL，但是由于 SSL 的所有版本都已被攻破，所以应尽量使用 TLS。
- Libgcrypt: 基于 GnuPG 的加密图形库。
- Libmcrypt: 线程安全库，提供统一的 API 以访问多个块和流加密算法。
- LibreSSL: 免费的 SSL/TLS 协议，属于 OpenSSL 的一个分支。

- LibTomCrypt: 一个非常全面的，模块化的，可移植的加密工具。
- libsodium: 基于 NaCI 的加密库，容易使用，主要用于加密、解密、签名和生成密码哈希等。
- Nettle: 底层的（low-level）加密库。
- OpenSSL: 一个强大的、商用的、功能齐全的开放源代码的加密库。
- PyNaCl：libsodium 库的 Python 实现，提供数字签名、密钥加密、公钥加密、哈希和消息身份验证、基于密码的密钥派生和密码散列功能。
- Cryptography：包括 The hazardous materials layer 和 The recipes layer 两个部分。The hazardous materials（低级密码学原语）提供了基本的密码概念，如加密、签名、Hash 等算法，但是直接使用密码学原语容易出错，所以 Cryptography 库对密码学原语进行了安全集成，形成了更高层次的 The recipes layer（密码学菜谱）。通过其集成的对称密码函数 Fernet，可以轻松实现对称加密。
- PyCryptodome:低级加密原语的自包含 Python 包,是对 PyCrypto 的替代(PyCrypto 已不再维护)。
- Passlib: 基于 Python2&3 的密码散列库，提供了 30 多种密码散列算法的跨平台实现，以及管理现有密码散列的框架。

参考文献

[1] Richard E. Blahut. 现代密码学及其应用（Cryptography and Secure Communication）. 机械工业出版社，2018.4

[2] Rivest R L, Shamir A, Adleman L M. On digital signatures and public-key cryptosystems[M]. Massachusetts Institute of Technology, Laboratory for Computer Science, 1977.

[3] Gamal T E. A public key cryptosystem and a signature scheme based on discrete logarithms[J]. IEEE Transactions on Information Theory, 1985, 31: 469-472.

[4] Okamoto T, Uchiyama S. A new public-key cryptosystem as secure as factoring[C]//International conference on the theory and applications of cryptographic techniques. Berlin, Heidelberg: Springer Berlin Heidelberg, 1998: 308-318.

[5] Paillier P. Public-key cryptosystems based on composite degree residuosity classes[C]//International conference on the theory and applications of cryptographic techniques. Springer, Berlin, Heidelberg, 1999: 223-238.

[6] 结城浩，《图解密码技术（第 3 版）》，周自恒译，人民邮电出版社，2016.6.

[7] 程朝辉. 数字签名技术概览 [J]. 信息安全与通信保密, 2020(07): 48-62.

[8] Chaum D. Blind signatures for untraceable payments[C]//Advances in Cryptology: Proceedings of Crypto 82. Boston, MA: Springer US, 1983: 199-203.

[9] Okamoto T. Provable secure and practical identification schemes and corresponding digital signature schemes[C]//Crypto. 92: 3-52.

[10] Nyberg K, Rueppel R A. A new signature scheme based on the DSA giving message recovery[C]//Proceedings of the 1st ACM Conference on Computer and Communications Security. 1993: 58-61.

[11] Camenisch J L, Piveteau J M, Stadler M A. Blind signatures based on the discrete logarithm problem[C]//Advances in Cryptology—EUROCRYPT'94: Workshop on the Theory and Application of Cryptographic Techniques Perugia, Italy, May 9–12, 1994 Proceedings 13. Springer Berlin Heidelberg, 1995: 428-432.

[12] Zhang F, Kim K. ID-based blind signature and ring signature from pairings[C]//Advances in Cryptology—ASIACRYPT 2002: 8th International Conference on the Theory and Application of Cryptology and Information Security Queenstown, New Zealand, December 1–5, 2002 Proceedings 8. Springer Berlin Heidelberg, 2002: 533-547.

[13] Boneh D, Lynn B, Shacham H. Short signatures from the Weil pairing[J]. Journal of cryptology, 2004, 17: 297-319.

[14] 张键红, 韦永壮, 王育民. 基于 RSA 的多重数字签名 [J]. 通信学报，2003, 24(8): 150-154.

[15] Bellare M, Neven G. Multi-signatures in the plain public-key model and a general forking lemma[C]//Proceedings of the 13th ACM conference on Computer and communications security. 2006: 390-399.

[16] Maxwell G, Poelstra A, Seurin Y, et al. Simple schnorr multi-signatures with applications to bitcoin[J]. Designs, Codes and Cryptography, 2019, 87(9): 2139-2164.

[17] Nick J, Ruffing T, Seurin Y, et al. MuSig-DN: Schnorr multi-signatures with verifiably deterministic nonces[C]//Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security. 2020: 1717-1731.

[18] Nick J, Ruffing T, Seurin Y. MuSig2: Simple two-round Schnorr multi-signatures[C]//Annual International Cryptology Conference. Cham: Springer International Publishing, 2021: 189-221.

[19] Boschini C, Takahashi A, Tibouchi M. MuSig-L: Lattice-based multi-signature with single-round online phase[C]//Annual International Cryptology Conference. Cham: Springer Nature Switzerland, 2022: 276-305.

[20] Chaum D, Van Heyst E. Group signatures[C]//Advances in Cryptology—EUROCRYPT' 91: Workshop on the Theory and Application of Cryptographic Techniques Brighton, UK, April 8–11, 1991 Proceedings 10. Springer Berlin Heidelberg, 1991: 257-265.

[21] Rivest R L, Shamir A, Tauman Y. How to leak a secret[C]//Advances in Cryptology—ASIACRYPT 2001: 7th International Conference on the Theory and Application of Cryptology and Information Security Gold Coast, Australia, December 9–13, 2001 Proceedings 7. Springer Berlin Heidelberg, 2001: 552-565.

[22] Camenisch J, Lysyanskaya A. Signature schemes and anonymous credentials from bilinear maps[C]//Annual international cryptology conference. Berlin, Heidelberg: Springer Berlin Heidelberg, 2004: 56-72.

[23] Boyle E, Couteau G, Gilboa N, et al. Compressing vector OLE[C]//Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security. 2018: 896-912.

[24] Bloom B H. Space/time trade-offs in hash coding with allowable errors[J]. Communications of the ACM, 1970, 13(7): 422-426.

[25] Dong C, Chen L, Wen Z. When private set intersection meets big data: an efficient and scalable protocol[C]//The 2013 ACM SIGSAC conference on Computer communications security. 2013: 789-800.

[26] Pagh R, Rodler F F. Cuckoo hashing[C]//European Symposium on Algorithms. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001: 121-133.

[27] Chandran N, Dasgupta N, Gupta D, et al. Efficient Linear Multiparty PSI and Extensions to Circuit/Quorum PSI[C]//Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security. 2021: 1182-1204.

[28] Pinkas B, Rosulek M, Trieu N, et al. PSI from PaXoS: fast, malicious private set intersection[C]//Annual International Conference on the Theory and Applications of Cryptographic Techniques. Cham: Springer International Publishing, 2020: 739-767.

[29] Fan B, Andersen D G, Kaminsky M, et al. Cuckoo filter: Practically better than bloom[C]//Proceedings of the 10th ACM International on Conference on emerging Networking Experiments and Technologies. 2014: 75-88.

[30] Micali S, Rabin M, Vadhan S. Verifiable random functions[C]//40th annual symposium on foundations of computer science (cat. No. 99CB37039). IEEE, 1999: 120-130.

[31] Hoffstein J, Pipher J, Silverman J H. NTRU: A ring-based public key cryptosystem[C]//International algorithmic number theory symposium. Berlin, Heidelberg: Springer Berlin Heidelberg, 1998: 267-288.

[32] Hoffstein J, Silverman J H. Optimizations for NTRU[C]//Proc. the Conf. on Public Key Cryptography and Computational Number Theory, Warsaw. 2000: 77-88.

[33] Stehlé D, Steinfeld R. Making NTRU as secure as worst-case problems over ideal lattices[C]//Advances in Cryptology–EUROCRYPT 2011: 30th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tallinn, Estonia, May 15-19, 2011. Proceedings 30. Springer Berlin Heidelberg, 2011: 27-47.